

AURIX

Microcontroller Infineon Software Architecture

User's Manual

PWM Driver

Release V5.3

2016-02

Edition 2016-02

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2017 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2017-02-08	5.3	Updated based on GTM code review finding that PWM init shall only be invoked after PLL is setup. Sections 6.2.1 and 10.1 are updated. Added section 9.3 - code listing for Init check (based on safety findings).
2016-05-04	5.2	Updated based on JIRA 0000051261-231 - Updated interface diagram for Pwm Driver in section 2.2. - Updated File structure in section 3.1 - Added configuration parameters in section 4.3 - Updated I/O mode in section 6.2
2015-10-28	5.1	Updated based on review comment REV_008894. Section 6.3 is updated
2015-10-27	5.0	Updated User mode and supervisory mode details in section 6.2. Updated limitation sections for better understanding of Notifications supported when SetOutputToldle function is called.
2015-04-30	4.9	Updated based on review comments REV_007628.Updated sections 10.1 (Assumptions), 4.7.1, 4.7.2, 6.1.9,10.2
2015-04-29	4.8	Updated based on JIRA 0000051124-49, Updated Interfaces to PWM driver (figure), sections 6.1.1.1, 10.2 (Limitations).Added sections 4.4.7, 4.7, 4.7.1, 4.7.2, 4.7.3, 6.1.9, 6.1.10, 6.2.10 based on JIRA 0000051124-61.
2015-03-10	4.7	PWM notification range modified based on JIRA task 0000051190-7. Section 4.5.1 updated
2015-01-22	4.6	AI00254582: Period limitation for ATOM in autosar mode updated.
2014-11-07	4.5	AI00251090: IO mode added for all API's.
2014-09-22	4.4	Added a note in section 2.1.2.4 for BC step design.
2014-08-28	4.3	Updated limitations for GTM errata GTM_AI.202.
2014-08-14	4.2	Added section on channel types, updated for design for shifted channels handled by offset, added configuration details for PwmEnable0Or100DutyNotification and PwmHandleShiftByOffset, updated limitations.
2014-07-14	4.1	Removed Safety information chapter and reference to Aurix_MC-ISAR_HE_SoftwareSafetyInformation.pdf
2014-05-30	4.0	Updated 2.1.2.3 for new design for shifted channels and updated limitations.
2014-03-26	3.9	Updated for TC27x CA step device. Updated section on expected values for set duty cycle and transition from 100% and 0% duty cycle.
2014-02-28	3.8	Updated for EP device.
2014-02-07	3.7	Updated based on review comments REV_005261
2014-02-02	3.6	Added a section on Design for Shifted channel with 100% duty and updated limitation based on UTP AI00237946, AI00237940,
2013-11-14	3.5	Removed limitation GTM errata GTM-IP-120, Updated condition of use, added SFR modified field to configuration.
2013-10-03	3.4	Updated based on UTP AI00233345
2013-08-25	3.3	Minor update. Updated ISR function and modified configuration parameter name PwmChannelType to PwmSafetySignal
2013-08-23	3.2	Updated for Safety, TC29x derivative
2013-07-10	3.1	Updated for 26x
2013-03-28	3.0	Review comments updated.
2013-03-22	2.3	Updated VendorApiInfix
2012-12-18	2.2	File structure updated
2012-08-02	2.1	Pwm module Deviation and Limitation are updated in Section 9.

CONFIDENTIAL

Date	Version	Change Description
2012-06-12	2.0	AS3.2.1 information added. Review comments updated.
2012-06-04	1.2	Reference files are corrected for Published Parameters
2012-05-16	1.1	PWM_FIXED_PERIOD_ZERO_SHIFTED pwm channel class reference removed.
2011-10-24	1.0	Updated review comments. Period range of the ATOM channel modified.
2011-10-18	0.2	Vendor ID added in the Pwm file name and API names
2011-03-08	0.1	Initial Version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents

Page

1	Introduction	12
1.1	Scope	12
1.2	Abbreviations.....	12
1.3	References	13
2	PWM Driver Overview	14
2.1	Software Hardware Mapping.....	14
2.1.1	GTM	15
2.1.2	PWM and TOM/ATOM Channel Resources	15
2.1.2.1	PWM Grouping.....	16
2.1.2.2	PWM Variable period channels.....	18
2.1.2.3	PWM Channel types	18
2.1.2.4	Design for Shifted channel (PwmHandleShiftByOffset = false)	18
2.1.2.5	Design for fixed period channel.....	20
2.1.2.6	Request for duty change in shifted channel.....	20
2.1.2.7	Design for Shifted channels handled by an offset.....	21
2.1.3	PORT	23
2.1.4	IRQ	23
2.1.5	SYSTEM CLOCK	23
2.2	Software Driver Description	23
3	File Structure	24
3.1	Driver File Structure	25
4	Configuration Documentation	26
4.1	Configuration Concept	26
4.1.1	Configuration Class.....	26
4.1.2	Configuration Variant	26
4.1.3	How to read the Configuration Class field.....	26
4.2	Configuration Parameter Variance.....	27
4.3	Container: PwmGeneral.....	27
4.3.1	PwmDevErrorDetect	27
4.3.2	PwmDutycycleUpdatedEndPeriod	27
4.3.3	PwmPeriodUpdatedEndPeriod	28
4.3.4	PwmNotificationSupported	29
4.3.5	PwmIndex.....	29
4.3.6	PwmChannelCoherentSelection	30
4.3.7	PwmPBFixedAddress	30
4.3.8	PwmDutyShiftInTicks	31
4.3.9	PwmEnable0Or100DutyNotification.....	31
4.3.10	PwmHandleShiftByOffset.....	32
4.3.11	PwmRunningInUser0Mode	32
4.3.12	PwmUserModeInitApiEnable	33
4.3.13	PwmUserModeDeInitApiEnable.....	33
4.3.14	PwmUserModeRuntimeApiEnable.....	34
4.4	Container: PwmConfigurationOfOptApiServices	34
4.4.1	PwmVersionInfoApi.....	34
4.4.2	PwmDeInitApi.....	35
4.4.3	PwmGetOutputStateApi	35
4.4.4	PwmSetDutyCycleApi	36
4.4.5	PwmSetOutputToldleApi.....	36
4.4.6	PwmSetPeriodAndDutyApi	36
4.4.7	PwmSyncGrpUpdateDutiesApi	37
4.5	Container: Pwm Channel Configuration.....	37
4.5.1	PwmNotification.....	37
4.5.2	PwmPeriodDefault	38
4.5.3	PwmDutycycleDefault	38
4.5.4	PwmShiftValue	39
4.5.5	PwmPolarity	40

Table of Contents	Page
4.5.6 PwmIdleState	40
4.5.7 PwmChannelClass	41
4.5.8 PwmReferenceChannel	41
4.5.9 PwmAssignedHwUnit	42
4.5.10 PwmChannelId	42
4.5.11 PwmCoherentUpdate	43
4.5.12 PwmSafetySignal	43
4.6 Container: PwmSafety	44
4.6.1 PwmSafetyEnable	44
4.7 Container: PwmSyncGroup	44
4.7.1 PWMDutycycleBufferMarker	45
4.7.2 PwmGroupChannels	45
4.7.3 PwmGroupId	45
4.8 Container: CommonPublishedInformation	46
4.8.1 ArMajorVersion	46
4.8.2 ArMinorVersion	46
4.8.3 ArPatchVersion	47
4.8.4 SwMajorVersion	47
4.8.5 SwMinorVersion	47
4.8.6 SwPatchVersion	48
4.8.7 ModuleId	48
4.8.8 VendorId	48
4.8.9 VendorApilInfix	49
4.8.10 Release	49
4.9 Derived Configuration Parameters	50
4.9.1 PwmMaxChannels	50
4.9.2 PwmMaxAsilChannels	50
4.9.3 PwmMaxQmChannels	51
5 Published Parameters	51
5.1 VendorID	51
5.2 ModuleID	51
5.3 InstanceID	52
5.4 ARReleaseMajorVersion	52
5.5 ARReleaseMinorVersion	52
5.6 ARReleaseRevisionVersion	52
5.7 SWMajorVersion	52
5.8 SWMinorVersion	53
5.9 SwPatchVersion	53
6 API Specification	53
6.1 API Type Definitions	53
6.1.1 Pwm_17_Gtm_ConfigType	53
6.1.2 Pwm_17_Gtm_ChannelConfigType	54
6.1.3 Pwm_17_Gtm_ChannelType	54
6.1.4 Pwm_17_Gtm_PeriodType	55
6.1.5 Pwm_17_Gtm_OutputStateType	55
6.1.6 Pwm_17_Gtm_EdgeNotificationType	55
6.1.7 Pwm_17_Gtm_ChannelClassType	56
6.1.8 Pwm_17_Gtm_NotifiPtrType	56
6.1.9 Pwm_17_Gtm_GroupConfigType	56
6.1.10 Pwm_17_Gtm_GrpChanConfigType	57
6.2 API Functions	57
6.2.1 Pwm_17_Gtm_Init	57
6.2.2 Pwm_17_Gtm_DeInit	58
6.2.3 Pwm_17_Gtm_SetDutyCycle	59
6.2.4 Pwm_17_Gtm_SetPeriodAndDuty	60
6.2.5 Pwm_17_Gtm_SetOutputToldle	62
6.2.6 Pwm_17_Gtm_GetOutputState	63

Table of Contents	Page
6.2.7 Pwm_17_Gtm_DisableNotification.....	64
6.2.8 Pwm_17_Gtm_EnableNotification	65
6.2.9 Pwm_17_Gtm_GetVersionInfo	66
6.2.10 Pwm_17_Gtm_SyncGrpUpdateDuties	67
6.3 Interrupt Handling.....	68
6.3.1 Pwm_17_Gtm_Isr	68
6.4 Exclusive Area (EA) Information	71
7 Data Description.....	71
8 Error Classification	73
9 Example Usage.....	73
9.1 Configuration of the Driver	73
9.1.1 Sample Configuration of the Driver	74
9.2 Driver Initialization	76
9.3 Driver Init check.....	76
9.4 API Usages	120
9.4.1 Set Duty Cycle.....	120
9.4.2 Set Period and Duty Cycle	120
9.4.3 Notification and other APIs.....	120
10 Limitations and Assumptions.....	121
10.1 Assumptions and Deviations from Software Specification.....	121
10.2 Limitations	122
10.3 Hardware features NOT Supported	124

List of Figures**Page**

Figure 1	Software Hardware mapping.....	15
Figure 2	GTM – Hardware Block Diagram	15
Figure 3	TOM/ATOM Channel	16
Figure 4	Shift Aligned PWM Waveform	17
Figure 5	Shifted Type	18
Figure 6	Center Aligned PWM type waveform	18
Figure 7	The transition from 100% to other percentages with a shift value	20
Figure 8	Interfaces of PWM Driver	24
Figure 9	File Structure	25

List of Tables

Page

Table 1	GTM Unit Used for PWM Driver	12
Table 2	Abbreviations.....	12
Table 3	Design for Fixed period and variable period channels.....	20
Table 4	Difference between the two types of handling for shifted channels	22
Table 5	File Contents	25
Table 6	Configuration Class Example 1.....	26
Table 7	Configuration Class Example 2.....	27
Table 8	PwmDevErrorDetect	27
Table 9	PwmDutycycleUpdatedEndPeriod	27
Table 10	PwmPeriodUpdatedEndPeriod	28
Table 11	PwmNotificationSupported.....	29
Table 12	PwmIndex.....	29
Table 13	PwmChannelCoherentSelection	30
Table 14	PwmPBFixedAddress	30
Table 15	PwmDutyShiftInTicks	31
Table 16	PwmEnable0Or100DutyNotification.....	31
Table 17	PwmHandleShiftByOffset.....	32
Table 18	PwmRunningInUser0Mode	32
Table 19	PwmUserModeInitApiEnable	33
Table 20	PwmUserModeDeInitApiEnable	33
Table 21	PwmUserModeRuntimeApiEnable	34
Table 22	PwmVersionInfoApi	34
Table 23	PwmDeInitApi.....	35
Table 24	PwmGetOutputStateApi	35
Table 25	PwmSetDutyCycleApi	36
Table 26	PwmSetOutputToldleApi.....	36
Table 27	PwmSetPeriodAndDutyApi	36
Table 28	PwmSyncGrpUpdateDutiesApi	37
Table 29	PwmNotification.....	37
Table 30	PwmPeriodDefault	38
Table 31	PwmDutycycleDefault	38
Table 32	PwmShiftValue	39
Table 33	PwmPolarity	40
Table 34	PwmIdleState	40
Table 35	PwmChannelClass	41
Table 36	PwmReferenceChannel	41
Table 37	PwmAssignedHwUnit.....	42
Table 38	PwmChannelId	42
Table 39	PwmCoherentUpdate.....	43
Table 40	Specification of PwmSafetySignal	43
Table 41	PwmSafetyEnable	44
Table 42	PWMDutycycleBufferMarker	45
Table 43	PwmGroupChannels	45
Table 44	PwmGroupId	45
Table 45	ArMajorVersion.....	46
Table 46	ArMinorVersion.....	46
Table 47	ArPatchVersion	47
Table 48	SwMajorVersion	47
Table 49	SwMinorVersion	47
Table 50	SwPatchVersion	48
Table 51	ModuleId.....	48
Table 52	VendorId.....	48
Table 53	VendorApiInfix.....	49
Table 54	Release	49
Table 55	PwmMaxChannels	50
Table 56	PwmMaxAsilChannels	50
Table 57	PwmMaxQmChannels	51
Table 58	VendorID	51

List of Tables

Page

Table 59	ModuleID	51
Table 60	InstanceID	52
Table 61	ARReleaseMajorVersion	52
Table 62	ARReleaseMinorVersion	52
Table 63	ARReleaseRevisionVersion	52
Table 64	SWMajorVersion	52
Table 65	SWMinorVersion	53
Table 66	SwPatchVersion	53
Table 67	Type Definition Pwm_17_Gtm_ConfigType	53
Table 68	Type Definition Pwm_17_Gtm_ChannelConfigType	54
Table 69	Pwm_17_Gtm_ChannelType	54
Table 70	Type Definition Pwm_17_Gtm_PeriodType	55
Table 71	Type Definition Pwm_17_Gtm_OutputStateType	55
Table 72	Pwm_17_Gtm_EdgeNotificationType	55
Table 73	Type Definition Pwm_17_Gtm_ChannelClassType	56
Table 74	Type Definition Pwm_17_Gtm_NotifiPtrType	56
Table 75	Type Definition Pwm_17_Gtm_GroupConfigType	56
Table 76	Type Definition Pwm_17_Gtm_GrpChanConfigType	57
Table 77	Specification of API Pwm_17_Gtm_Init	57
Table 78	Specification of API Pwm_17_Gtm_DeInit	58
Table 79	Specification of API Pwm_17_Gtm_SetDutyCycle	59
Table 80	Specification of API Pwm_17_Gtm_SetPeriodAndDuty	60
Table 81	Specification of API Pwm_17_Gtm_SetOutputTogle	62
Table 82	Specification of API Pwm_17_Gtm_GetOutputState	63
Table 83	Specification of API Pwm_17_Gtm_DisableNotification	64
Table 84	Specification of API Pwm_17_Gtm_EnableNotification	65
Table 85	Specification of API Pwm_17_Gtm_GetVersionInfo	66
Table 86	Specification of API Pwm_17_Gtm_SyncGrpUpdateDuties	67
Table 87	Specification of API Pwm_17_Gtm_Isr	68
Table 88	Service Request Node – Hardware Resource GTM	69
Table 89	Eg: Global Data Description	71
Table 90	Error Classification	73
Table 91	Safety Error Classification	73

List of Code Listings

Page

Code Listing 1	Pwm Configuration Header file.....	74
Code Listing 2	Pwm Channel Configuration.....	75
Code Listing 3	Driver Initialization	76
Code Listing 4	Driver Initialization	76
Code Listing 5	Global variable declaration.....	77
Code Listing 6	Macros used in code	78
Code Listing 7	Local function declarations.....	80
Code Listing 8	Functions reused from code.....	84
Code Listing 9	Local functions to check SFR values	94
Code Listing 10	Local functions to check Global variables	111
Code Listing 11	Driver Set Dutycycle	120
Code Listing 12	Driver Set Period and Duty Cycle	120
Code Listing 13	Driver Utility API's	120

1 Introduction

This user manual provides information regarding the functionality and API implementation of the PWM Driver for AUTOSAR[1]. This User Manual is intended to help the user to get acquainted with the PWM Driver implementation for the AURIX Hardware Platform. It is the documentation that describes how to use the PWM Driver.

The user manual covers the hardware features of the AURIX platform used in the PWM Driver implementation, Driver configuration and API implementation.

1.1 Scope

This document addresses the following features of the PWM Driver implementation,

- AURIX microcontroller HW peripheral used to realize the Driver
- File structure
- Configuration parameters of the Driver
- Parameters published by PWM Driver
- API specification
- Code Snippets illustrating the common use cases.
- Limitations

Note: This document addresses the PWM Driver implementation for the following variants of AURIX family: TC27x, TC26x, TC29x and TC23x.

Some of the peripheral units/features might not be available in all the variants. In such cases, the relevant information should be discarded for the variant where the unit/feature is not applicable. Similarly all the TC variants might not have the same number of TOM/ATOM channels in each GTM unit.

The following table gives the available units and available number of TOM/ATOM channels in each unit in the variants of AURIX family for PWM Driver usage.

Table 1 GTM Unit Used for PWM Driver

TC Variant	GTM sub module	Sub module units	Channels
TC27x/TC2Dx	TOM	3	16
	ATOM	5	8
TC26x	TOM	2	16
	ATOM	4	8
TC29x	TOM	5	16
	ATOM	9	8
TC23x	TOM	2	16
	ATOM	0	0

Note: All the references to ATOM in this document are not applicable to the EP devices.

This documentation applies to the PWM Driver Variant: Variant PB.

1.2 Abbreviations

Table 2 Abbreviations

Abbreviation	Explanation
μC	Micro Controller
ALT	Alternate Function
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
ATOM	ARU Timer Output Module
ARU	Advanced Routing Unit
BSW	Basic Software
DET	Development Error Tracer
DIO	Digital Input/Output
GPIO	General Purpose Input Output
GPT	General Purpose Timer
GTM	Generic Timer Module
GTC	Global Timer Cells
HW	Hardware
I/O	Input/Output
ICU	Input Capture Unit
ISR	Interrupt Service Routine
MCAL	MicroController Abstraction Layer
MCU	MicroController Unit
Mux/mux	Multiplexer
PLL	Digital Phase Locked Loop
RAM	Random Access Memory
ROM	Read Only Memory
SPI	Serial Peripheral Interface
SW	Software
TOM	Timer Output Module
EP	Efficiency Platform
HE	High End

1.3 References

This section lists down all relevant documents for this User Manual. Some of the artifacts listed below are given without the version number. For those artifacts, kindly refer the latest version.

- [1] Specification of PWM Driver AS4.0.3,
AUTOSAR_SWS_PWM_Driver.pdf, v2.5.0
- [2] Specification of PWM Driver AS3.2.1,
AUTOSAR_SWS_PWM_Driver.pdf, v2.3.0
- [3] User's Manual of Mcu Driver,
Aurix_MC-ISAR_UM_MCUDriver.pdf
- [4] User's Manual of Port Driver,
Aurix_MC-ISAR_UM_PORTDriver.pdf
- [5] User's Manual GTM Driver,

Aurix_MC-ISAR_UM_GTMDriver.pdf

[6] User's Manual IRQ Driver,

Aurix_MC-ISAR_UM_IRQDriver.pdf

[7] Target Data Sheet :

tc29x_um_V1.0.pdf, tc27x_um_V1.3.1.pdf, tc26x_um_v1.0_OPEN_MARKET.pdf,
tc23x_tc22x_um_v1.0.pdf, tc27xC_um_v1.5.pdf, tc27xD_um_v2.2.pdf

2 PWM Driver Overview

The PWM Driver uses the GTM peripheral of the AURIX platform to generate PWM signals. The Driver can be configured with multiple channels and has features to change the duty cycle, period during runtime. The Driver also supports notification to the upper layer upon configured edge.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the Aurix HW peripherals and their interaction with the Driver.

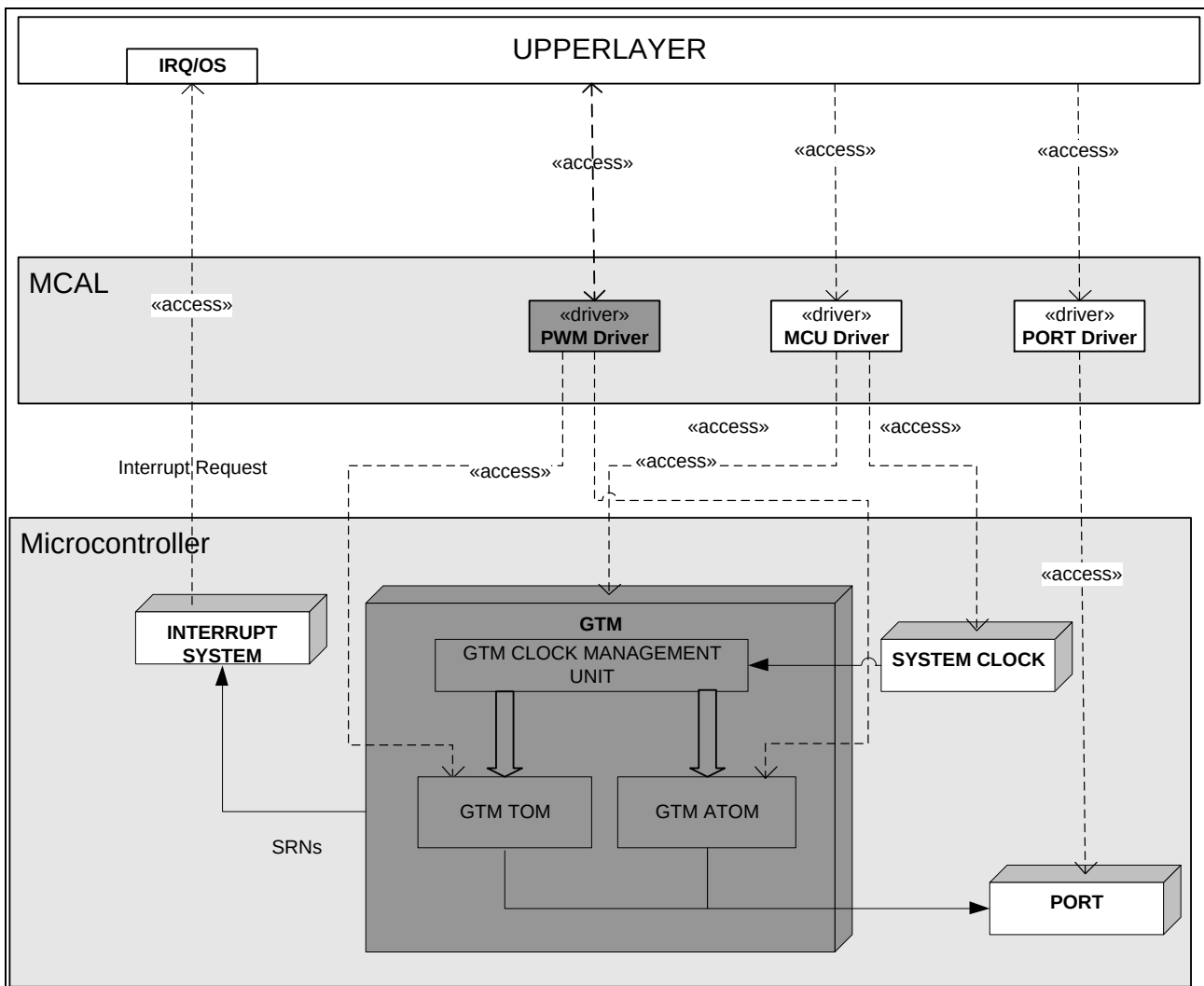


Figure 1 Software Hardware mapping

The different HW peripherals needed by the Driver and their purpose is explained below.

2.1.1 GTM

The GTM hardware block diagram illustrated in [Figure 2](#) is used as the basis for discussion of the hardware resources. The purpose of the chapter is only to focus on the resources used by the PWM Driver i.e. TOM/ATOM Channels.

The use of GTM is not limited only to the PWM Driver. Additionally ICU, GPT, WDG and ADC use the common resources of GTM. In order to initialize these common resources of GTM, the GTM Driver is needed. GTM Driver is also responsible for clock initialization and Input/Output MUX programming. The initialization of the common resources of the GTM is covered in detail in the User Manual of GTM [5].

To generate PWM signals, TOM/ ATOM Channels are used. It has to be ensured by proper configuration that same resources should not be used by PWM or GPT drivers.

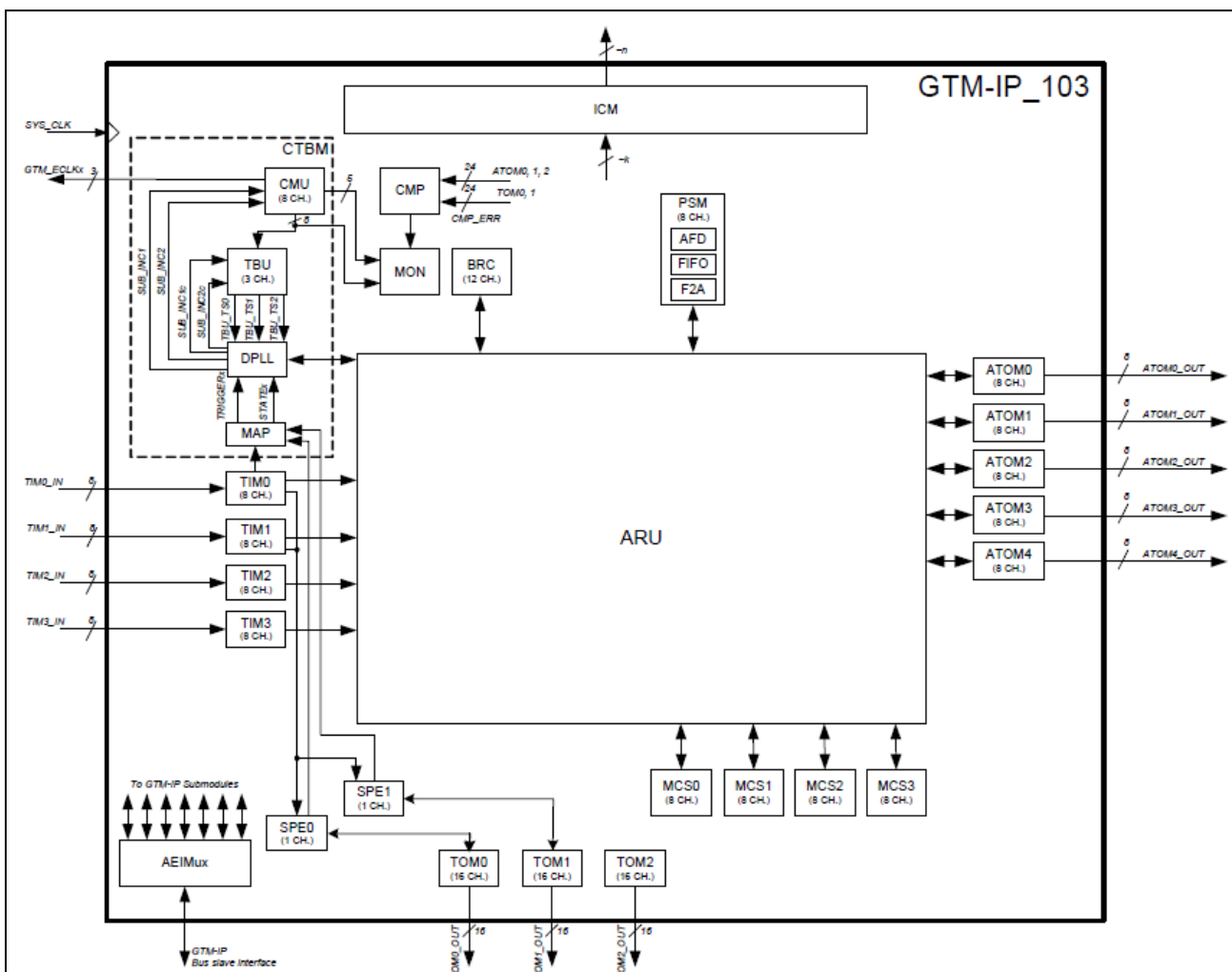


Figure 2 GTM – Hardware Block Diagram

2.1.2 PWM and TOM/ATOM Channel Resources

The PWM is generated using the TOM/ATOM Channel of the GTM hardware. The TOM/ATOM Channel of the GTM has the capability to act as timer and compare cells for Period and Duty.

TOM/ATOM Channel consists of counter (CN0), Counter Compare Unit0 (CCU0) for Period and Counter Compare Unit1 (CCU1) for Duty, The CN0 acts as a timer for PWM Signal, CCU0 comparator for Period and CCU1 Comparator for Duty Cycle of the PWM signal.

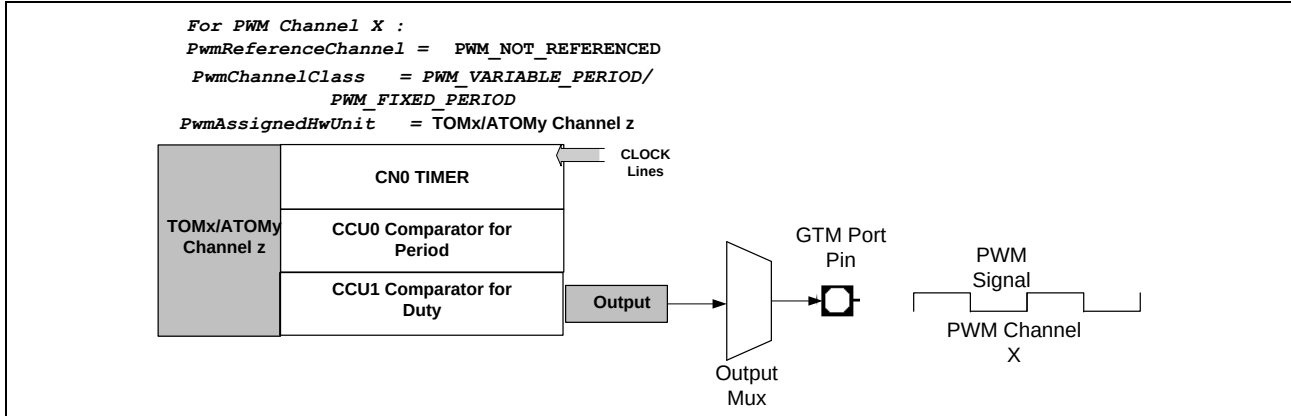


Figure 3 TOM/ATOM Channel

The duty cycle can be varied by changing the compare value on CCU1. The period of the signal can be varied by changing the compare value on CCU0. Using this strategy as the building block, additional TOM/ATOM Channels can be tied together to form PWM signals with different properties.

Clock Source for Timer Cells

The clock source for each of TOM/ATOM Channel is derived from the GTM Clock Management Unit (CMU). The CMU distributes 5 fixed clock (fixed divider) lines for usage by the TOM, 8 configurable clock lines for usage by the ATOM. The clock source configuration is part of the GTM global configuration and is explained in detail in the user manual of GTM [5].

A PWM Channel is tied with an attribute PWM channel class. The following are the possible PWM channel classes

- PWM_FIXED_PERIOD
- PWM_FIXED_PERIOD_SHIFTED
- PWM_FIXED_PERIOD_CENTER_ALIGNED
- PWM_VARIABLE_PERIOD

The description for each class is given in [Table 35](#).

The following table shows the dependency between Channel class and PWM Driver configuration parameters.

2.1.2.1 PWM Grouping

The PWM group is built with more than one PWM Channel. PWM channels can be grouped to get PWM signals with offset from the Reference Channel. To achieve this functionality user should configure these channels sequentially.

The reference channel for this group is configured first and class must be of PWM_FIXED_PERIOD type. The channels that are tied with this reference channel can be of PWM_FIXED_PERIOD_SHIFTED or PWM_FIXED_PERIOD_CENTER_ALIGNED type. The behavior and HW resource used for these types are described below.

- **PWM_FIXED_PERIOD:**

This acts as the reference PWM signal for the following Channels.

- **PWM_FIXED_PERIOD_SHIFTED:**

The PWM signal generated by this channel will be shifted from the reference channel.

Note: With 0% PwmShiftValue the channel will behave like PWM_FIXED_PERIOD type PWM channel.

The following **Figure 4** shows a shift aligned waveform; PwmShiftValue is given in terms of percentage like it is given in PwmDutycycleDefault parameter. The first waveform shows the channel with a shift value that doesn't cross the period boundary and the second waveform depicts when duty cycle and shift crosses 100%.

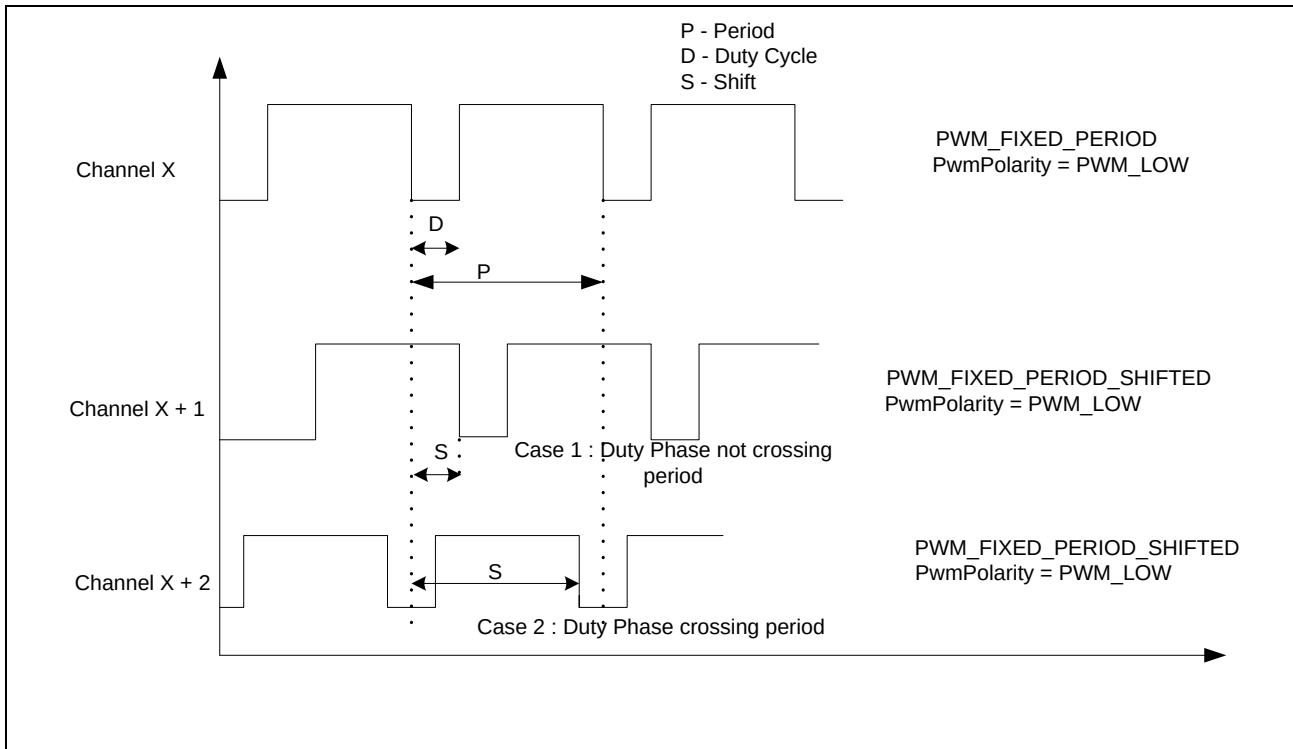


Figure 4 Shift Aligned PWM Waveform

TOM/ATOM channel arrangement with one PWM_FIXED_PERIOD channel and PWM_FIXED_PERIOD_SHIFTED channel is shown in the following **Figure 5**.

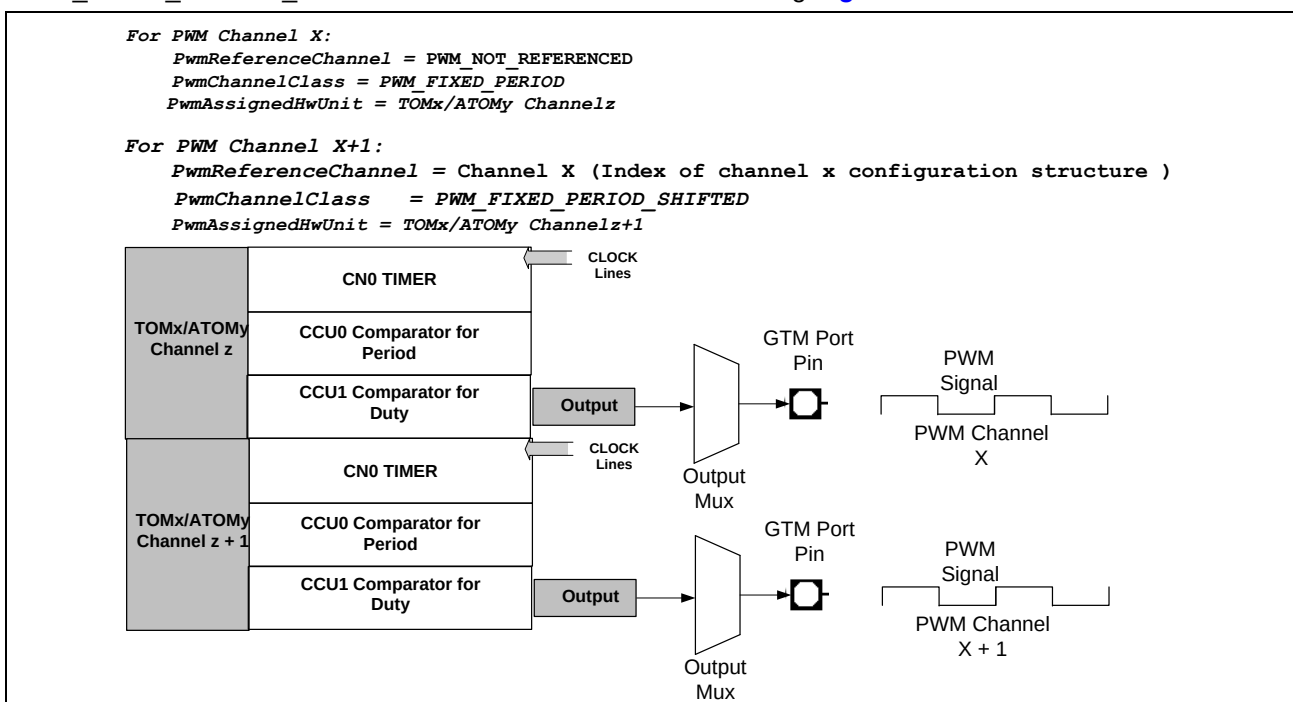


Figure 5 Shifted Type

- **PWM_FIXED_PERIOD_CENTER_ALIGNED:**

The PWM signal generated by this channel will be center aligned with in a period from the reference channel. The following [Figure 6](#) shows a center aligned PWM waveform.

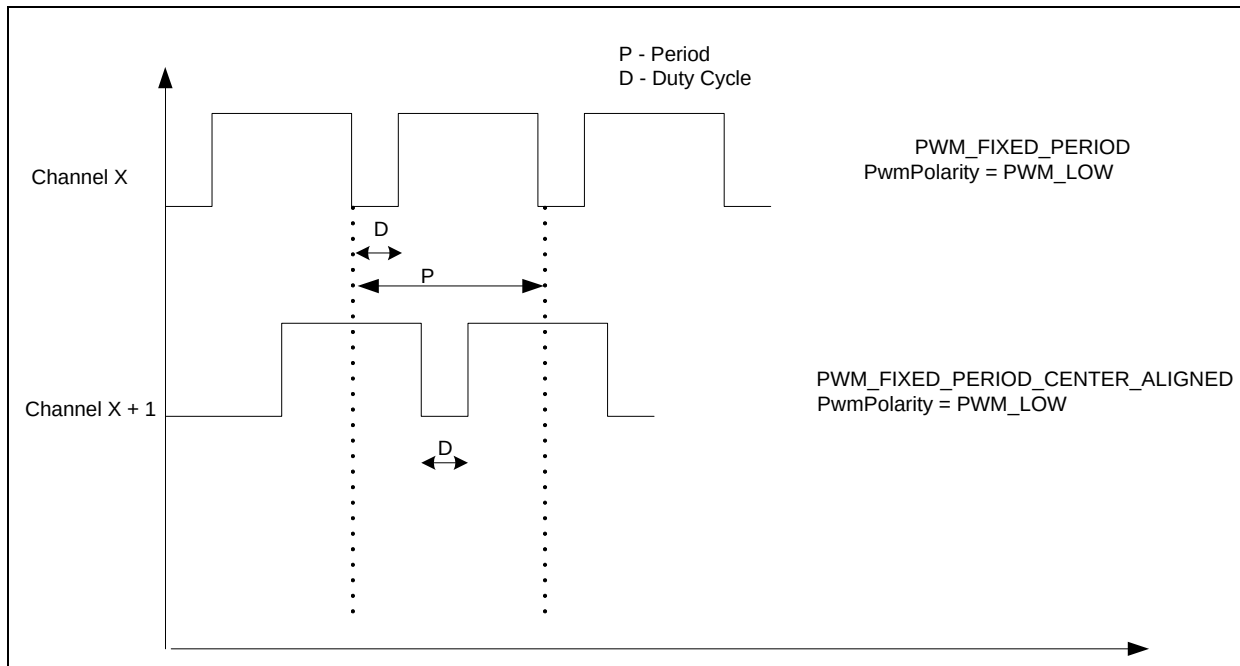


Figure 6 Center Aligned PWM type waveform

TOM/ATOM Channel arrangement with one PWM_FIXED_PERIOD Channel and PWM_FIXED_PERIOD_CENTER_ALIGNED channel is shown in the following figure [Figure 5](#).

Note: The reference for grouping of channels is only possible with immediate previous fixed period channel. During configuration the user should take care.

2.1.2.2 PWM Variable period channels

The PWM_VARIABLE_PERIOD channels and PWM_FIXED_PERIOD channels can work independently. As stated earlier the other classes of channels (PWM_FIXED_PERIOD_SHIFTED, and PWM_FIXED_PERIOD_CENTER_ALIGNED) require a reference channel of class PWM_FIXED_PERIOD.

2.1.2.3 PWM Channel types

The following channel types are supported

1. PWM_FIXED_PERIOD
2. PWM_VARIABLE_PERIOD
3. PWM_FIXED_PERIOD_CENTER_ALIGNED
4. PWM_FIXED_PERIOD_SHIFTED (PwmHandleShiftByOffset = false)
5. PWM_FIXED_PERIOD_SHIFTED (PwmHandleShiftByOffset = true)

2.1.2.4 Design for Shifted channel (PwmHandleShiftByOffset = false)

Different solution proposed and their limitation is listed below.

Case 1:

Type	Register values	Problem in entering 0%	Problem in leaving 0%
Shift =0	SR1 =2 SR0 = period + 1	Yes	Yes
Shift != 0	SR1 =2 SR0 = period + 1	No*	No

Type	Register values	Problem in entering 100%	Problem in leaving 100%
Shift =0	SR1 =2 SR0 =2	No**	No
Shift != 0	SR1 =2 SR0 =2	Yes	Yes

* Changing from 100 to 0% will behave like shift =0

** Changing from 0 to 100% will behave like shift != 0

Case 2:

Design for shifted channel with coherent update for 0% duty

Type	Register values	Problem in entering 0%	Problem in leaving 0%
Shift =0	SR1 = Period + 1 SR0 = 1	No	No
Shift != 0	SR1 = Period + 1 SR0 = 1	No	No

For non-coherent update CM0 and CM1 are updated instead of SR0 and SR1

Design for shifted channel with coherent update for 100% duty

Type	Register values	Problem in entering 100%	Problem in leaving 100%
Shift =0	SR1 =0 SR0 =Period + 1	No	No
Shift != 0	SR1 =0 SR0 =Period + 1	No	Yes

Note:

For BC step for 0% duty the registers are updated with the value CM1 = Period+1 and CM0=2. This is because the events on the compare match with the value CM0=1 does not happen in BC step. Similarly for 100% duty CM1=1 and CM0 = Period+1 values are updated

Decision:

The case 2 is more superior in terms of handling 0 and 100% duty cycle. This (case 2) also has a limitation of handling leading edge notification of the channel due to a HW (GTM) issue. There is an easier workaround possible for this issue i.e. to use the referred fixed period channel to handle the notification. There is another limitation with respect the condition, leaving 100% duty when shift not equal to zero. This limitation is described below. Case 2 is chosen for implementation due to better signal output.

Limitation on leaving 100% duty when shift not equal to zero

The transition from 100% to other percentages with a shift will not happen as per expected signal in case of coherent update. This is because three signal level changes are needed to produce the expected waveform. The registers CM0 and CM1 are updated from shadow registers at the end of the cycle. The signal will remain is the same level till the CM0 value is reached, as shown below.

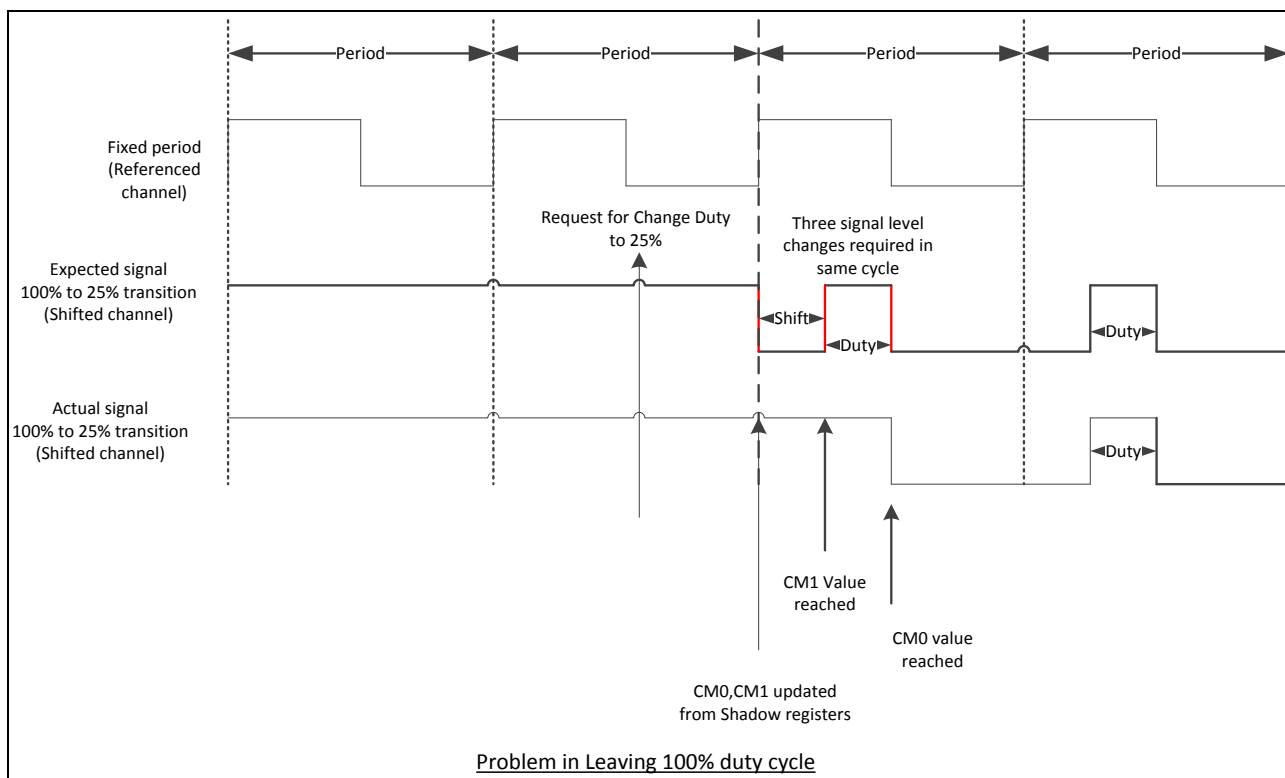


Figure 7 The transition from 100% to other percentages with a shift value

Note: This limitation is also applicable for cases where call is made from SetOutputToldle API to some other duty.

2.1.2.5 Design for fixed period channel

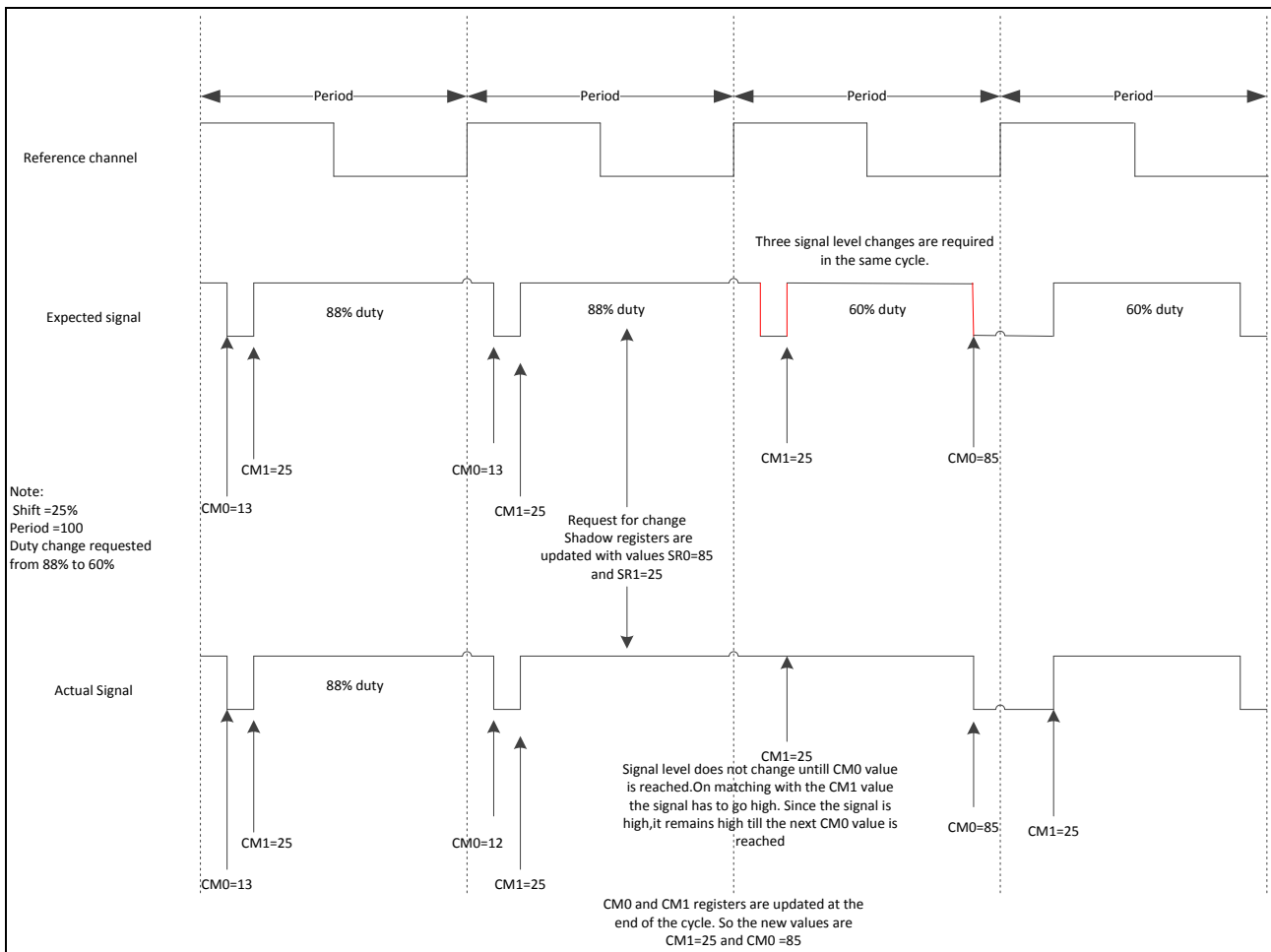
There are no limitations with respect to a fixed / variable period channel the registers will hold the following values in the implementation.

Table 3 Design for Fixed period and variable period channels

Register\Duty	0%	100%	0% < duty > 100%
SR1	0	Period	duty
SR0	Period	Period	Period

2.1.2.6 Request for duty change in shifted channel

If a request is made to change the duty from the condition shift and dutycycle value together is greater than period ($\text{Shift} + \text{Duty} > \text{Period}$) to less than period ($\text{Shift} + \text{Duty} < \text{Period}$) in the case for a coherent updated shifted channel where shift value is not equal to zero, the following behavior will be observed



In the figure above, on compare match with CM1 value the signal will go high and on compare match with CM0 value the signal will go low. The values are calculated as $CM1/SR1 = \text{Shift}$ and $CM0/SR0 = (\text{Shift} + \text{Duty \% period})$. Here in the figure initially 88% duty was requested and shift was 25%. So if the period is 100 ticks then, $CM1 = 25$ and $CM0 = 13$ ($(25 + 88) \% 100$). Later when the duty of 60% is requested the values calculated are $SR1 = 25$ (Shift) and $SR0 = 85$ using $((25 + 60) \% 100)$.

The CM1 and CM0 values are updated from Shadow registers SR1 and SR0 respectively (in the case of coherent update). When the duty change is requested then the function `Pwm_17_Gtm_SetDutyCycle` will update the corresponding shadow registers and return immediately after completion of the function. The updating of shadow registers to CM0 and CM1 registers will happen only after the end of the Period.

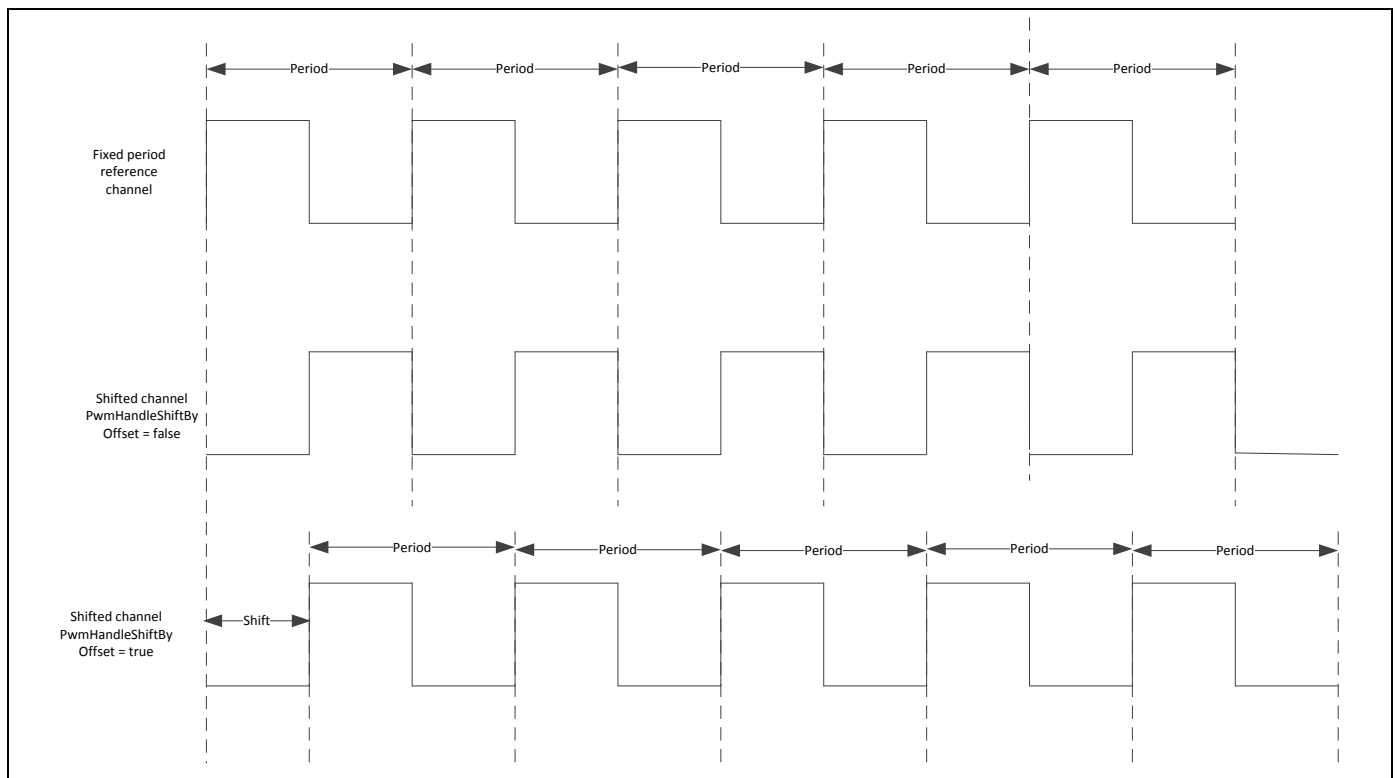
Since the new values are updated to CM0 register from shadow register the signal will remain in the same state till the CM0 value is reached as shown in the figure. The signal will remain high for a time greater than period as shown in the figure above because there are three signal level changes required in the same cycle which could not be handled with two values CM0 and CM1 available at our disposal.

2.1.2.7 Design for Shifted channels handled by an offset

A global configuration parameter has been added `PwmHandleShiftByOffset` in the `PwmGeneral` container and once this parameter is selected then the shifted channels are configured similar to Fixed period channels and but they are started by an offset. The offset is calculated by $\text{Offset} = \text{Period} - \text{Shift}$. This offset acts as the required shift. All the channels in the TGC where these kind of shifted channels are present are triggered by a global host trigger. This host trigger will help in enabling all the channels in the TGC at the same time, and since all the channels are started the same time the corresponding shift is achieved accordingly. Please refer to respective API's to know how these kind of shifted channels are handled.

Table 4 Difference between the two types of handling for shifted channels

Subject	<i>FPS(PwmHandleShiftByOffset=false)</i>	<i>FPS(PwmHandleShiftByOffset=true)</i>
Handling of shift	Compare register is used	Counter register is preloaded with Period - Shift
Handling of period	Fixed period reference channel's compare register is used	No reference channel, Same channels compare register is used, however in configuration reference channel is provided for AUTOSAR compatibility
Host trigger	Host trigger is not used	Host trigger is used.
TGC restriction due to Host trigger	Channels in the same TGC can be shared across other drivers.	Channels in the same TGC cannot be shared across other drivers.
Handling of shift + duty crossing period	For more than one cycle the output line is either high or low during change of duty	No limitation in this case.
Leaving 100% duty cycle	The signal will remain in the 100% duty till the next compare value is reached; The shift will not be visible.	The signal will remain in the 100% duty till the next compare value is reached; The shift will not be visible.
Update of new duty cycle	In case of coherent update the new duty cycle always happens at the end of reference channel period	In case of coherent update the new duty cycle happens at the end of its own period. This means in corner cases the new duty cycle might happen in the same cycle of the configured reference channel.
Pwm_17_Gtm_SetOutputTogle function	Always happens in the next cycle.	Always happens in the next cycle.
Update of 0% and 100% duty cycle	Happens at the start of the next cycle of reference channel	Happens at the start of the next cycle after the shift value is elapsed.
Compare register value for 0 and 100%	Please refer Design for shifted channels for details	Compare values are similar to Fixed Period



For the shifted channel with `PwmHandleShiftByOffset = false` in case of coherent update the values are updated at the end of the period of the “reference fixed period channel”. So any change will be visible in the next cycle of the reference channel.

For the shifted channel with `PwmHandleShiftByOffset = true` in case of coherent update the values are updated at the end of the period of the “same channel” (The shifted channel is independent of reference channel).

So the output may vary for the two types of shifted channels depending on configuration and moment of request for change of duty.

2.1.3 PORT

The PORT Driver controls all access to the port pins required by PWM. The PORT Driver has to be configured to set the direction of pins as output and the functionality of each pin should be in ALT_x (x = 1 to 7) based on the pin output functionality for GTM) as per the grouping. For details on grouping of pins with each TOM/ATOM Channel, refer [5] GTM user manual.

It is important to note, that the PORT Driver must be initialized for the pins required by PWM. See PORT User Manual [4] for details about PORT configuration.

2.1.4 IRQ

IRQ Driver is responsible for initializing the priority of interrupts required by GTM. The mapping of TOM/ATOM Channel resources to respective SRN is given in Table 88. Refer IRQ User Manual [6] for more details.

2.1.5 SYSTEM CLOCK

The system clock is set up via the MCU Driver. This is mandatory to set up an appropriate system clock. Note that the GTM peripheral clock is derived from the configured system clock.

See also MCU User Manual for details about MCU configuration [3].

2.2 Software Driver Description

The configuration parameters and API of PWM Driver are shown in Figure 8. The Application Programming Interface (API) for the PWM Driver and its interaction with other drivers are also depicted. The Development Error Tracer (DET) Driver is required if DET is enabled for the PWM Driver. `SafeMcal_ReporError` is reported in case of any safety error if safety features are enabled.

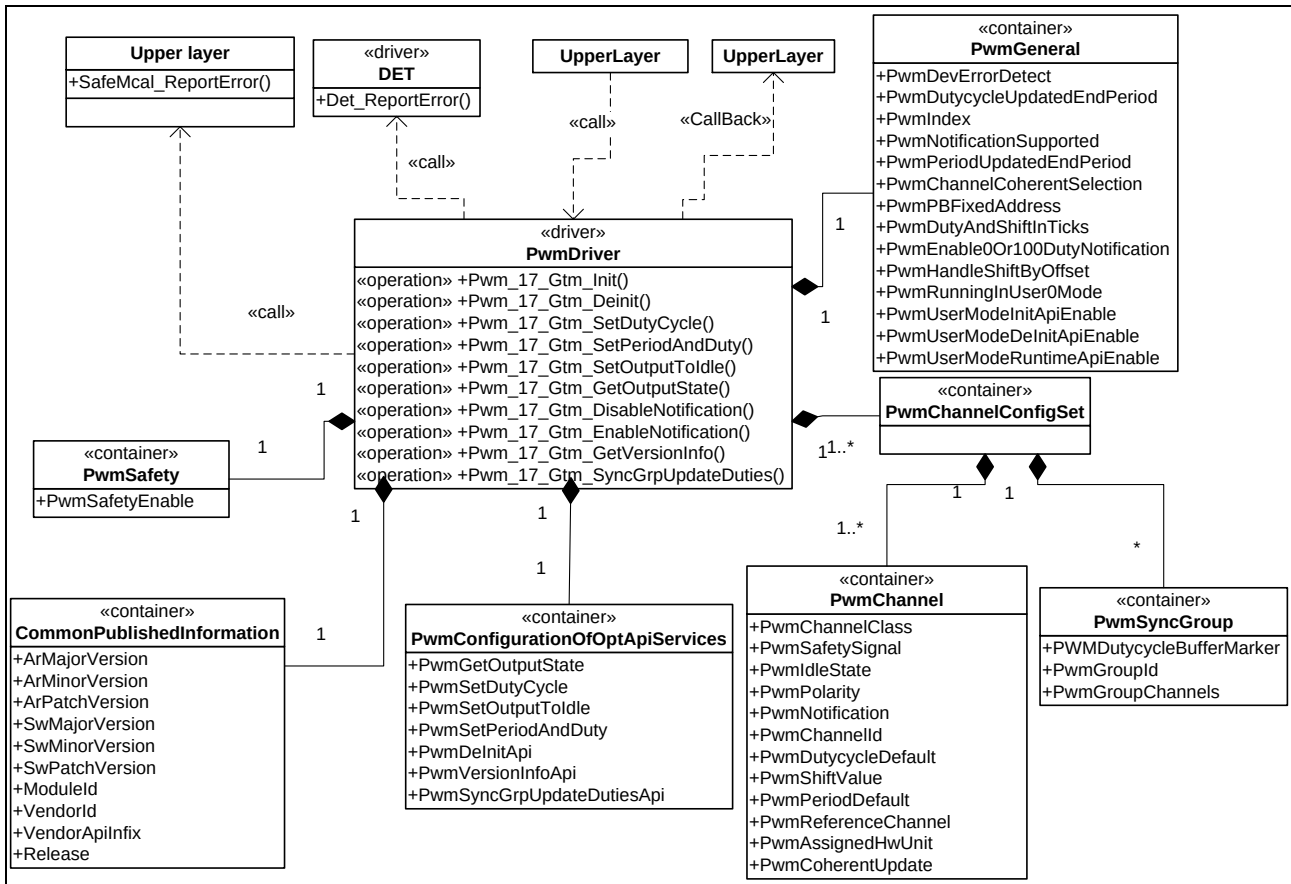


Figure 8 Interfaces of PWM Driver

3 File Structure

This section provides details about the PWM Driver files and other related files.

3.1 Driver File Structure

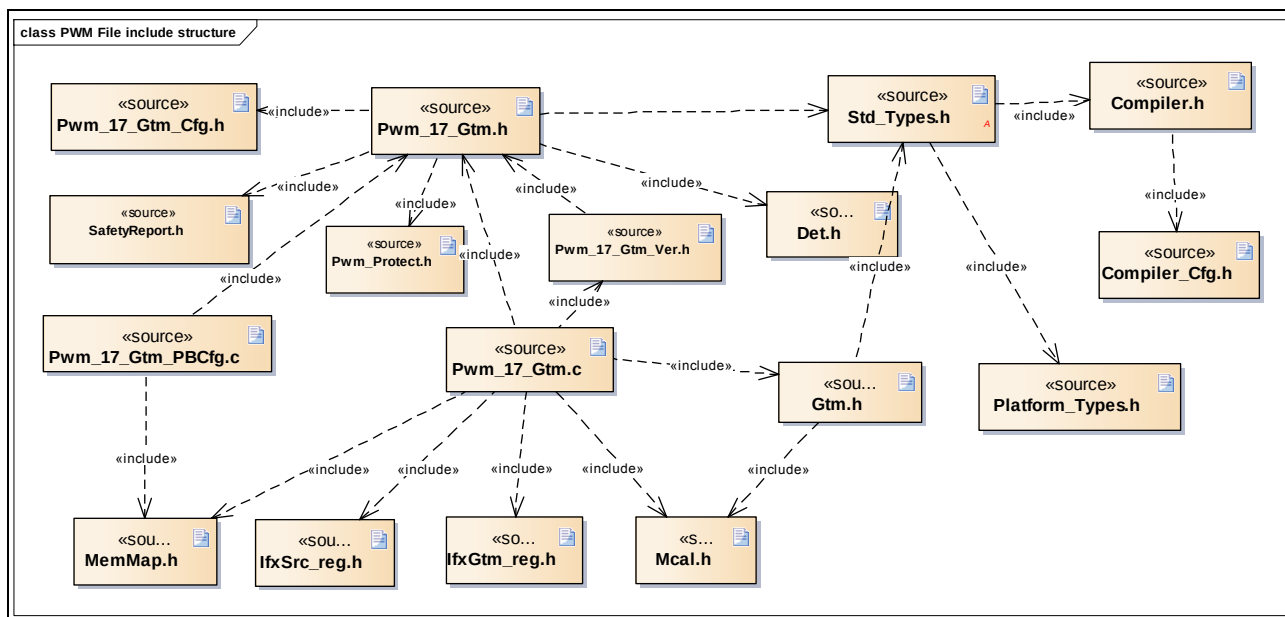


Figure 9 File Structure

Table 5 File Contents

File Name	File Contents
Platform_Types.h	Platform dependencies are declared here.
Compiler.h	Compiler dependencies are declared here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code.
Std_Types.h	Standard data types to be used are declared here.
IfxGtm_reg.h	Includes the Gtm register definition file for Aurix microcontroller.
IfxSrc_reg.h	Includes the Src register definition file for Aurix microcontroller.
Mcal.h	Provides library functions for MCAL layer
Det.h	Provides the functionality of Development Error Tracer module
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections
Gtm.h	Provides GTM definitions.
Pwm_17_Gtm.h	Driver configuration structure and API functions are exported to the user.
Pwm_17_Gtm.c	Driver implementation
Pwm_17_Gtm_Cfg.h	Contains Driver precompile configuration parameters.
Pwm_17_Gtm_PBCfg.c	Contains Driver postbuild configuration parameters.
Pwm_17_Gtm_Ver.h	Version specific checks.
Pwm_Protect.h	Included to define all the first level indirection macros for Protected mode support for supervisory access via OS calls

4 Configuration Documentation

The following chapters summarize all configuration parameters, their description and the type definition used to represent them in the code. This includes those specified in the AUTOSAR PWM Specification and the microcontroller specific parameters.

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be frozen before compilation and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile-configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time-, link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the parameters configuration for a re-loadable binary.

The PWM Driver is delivered as a Variant PB.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Note: Topics [4.1.1](#), [4.1.2](#) should be read for understanding the configuration class field

Table 6 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, and Variant PC.

Table 7 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Configuration Parameter Variance

This section describes the variance in the configuration parameter w.r.t supported microcontroller. PWM Driver does not have varying parameters across the supported microcontrollers.

4.3 Container: PwmGeneral

This section describes all configuration parameters of the container PWMGeneral. It collects mainly global switches of the PWM Driver described by the AUTOSAR PWM Driver specification [\[1\]](#)

Note: The “**configuration class**” details are given for all Variants specified in the SWS. The section “type” in the configuration table is explained only for Configuration Class “Post Build”. The configuration class states the possible support for each delivery.

4.3.1 PwmDevErrorDetect

Table 8 PwmDevErrorDetect

Name:	PwmDevErrorDetect		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switches the development error detection ON or OFF.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.3.2 PwmDutycycleUpdatedEndPeriod

Table 9 PwmDutycycleUpdatedEndPeriod

Name:	PwmDutycycleUpdatedEndPeriod		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The global (all configured PWM channels) switch for enabling the update of the duty cycle parameter at the end of the current period. TRUE: update of duty cycle is done at the end of period of currently generated waveform (current waveform is finished). FALSE: update of duty cycle is done immediately (just after service call, current waveform is cut). <u>PWM duty immediate update:</u> As per the hardware behavior, if duty cycle of the PWM signal is requested for a change, after the completion of the duty cycle (active) time of the ongoing signal then the new Duty Cycle will take effect only after the completion of the ongoing signal.		
SFR Modified	None		
Scope:	Driver		
Dependency:	PwmChannelCoherentSelection		

4.3.3 PwmPeriodUpdatedEndPeriod

Table 10 PwmPeriodUpdatedEndPeriod

Name:	PwmPeriodUpdatedEndPeriod		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

Description:	The global (all configured PWM channels) switch for enabling the update of the period parameter at the end of the current period. TRUE: update of period/duty cycle is done at the end of period of currently generated waveform (current waveform is finished). FALSE: update of period/duty cycle is done immediately (just after service call, current waveform is cut). PWM_FIXED_PERIOD, PWM_FIXED_PERIOD_SHIFTED and PWM_FIXED_PERIOD_CENTER_ALIGNED class channels do not get affected due to this parameter setting. <u>PWM duty immediate update:</u> As per the hardware behavior, if duty cycle of the PWM signal is requested for a change, after the completion of the duty cycle (active) time of the ongoing signal then the new Duty Cycle will take effect only after the completion of the ongoing signal.
SFR Modified	None
Scope:	Driver
Dependency:	PwmChannelCoherentSelection

4.3.4 PwmNotificationSupported

Table 11 PwmNotificationSupported

Name:	PwmNotificationSupported		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--
Description:	If ON, notifications are supported. OFF, notifications are not supported. Configures the generation of Notification functions.		
SFR Modified	None		
Scope:	Driver		
Dependency:	--		

4.3.5 PwmIndex

Table 12 PwmIndex

Name:	PwmIndex
Source:	This parameter is defined by the AUTOSAR, refer [1]
File:	Pwm_17_Gtm_Cfg.h
Range:	Values:0 to 255 Default value: 0

Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Module ID of the PWM module instance.		
SFR Modified	None		
Scope:	Driver		
Dependency:	--		

4.3.6 PwmChannelCoherentSelection

Table 13 PwmChannelCoherentSelection

Name:	PwmChannelCoherentSelection		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Switch for enabling the channel wise update of the duty/period parameter at the end of the current period. TRUE: update of period/duty cycle is done based on the PWM channel specific configuration parameter PwmCoherentUpdate. FALSE: update of period/duty cycle is done based on the global parameters PwmDutycycleUpdatedEndPeriod and PwmPeriodUpdatedEndPeriod. <u>PWM duty immediate update:</u> As per the hardware behavior, if duty cycle of the PWM signal is requested for a change, after the completion of the duty cycle (active) time of the ongoing signal then the new Duty Cycle will take effect only after the completion of the ongoing signal.		
SFR Modified	None		
Scope:	Driver		
Dependency:	Enabling this parameter will obsolete PwmPeriodUpdatedEndperiod and PwmDutycycleUpdatedEndperiod		

4.3.7 PwmPBFixedAddress

Table 14 PwmPBFixedAddress

Name:	PwmPBFixedAddress		
Source:	This parameter is microcontroller specific.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON – Fixed address feature used OFF – Fixed address feature not used Default value: OFF		
Configuration Class:	Pre-Compile	X	All Variants

	Link Time	--	--
	Post Build	--	--
Description:	The fixed address feature supports the optimization of a post build configuration with a fixed address by avoiding unnecessary indirection.		
SFR Modified	None		
Scope:	--		
Dependency:	--		

4.3.8 PwmDutyShiftInTicks

Table 15 PwmDutyShiftInTicks

Name:	PwmDutyShiftInTicks		
Source:	This parameter is microcontroller specific.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON – Duty and Shift values are in absolute ticks OFF – Duty and Shift values are relative to period Default value: OFF		
Configuration Class:	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter determines whether Dutycycle and Shift values are absolute or relative. Setting this parameter to true enables the user to enter the dutycycle and shift values as absolute ticks. Setting this parameter to false enables the user to enter the dutycycle and shift values as percentage, relative to period. If this parameter is set to ON, Up to 32 bit values (Ticks) are supported in ATOM/TOM channels but the maximum values supported are:. ATOM supports 24 values (up to 16777214). TOM supports 16 bit values (up to 65534).		
SFR Modified	None		
Scope:	--		
Dependency:	--		

4.3.9 PwmEnable0Or100DutyNotification

Table 16 PwmEnable0Or100DutyNotification

Name:	PwmEnable0Or100DutyNotification		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--

Name:	PwmEnable0Or100DutyNotification
Description:	If ON, notifications are supported for 0 and 100% for Fixed and variable period channels. OFF, notifications are not supported for 0 and 100% for Fixed and variable period channels.
Scope:	Driver
Dependency:	PwmEnable0Or100DutyNotification is dependent on PwmNotificationSupported

4.3.10 PwmHandleShiftByOffset

Table 17 PwmHandleShiftByOffset

Name:	PwmHandleShiftByOffset		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	-	--
	Post Build	--	--
Description:	If PwmHandleShiftByOffset is enabled then the shifted channels will be started with an offset and will work similar to fixed period channel. If PwmHandleShiftByOffset is not selected then the shifted channels are triggered from the referenced fixed period channel		
Scope:	Driver		
Dependency:	Please note that the behavior of the shifted channels is altered with this switch.		

4.3.11 PwmRunningInUser0Mode

Table 18 PwmRunningInUser0Mode

Name:	PwmRunningInUser0Mode		
Source:	This parameter is microcontroller specific		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If 'PwmRunningInUser0Mode' is enabled then the API's selected by 'PwmUserModeInitApiEnable' or 'PwmUserModeDeInitApiEnable' or 'PwmUserModeRuntimeApiEnable' will run in user mode 0.		

	If the 'PwmRunningInUser0Mode' switch is not selected and if any of the switches 'PwmUserModeInitApiEnable', 'PwmUserModeDeInitApiEnable', 'PwmUserModeRuntimeApiEnable' are selected then those APIs will run in user mode 1. This switch selects between user mode0 and user mode 1 PwmRunningInUser0Mode – STD_ON implies User Mode 0 PwmRunningInUser0Mode – STD_OFF implies User Mode 1 if any of the switches (PwmUserModeInitApiEnable or PwmUserModeDeInitApiEnable or PwmUserModeRuntimeApiEnable) are true else Supervisor Mode (If all the four switches mentioned are OFF).
Scope:	Driver
Dependency:	If 'PwmRunningInUser0Mode' is enabled then at least any one of the switches 'PwmUserModeInitApiEnable' or 'PwmUserModeDeInitApiEnable' or 'PwmUserModeRuntimeApiEnable' should be enabled.

4.3.12 PwmUserModeInitApiEnable

Table 19 PwmUserModeInitApiEnable

Name:	PwmUserModeInitApiEnable		
Source:	This parameter is microcontroller specific		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If PwmUserModeInitApiEnable is enabled then the Pwm_17_Gtm_Init API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch PwmRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.3.13 PwmUserModeDeInitApiEnable

Table 20 PwmUserModeDeInitApiEnable

Name:	PwmUserModeDeInitApiEnable		
Source:	This parameter is microcontroller specific		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration	Pre-compile	x	All Variants

Class:	Link Time	--	--
	Post build	--	--
Description:	If PwmUserModeDeInitApiEnable is enabled then the Pwm_17_Gtm_DeInit API will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch PwmRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.3.14 PwmUserModeRuntimeApiEnable

Table 21 PwmUserModeRuntimeApiEnable

Name:	PwmUserModeRuntimeApiEnable		
Source:	This parameter is microcontroller specific		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: TRUE FALSE Default Value : FALSE		
Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	If PwmUserModeRuntimeApiEnable is enabled then all APIs other than Pwm_17_Gtm_Init and Pwm_17_Gtm_DeInit APIs will run in User mode. Selection between User Mode 0 and User mode 1 is based on the switch PwmRunningInUser0Mode		
Scope:	Driver		
Dependency:			

4.4 Container: PwmConfigurationOfOptApiServices

This section describes all configuration parameters of the container PWM ConfigurationOfOptApiServices. It collects mainly global switches of the PWM Driver APIs described by the AUTOSAR PWM Driver specification [\[1\]](#).

4.4.1 PwmVersionInfoApi

Table 22 PwmVersionInfoApi

Name:	PwmVersionInfoApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--

Name:	PwmVersionInfoApi		
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_GetVersionInfo API is available to the user. OFF: Pwm_17_Gtm_GetVersionInfo API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None.		

4.4.2 PwmDelInitApi

Table 23 PwmDelInitApi

Name:	PwmDelInitApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_DelInit API is available to the user. OFF: Pwm_17_Gtm_DelInit API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.3 PwmGetOutputStateApi

Table 24 PwmGetOutputStateApi

Name:	PwmGetOutputStateApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_GetOutputState API is available to the user. OFF: Pwm_17_Gtm_GetOutputState API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.4 PwmSetDutyCycleApi

Table 25 PwmSetDutyCycleApi

Name:	PwmSetDutyCycleApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_SetDutyCycle API is available to the user. OFF: Pwm_17_Gtm_SetDutyCycle API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.5 PwmSetOutputToldleApi

Table 26 PwmSetOutputToldleApi

Name:	PwmSetOutputToldleApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_SetOutputToldle API is available to the user. OFF: Pwm_17_Gtm_SetOutputToldle API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.6 PwmSetPeriodAndDutyApi

Table 27 PwmSetPeriodAndDutyApi

Name:	PwmSetPeriodAndDutyApi		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	ON - true OFF - false		

Name:	PwmSetPeriodAndDutyApi		
	Default value : OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	ON: Pwm_17_Gtm_SetPeriodAndDuty API is available to the user. OFF: Pwm_17_Gtm_SetPeriodAndDuty API is not available to the user.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.4.7 PwmSyncGrpUpdateDutiesApi

Table 28 PwmSyncGrpUpdateDutiesApi

Name:	PwmSyncGrpUpdateDutiesApi		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	STD_ON - true STD_OFF - false Default value : STD_OFF		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	STD_ON Pwm_17_Gtm_SyncGrpUpdateDuties API is available to the user. STD_OFF Pwm_17_Gtm_SyncGrpUpdateDuties API is not available to the user.		
Scope:	Driver		
Dependency:	None		

4.5 Container: Pwm Channel Configuration

This section describes all configuration parameters of the container PWM channel configuration. It collects parameters required for each channel as described by the AUTOSAR PWM Driver specification [\[1\]](#).

4.5.1 PwmNotification

Table 29 PwmNotification

Name:	PwmNotification		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Values: Pwm notification pointer of type FUNCTION_NAME(Selectable) / Address (Loadable)		
Configuration Class:	Pre Compile	x	Variant PC

Name:	PwmNotification		
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Notification Pointer function or callback function: When the signal edge is generated this function will be called by the PWM Driver for each individual channel.		
Scope:	Driver		
SFR Modified	None		
Dependency:	PwmPeriodUpdatedEndPeriod, PwmDutycycleUpdatedEndPeriod, PwmChannelCoherentSelection and PwmCoherentUpdate		

4.5.2 PwmPeriodDefault

Table 30 PwmPeriodDefault

Name:	PwmPeriodDefault		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Significant if PwmChannelClass is PWM_FIXED_PERIOD or PWM_VARIABLE_PERIOD Values:0 to 0xFFFFFE Default value: 0 If the assigned hardware unit is TOM channel, user must ensure the value is within the range 0 to 0xFFFE. The driver supports period range of up to 0xFFFE for TOM and ATOM in autosar mode.		
Configuration Class:	Pre Compile	x	Variant PC
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Value of period to be used in initialization function.		
SFR Modified	GTM_TOMi_CHj_CM0 GTM_ATOMi_CHj_CM0		
Scope:	Driver		
Dependency:	None		

4.5.3 PwmDutycycleDefault

Table 31 PwmDutycycleDefault

Name:	PwmDutycycleDefault		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Depends on PwmDutyShiftInTicks parameter Values:0 to 0x8000, if PwmDutyShiftInTicks is OFF Values:0 to 0xFFFE for TOM and 0xFFFFFE for ATOM, if PwmDutyShiftInTicks is ON Default value: 0		
Configuration Class:	Pre Compile	x	Variant PC
	Link Time	--	--

Name:	PwmDutycycleDefault		
	Post Build	x	Variant PB
Description:	Value of duty cycle for a channel. It can be entered as absolute ticks or as relative percentage based on the parameter PwmDutyShiftInTicks. When PwmDutyShiftInTicks is OFF, the value is relative to period. 0 – 0%, 0x8000 – 100% When PwmDutyShiftInTicks is ON, the value is in absolute ticks. 0 – 0%, Value same as or above PwmPeriodDefault value – 100% If parameter PwmDutyShiftInTicks is set to ON, values from 0 to 0xFFFFE for TOM and 0 to 0xFFFFFE for ATOM are supported. User must ensure the value is within the range.		
SFR Modified	GTM_TOMi_CHj_CM0 GTM_ATOMi_CHj_CM0		
Scope:	Driver		
Dependency:	PwmDutyShiftInTicks		

4.5.4 PwmShiftValue

Table 32 PwmShiftValue

Name:	PwmShiftValue		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	[PWM_CR_029] Significant if PwmChannelClass is PWM_FIXED_PERIOD_SHIFTED Depends on PwmDutyShiftInTicks parameter Values: 0..0x8000, if PwmDutyShiftInTicks is OFF Values: 0 to PwmPeriodDefault of reference channel, if PwmDutyShiftInTicks is ON Default value: 0		
Configuration Class:	Pre Compile	x	Variant PC
	Link Time	--	--
	Post Build	x	Variant PB
Description:	Parameter to configure the initial shift value with respect to PWM_FIXED_PERIOD class channel. It can be entered as absolute ticks or as relative percentage based on the parameter PwmDutyShiftInTicks. When PwmDutyShiftInTicks is OFF, the value is relative to period. 0 – 0%, 0x8000 – 100% When PwmDutyShiftInTicks is ON, the value is in absolute ticks. 0 – 0%, Value same as PwmPeriodDefault of reference channel – 100% If parameter PwmDutyShiftInTicks is set to ON, values from 0 to PwmPeriodDefault of reference channel.		
SFR Modified	NA		

Name:	PwmShiftValue
Scope:	Driver
Dependency:	PwmChannelClass, PwmDutyShiftInTicks

4.5.5 PwmPolarity

Table 33 PwmPolarity

Name:	PwmPolarity		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Significant if PwmChannelClass is PWM_FIXED_PERIOD / PWM_FIXED_PERIOD_SHIFTED / PWM_FIXED_PERIOD_CENTER_ALIGNED / PWM_VARIABLE_PERIOD. Values: PwmPolarityHigh - The PWM Channel output is high at the beginning of the cycle. PwmPolarityLow - The PWM Channel output is low at the beginning of the cycle. Default value: PwmPolarityLow		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Initial State of PWM line.		
	GTM_TOMi_CHj_CTRL.U GTM_ATOMi_CHj_CTRL.U		
Scope:	Driver		
Dependency:	None		

4.5.6 PwmIdleState

Table 34 PwmIdleState

Name:	PwmIdleState		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Values: PwmIdleStateHigh -The PWM Channel output is high. PwmIdleStateLow -The PWM Channel output is low. Default value: PwmIdleStateLow		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	Value of PWM output signal to be used in Pwm_17_Gtm_SetOutputToldle and Pwm_17_Gtm_DeInit.		
Scope:	Driver		
SFR Modified	None		

Name:	PwmIdleState
Dependency:	None

4.5.7 PwmChannelClass

Table 35 PwmChannelClass

Name:	PwmChannelClass		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Values: PWM_FIXED_PERIOD PWM_FIXED_PERIOD_SHIFTED PWM_FIXED_PERIOD_CENTER_ALIGNED PWM_VARIABLE_PERIOD Default value: PWM_FIXED_PERIOD		
Configuration Class:	Pre Compile	X	Variant PC
	Link Time	--	--
	Post Build	X	Variant PB
Description:	PWM_FIXED_PERIOD: The PWM channel has a fixed period. Only duty cycle can be changed. PWM_FIXED_PERIOD_SHIFTED: This class of PWM channel refers to a PWM_FIXED_PERIOD channel for the period and the duty cycle is shifted by a required fixed percentage. Only duty cycle can be changed. PWM_FIXED_PERIOD_CENTER_ALIGNED: This class of PWM channel refers to a PWM_FIXED_PERIOD channel for the period and the duty cycle will fall exactly on the middle of the period. Only duty cycle can be changed. PWM_VARIABLE_PERIOD: This class of PWM channel has a variable period and duty cycle. Both can be changed during run time. This channel should not be referred by any other channel and doesn't refer any channel either.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.5.8 PwmReferenceChannel

Table 36 PwmReferenceChannel

Name:	PwmReferenceChannel
Source:	This parameter is defined by the AUTOSAR, refer [1]
File:	Pwm_17_Gtm_PBCfg.c
Range:	Significant if PwmChannelClass is PWM_FIXED_PERIOD_SHIFTED or PWM_FIXED_PERIOD_CENTER_ALIGNED Values: Index of the Reference Channel (0 to x) Reference channel must be provided if PwmChannelClass is PWM_FIXED_PERIOD_SHIFTED or PWM_FIXED_PERIOD_CENTER_ALIGNED

Name:	PwmReferenceChannel		
	A referred channel can only be of Fixed Period Class(i.e. PwmChannelClass is PWM_FIXED_PERIOD)		
Configuration Class:	Pre Compile	x	Variant PC
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The reference channel used in a PWM Group. Reference channel for PWM_FIXED_PERIOD_SHIFTED, and PWM_FIXED_PERIOD_CENTER_ALIGNED.		
Scope:	Driver		
SFR Modified	None		
Dependency:	The reference for grouping of channels is only possible with immediate previous fixed period channel. During configuration the user should take care.		

4.5.9 PwmAssignedHwUnit

Table 37 PwmAssignedHwUnit

Name:	PwmAssignedHwUnit		
Source:	This parameter is microcontroller implementation specific.		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Values: This is a reference parameter of type TOM/ATOM Channel TOM/ATOM Channel reference must be provided		
Configuration Class:	Pre Compile	x	Variant PC
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The TOM/ATOM Channel resource assigned to the Pwm channel.		
Scope:	Driver		
SFR Modified	None		
Dependency:	None		

4.5.10 PwmChannelId

Table 38 PwmChannelId

Name:	PwmChannelId		
Source:	This parameter is defined by the AUTOSAR, refer [1]		
File:	Pwm_17_Gtm_Cfg.h		
Range:	This is the Id of the Pwm Channel. This value will be assigned to the symbolic name derived of the PwmChannel The Id value must be consecutive Values : 0 to xx Default : 0 (A consecutive value is calculated for each new Pwm Channel)		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

Name:	PwmChannelId
Description:	This value will be assigned to the symbolic name derived of the PwmChannel
SFR Modified	NA
Scope:	Driver
Dependency:	None

4.5.11 PwmCoherentUpdate

Table 39 PwmCoherentUpdate

Name:	PwmCoherentUpdate		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	Values : ON - true OFF - false Default value : OFF		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	Switch for enabling the update of the duty/period parameter at the end of the current period. TRUE: update of period/duty cycle is done at the end of period of currently generated waveform (current waveform is finished). FALSE: update of period/duty cycle is done immediately (just after service call, current waveform is cut). Depending on the PwmChannelClass, the update end period will either work for duty cycle / duty and period. Both Duty and period will be updated for variable period channel and only duty cycle will be updated for other PwmChannelClass channels. <u>PWM duty immediate update:</u> As per the hardware behavior, if duty cycle of the PWM signal is requested for a change, after the completion of the duty cycle (active) time of the ongoing signal then the new Duty Cycle will take effect only after the completion of the ongoing signal.		
SFR Modified	None		
Scope:	Driver		
Dependency:	PwmChannelCoherentSelection		

4.5.12 PwmSafetySignal

Table 40 Specification of PwmSafetySignal

Name:	PwmSafetySignal
Source:	This parameter is defined by IFX
File:	Pwm_17_Gtm_PBCfg.c
Range:	Values:

	PWM_QM_TYPE PWM_ASIL_TYPE		
Default value:	PWM_QM_TYPE		
Configuration Class:	Pre-Compile	--	--
	Link Time	--	--
	Post Build	x	Variant PB
Description:	The parameter defines the Pwm channel is a part of ASIL signal or QM signal. This parameter is significant only if PwmSafetyEnable is set to True.		
SFR Modified	None		
Scope:	Driver		
Dependency:	PwmSafetyEnable		

4.6 Container: PwmSafety

This container contains the configuration parameters for the safety features of the Pwm driver. It is 1:1.

4.6.1 PwmSafetyEnable

Table 41 PwmSafetyEnable

Syntax:	PwmSafetyEnable		
Source:	This parameter is defined by IFX		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: TRUE - Safety features enabled for Pwm driver FALSE - Safety error detection disabled Default value: FALSE		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch for enabling the safety features of PWM driver. By enabling this flag, implicitly the following safety features are enabled: - Range check on API parameters will be done - PWM configuration structure will have a unique marker field which will be verified by the driver		
SFR Modified	NA		
Scope:	Driver.		
Dependency:	None		

4.7 Container: PwmSyncGroup

This section describes all configuration parameters of the container PwmSyncGroup configuration.

It collects parameters required grouping of channels used for Pwm_17_Gtm_SyncGrpUpdateDuties Api

4.7.1 PWMDutycycleBufferMarker

Table 42 PWMDutycycleBufferMarker

Name:	PWMDutycycleBufferMarker		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	0 -255		
Configuration Class:	Pre Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
Description:	If Safety feature is ON then this marker value is used to validate the duty cycle array sent as parameter to Pwm_17_Gtm_SyncGrpUpdateDuties Api		
Scope:	Driver		
Dependency:	PwmSafetyEnable switch in PwmSafety container and PwmSyncGrpUpdateDutiesApi in PwmConfigurationOfOptApiServices container		

4.7.2 PwmGroupChannels

Table 43 PwmGroupChannels

Name:	PwmGroupChannels.		
Source:	This is microcontroller specific parameter.		
File:	Pwm_17_Gtm_PBCfg.c		
Range:	This is a reference parameter of type PwmChannels to be added to the group.		
Configuration Class:	Pre Compile	--	--
	Link Time	--	--
	Post Build	x	All Variants
Description:	The PWM channels are added to the group. The Duty cycles for channels in group can be updated synchronously using Pwm_17_Gtm_SyncGrpUpdateDuties		
Scope:	Driver		
Dependency:	PwmSyncGrpUpdateDutiesApi in PwmConfigurationOfOptApiServices container		

4.7.3 PwmGroupId

Table 44 PwmGroupId

Name:	PwmGroupId		
Source:	This parameter is microcontroller specific		
File:	Pwm_17_Gtm_Cfg.h		
Range:	0 to xx Default : 0 (A consecutive value is calculated for each new Pwm Channel)		

Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This is the Id of the Pwm Group. This value will be assigned to the symbolic name derived of the PwmSyncGroup The Id value must be consecutive		
Scope:	Driver		
Dependency:	PwmSyncGrpUpdateDutiesApi in PwmConfigurationOfOptApiServices container		

4.8 Container: CommonPublishedInformation

It contains published information about vendor and versions.

4.8.1 ArMajorVersion

Table 45 ArMajorVersion

Name:	ArMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255 Default value : As per the supported Autosar version		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Major version number of AUTOSAR specification on which the appropriate implementation is based on.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.2 ArMinorVersion

Table 46 ArMinorVersion

Name:	ArMinorVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255 Default value : Minor version number of AUTOSAR specification on which the appropriate implementation is based on.		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.		

SFR Modified	None
Scope:	Driver
Dependency:	None

4.8.3 ArPatchVersion

Table 47 ArPatchVersion

Name:	ArPatchVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255 Default value : As per the supported Autosar version		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Patch level version number of AUTOSAR specification on which the appropriate implementation is based on.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.4 SwMajorVersion

Table 48 SwMajorVersion

Name:	SwMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255 Default value : Refer Pwm_17_Gtm_Cfg.h		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Major version number of the vendor specific implementation of the module.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.5 SwMinorVersion

Table 49 SwMinorVersion

Name:	SwMinorVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255		

	Default value : Refer Pwm_17_Gtm_Cfg.h		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Minor version number of the vendor specific implementation of the module.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.6 SwPatchVersion

Table 50 SwPatchVersion

Name:	SwMajorVersion		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values :0-255 Default value : Refer Pwm_17_Gtm_Cfg.h		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Patch level version number of the vendor specific implementation of the module.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.7 ModuleId

Table 51 ModuleId

Name:	ModuleId		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm.h		
Range:	Values :0-255 Default value : 121		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Module ID of PWM module from Module List		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.8 VendorId

Table 52 VendorId

Name:	VendorId		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm.h		
Range:	Values :0-255 Default value : 17		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Vendor ID of the PWM module according to the AUTOSAR vendor list.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.9 VendorApilnfix

Table 53 VendorApilnfix

Name:	VendorApilnfix		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm.xdm/Pwm_17_Gtm.bmd		
Range:	Gtm		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	In driver modules which can be instantiated several times on a single ECU then the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name.		
SFR Modified	None		
Scope:	Driver		
Dependency:	None		

4.8.10 Release

Table 54 Release

Name:	Release		
Source:	This is Infineon specific parameter.		
File:	Pwm_17_Gtm.xdm/Pwm_17_Gtm.bmd		
Range:	_TRICORE_TC275 _TRICORE_TC277 _TRICORE_TC264 _TRICORE_TC265 _TRICORE_TC298 _TRICORE_TC299		
Configuration Class	Pre-Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--

Description:	Aurix derivative used for the implementation.
SFR Modified	None
Scope:	Driver
Dependency:	None

4.9 Derived Configuration Parameters

The following parameters represent the additional information required by the Driver and they are automatically calculated by the tool.

Note: These derived parameters are **precompile** parameters and hence the parameters in the "Dependency" field should not be modified after the build process.

4.9.1 PwmMaxChannels

Table 55 PwmMaxChannels

Name:	PWM_MAX_CHANNELS		
Source:	This is a derived parameter based on the number of channels configured by the user.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: 0 to x		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The value of this parameter is automatically calculated. For instance if there are two configuration sets , if one configuration set has 5 channels and the other has 7 channels then this precompile parameter will hold a value 7.		
SFR Modified	None		
Scope:	Driver		
Dependency:	Pwm channel configurations across configuration sets.		

4.9.2 PwmMaxAsilChannels

Table 56 PwmMaxAsilChannels

Name:	PWM_MAX_ASIL_CHANNELS		
Source:	This is a derived parameter based on the number of channels configured for ASIL signal by the user.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: 0 to x		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The value of this parameter is automatically calculated. For instance if there are two configuration sets , if one configuration set has 5 ASIL channels and the other has 7 ASIL channels then this precompile parameter will hold a value 7.		

Name:	PWM_MAX_ASIL_CHANNELS
SFR Modified	None
Scope:	Driver
Dependency:	PwmSafetyEnable, Pwm ASIL channel configurations across configuration sets.

4.9.3 PwmMaxQmChannels

Table 57 PwmMaxQmChannels

Name:	PWM_MAX_QM_CHANNELS		
Source:	This is a derived parameter based on the number of channels configured for QM signal by the user.		
File:	Pwm_17_Gtm_Cfg.h		
Range:	Values: 0 to x		
Configuration Class:	Pre Compile	x	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The value of this parameter is automatically calculated. For instance if there are two configuration sets , if one configuration set has 5 QM channels and the other has 7 QM channels then this precompile parameter will hold a value 7.		
SFR Modified	None		
Scope:	Driver		
Dependency:	Pwm QM channel configurations across configuration sets.		

5 Published Parameters

This section describes the published parameters by PWM Driver.

5.1 VendorID

Table 58 VendorID

Syntax:	PWM_17_GTM_VENDOR_ID
Type:	Macro / uint16
File:	Pwm_17_Gtm.h
Value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list. Vendor is Infineon Technologies

5.2 ModuleID

Table 59 ModuleID

Syntax:	PWM_17_GTM_MODULE_ID
Type:	Macro / uint16

Syntax:	PWM_17_GTM_MODULE_ID
File:	Pwm_17_Gtm.h
Value:	121
Description:	This macro gives the PWM Driver module ID as described by AUTOSAR

5.3 InstanceID

Table 60 InstanceID

Syntax:	PWM_17_GTM_INSTANCE_ID
Type:	Macro / uint8
File:	Pwm_17_Gtm_Cfg.h
Value:	0
Description:	This macro gives the PWM Driver instance ID as described by AUTOSAR.

5.4 ARReleaseMajorVersion

Table 61 ARReleaseMajorVersion

Syntax:	PWM_17_GTM_AR_RELEASE_MAJOR_VERSION for AS4.0.x PWM_17_GTM_AR_MAJOR_VERSION for AS3.2.1
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Major version number of the AUTOSAR specification.

5.5 ARReleaseMinorVersion

Table 62 ARReleaseMinorVersion

Syntax:	PWM_17_GTM_AR_RELEASE_MINOR_VERSION for AS4.0.x PWM_17_GTM_AR_MINOR_VERSION for AS3.2.1
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Minor version number of the AUTOSAR specification.

5.6 ARReleaseRevisionVersion

Table 63 ARReleaseRevisionVersion

Syntax:	PWM_17_GTM_AR_RELEASE_REVISION_VERSION for AS4.0.x PWM_17_GTM_AR_PATCH_VERSION for AS3.2.1
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Revision version number of the AUTOSAR specification.

5.7 SWMajorVersion

Table 64 SWMajorVersion

Syntax:	PWM_17_GTM_SW_MAJOR_VERSION
----------------	-----------------------------

Syntax:	PWM_17_GTM_SW_MAJOR_VERSION
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver.

5.8 SWMinorVersion

Table 65 SWMinorVersion

Syntax:	PWM_17_GTM_SW_MINOR_VERSION
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Minor Version Number of the vendor specific implementation of the Driver.

5.9 SwPatchVersion

Table 66 SwPatchVersion

Syntax:	PWM_17_GTM_SW_PATCH_VERSION
Value:	Refer Pwm_17_Gtm_Cfg.h
Description:	Patch Version Number of the vendor specific implementation of the Driver.

6 API Specification

This chapter describes the API interfaces provided by PWM Driver.

6.1 API Type Definitions

6.1.1 Pwm_17_Gtm_ConfigType

Table 67 Type Definition Pwm_17_Gtm_ConfigType

Syntax:	Pwm_17_Gtm_ConfigType	
Type:	Struct	
File:	Pwm_17_Gtm.h	
Range:	const uint32 Marker	Holds the module id and instance id information; required only when PWM_SAFETY_ENABLE is STD_ON
	Pwm_MaxChannels (Pwm_17_Gtm_ChannelType)	Number of PWM Channels present in configuration.(PwmMaxChannels)
	const Pwm_17_Gtm_ChannelConfigType *ChannelConfigPtr	Pointer to Pwm_ChannelConfigType to hold the details of each configured PWM channels
	const Pwm_17_Gtm_GroupConfigType *GroupConfigPtr;	This member holds the details of the groups configured. This member is generated only when PWM_17_GTM_SYNCGRPUPDATEDUTIES_ENABLE is STD_ON.

	uint8 MaxGroups	Number of groups in configuration. This member is generated only when PWM_17_GTM_SYNCGRPUPDATEDUTIES_ENABLE is STD_ON.
Description:	This type definition is used to configure overall PWM configuration. The pointer to the object of this type is used in Pwm_17_Gtm_Init API to initialize the PWM Driver.	

6.1.2 Pwm_17_Gtm_ChannelConfigType

Table 68 Type Definition Pwm_17_Gtm_ChannelConfigType

Syntax:	Pwm_17_Gtm_ChannelConfigType	
Type:	Struct	
File:	Pwm_17_Gtm.h	
Range:	PwmNotification (Pwm_17_Gtm_NotifiPtrType)	Function Pointer to provide Notification.
	PwmChannelInfo (uint32)	PwmChannelInfo is a 32 bit member containing the following information (PwmReferenceChannel, PwmChannelClass, PwmIdleState, PwmPolarity, and PwmAssignedHwUnit).
	PwmPeriodDefault (Pwm_17_Gtm_PeriodType)	Default value for Period
	PwmDutycycleDefault (uint32)	Default value of duty cycle for the given channel
	PwmShiftValue (uint32)	Shift required in PWM_FIXED_PERIOD_SHIFTED Class channel.
	SafetyChannelSignal (uint8)	The parameter defines the Pwm channel is an ASIL signal or QM signal. This parameter is significant only if PwmSafetyEnable is set to True.
	SafetyChannelIndex(uint8)	The parameter holds the index for the ASIL signal or QM signal for which the channel is configured.
Description:	This is the type of data structure containing the configuration data of all the PWM Channels.	

6.1.3 Pwm_17_Gtm_ChannelType

Table 69 Pwm_17_Gtm_ChannelType

Syntax:	Pwm_17_Gtm_ChannelType
Type:	uint32
File:	Pwm_17_Gtm.h
Range:	0 to x
Description:	Specifies the identification (ID) for a Channel.

6.1.4 Pwm_17_Gtm_PeriodType

This type is used in API Pwm_17_Gtm_SetPeriodAndDuty.

Table 70 Type Definition Pwm_17_Gtm_PeriodType

Syntax:	Pwm_17_Gtm_PeriodType
Type:	Uint32
File:	Pwm_17_Gtm.h
Range:	0 to 0xFFFFFE
Description:	Definition of the period of a PWM channel.

6.1.5 Pwm_17_Gtm_OutputStateType

This type is used in the API Pwm_17_Gtm_GetOutputState.

Table 71 Type Definition Pwm_17_Gtm_OutputStateType

Syntax:	Pwm_17_Gtm_OutputStateType	
Type:	uint8	
File:	Pwm_17_Gtm.h	
Range:	PwmidleStateHigh	The PWM channel is in high state. The PWM channel output will be in high (3 or 5 V) state.
	PwmidleStateLow	The PWM channel is in low state. The PWM channel output will be in low (0V) state.
Description:	Output state of a PWM channel. Note that this will be read from the output state on the TOM/ATOM Channel connected to the port pin. This will not be read directly from the Port Pin. However both will be the same. This type is used to read PwmPolarity and PwmidleState.	

6.1.6 Pwm_17_Gtm_EdgeNotificationType

Table 72 Pwm_17_Gtm_EdgeNotificationType

Syntax:	Pwm_17_Gtm_EdgeNotificationType	
Type:	uint8	
File:	Pwm_17_Gtm.h	
Range:	PWM_RISING_EDGE	Notification will be called when a rising edge occurs on the PWM output signal.
	PWM_FALLING_EDGE	Notification will be called when a falling edge occurs on the PWM output signal.
	PWM_BOTH_EDGES	Notification will be called when both a rising edge and falling edge occur on the PWM output signal.
Description:	Definition of the type of edge notification of a PWM channel. The Edges are passed in Pwm_17_Gtm_EnableNotification API.	

6.1.7 Pwm_17_Gtm_ChannelClassType

Table 73 Type Definition Pwm_17_Gtm_ChannelClassType

Syntax:	Pwm_17_Gtm_ChannelClassType	
Type:	uint8	
File:	Pwm_17_Gtm.h	
Range:	PWM_VARIABLE_PERIOD	The Pwm channel has a variable period. The duty cycle and the period can be changed.
	PWM_FIXED_PERIOD	The PWM channel as a fixed period. Only the duty cycle can be changed.
	PWM_FIXED_PERIOD_SHIFT	The PWM channel as a fixed shifted period.
	PWM_FIXED_PERIOD_CENTER_ALIGNED	The AURIX family implementation also supports PWM_FIXED_PERIOD_CENTER_ALIGNED class. This will provide the duty cycle at middle of a Center align channel.
Description:	Class of a PWM Channel.	

6.1.8 Pwm_17_Gtm_NotifiPtrType

Table 74 Type Definition Pwm_17_Gtm_NotifiPtrType

Syntax:	Pwm_17_Gtm_NotifiPtrType
Type:	Function Pointer
File:	Pwm_17_Gtm.h
Range:	Function pointer
Description:	Channel notification function pointer

6.1.9 Pwm_17_Gtm_GroupConfigType

Table 75 Type Definition Pwm_17_Gtm_GroupConfigType

Syntax:	typedef struct Pwm_17_Gtm_GroupConfigType	
Type:	Struct	
File:	Pwm_17_Gtm.h	
Range:	uint32 PwmDutyBufMarker	This marker is used to check the validity of the Duty array passed. The Marker should have a value outside the range of normal duty. This member is enabled only when safety is ON. The recommended values are from 0xFF000000 to 0xFFFFFFFF.
	Pwm_17_Gtm_GrpChanConfigType *GrpChanConfigPtr	Pointer holding group channel configurations
	uint8 MaxGroupChannels	Number of channels in the group
	uint8 TomOrAtom	Indicating if the channel is Tom or Atom
	uint8 TomOrAtomModuleNumber	Module Number of the Tom/Atom selected
Description:	This type is used to hold different attributes of the Pwm group.	

6.1.10 Pwm_17_Gtm_GrpChanConfigType

Table 76 Type Definition Pwm_17_Gtm_GrpChanConfigType

Syntax:	Pwm_17_Gtm_GrpChanConfigType	
Type:	Struct	
File:	Pwm_17_Gtm.h	
Range	uint8 TomAtomNum	Module channel Number of the channel
	Pwm_17_Gtm_ChannelType	Pwm Channel Number in configuration
	PwmChannelNumber	
Description:	This type is used to hold different attributes of the channels in PWM group.	

6.2 API Functions

This chapter describes the API interfaces provided by PWM Driver.

6.2.1 Pwm_17_Gtm_Init

Table 77 Specification of API Pwm_17_Gtm_Init

Service Name:	Pwm_17_Gtm_Init	
Syntax:	void Pwm_17_Gtm_Init (const Pwm_17_Gtm_ConfigType *ConfigPtr)	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x00	
Sync / Async:	Synchronous	
Re-entrancy:	Non reentrant	
Parameters (in):	ConfigPtr - Pointer to configuration set.	
Parameters (out):	None	
Return Value:	None	
Description:	Service initializes the required hardware registers for each channel as per the configuration set.	
Caveats:	Service should not be called during running operation. Pwm_17_Gtm_Init shall be invoked by the user only after PLL setup (Using Mcu_DistributePllClock).	
Configuration:	None	
DET:	PWM_E_PARAM_CONFIG	ConfigPtr is NULL if PWM_PB_FIXEDADDR is OFF ConfigPtr is not equal to the Fixed address (&Pwm_ConfigRoot[0]) if PWM_PB_FIXEDADDR is ON
	PWM_E_ALREADY_INITIALIZED	Pwm_17_Gtm_Init() is called while PWM has already been initialized.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CONFIG	This error is reported when the Marker value in ConfigPtr does not match with the calculated value. This check is enabled only when Safety is enabled for PWM driver.

Implementation Comments:	- Mcu_Init() must be called prior to calling Pwm_17_Gtm_Init(). - The assigned HW resource should be available for allocation, i.e. not used for any other driver implementation. - The Reference channel for a PWM_FIXED_PERIOD_SHIFTED, and PWM_FIXED_PERIOD_CENTER_ALIGNED channel class should be given and the referred channel must be of class PWM_FIXED_PERIOD. - For a channel with zero period, the output of the channel is set to 0% duty, irrespective of the default duty value specified for the channel. - Support for FixedAddress: The Pwm config pointer (Pwm_kConfigPtr) is initialized to &Pwm_ConfigRoot[0] if the PWM_PB_FIXEDADDR is ON. When PWM_PB_FIXEDADDR is OFF, Pwm_kConfigPtr is initialized with the passed config pointer parameter in Pwm_17_Gtm_Init. - Support for Duty and Shift in ticks: When the PwmDutyShiftInTicks is true, the default dutycycle for all PWM channels and shift values for PWM_FIXED_PERIOD_SHIFTED channels should be entered as absolute ticks.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModelInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModelInitApiEnable is 'False'.

6.2.2 Pwm_17_Gtm_DeInit

Table 78 Specification of API Pwm_17_Gtm_DeInit

Service Name:	Pwm_17_Gtm_DeInit	
Syntax:	void Pwm_17_Gtm_DeInit (void)	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x01	
Sync / Async:	Synchronous	
Re-entrancy:	Non reentrant	
Parameters (in):	None	
Parameters (out):	None	
Return Value:	None	
Description:	Service to disable notifications and PWM interrupts. The state of the PWM output signals will be put in IDLE state.	
Caveats:	None	
Configuration:	PWM_DEINIT_API	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
DEM:	None	
Implementation Comments:	- For every channel, all the associated registers are put to reset value.	

Service Name:	Pwm_17_Gtm_DeInit
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeDeInitApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeDeInitApiEnable is 'False'.

6.2.3 Pwm_17_Gtm_SetDutyCycle

Table 79 Specification of API Pwm_17_Gtm_SetDutyCycle

Service Name:	Pwm_17_Gtm_SetDutyCycle	
Syntax:	<pre>void Pwm_17_Gtm_SetDutyCycle (Pwm_17_Gtm_ChannelType ChannelNumber, uint16 DutyCycle) Note: DutyCycle is uint32 if PwmDutyShiftInTicks is ON.(Not applicable for EP device)</pre>	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x02	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNumber	The associated Channel where the duty cycle needs to be changed. (Numeric Identifier)
	DutyCycle	Required duty cycle to be set. 0x0000 to 0x8000 when PwmDutyShiftInTicks is OFF 0 to Period value when PwmDutyShiftInTicks is ON
Parameters (out):	None	
Return Value:	None	
Description:	Service to set the required duty cycle for a given channel.	
Caveats:	None	
Configuration:	PWM_SET_DUTY_CYCLE_API	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum

Service Name:	Pwm_17_Gtm_SetDutyCycle	
		number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
	PWM_E_DUTY_OUT_OF_RANGE	If PWM_DUTY_SHIFT_IN_TICKS is OFF then if Duty > 0x8000 then error is reported. if PWM_DUTY_SHIFT_IN_TICKS is ON then If TOM module then if Duty > 0xFFFFE or If ATOM module then if Duty > 0xFFFFFE then error is reported. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	- For a channel with period 0, the output of the channel is set to 0% duty, irrespective of the specified duty cycle value. - When the PwmDutyShiftInTicks is OFF, the dutycycle should be passed as percentage, relative to period. In this case, the range will be 0x0 – 0% 0x8000 – 100% - When the PwmDutyShiftInTicks is OFF, the dutycycle should be passed as absolute ticks. In this case, the range will be 0 – 0% Value same as period value of the channel (for Shift, Center Aligned and Zero Shift Channels, it is period of reference channel) – 100%	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.4 Pwm_17_Gtm_SetPeriodAndDuty

Table 80 Specification of API Pwm_17_Gtm_SetPeriodAndDuty

Service Name:	Pwm_17_Gtm_SetPeriodAndDuty
Syntax:	<pre>void Pwm_17_Gtm_SetPeriodAndDuty (Pwm_17_Gtm_ChannelType ChannelNumber, Pwm_17_Gtm_PeriodType Period, uint16 DutyCycle) </pre> Note: DutyCycle is uint32 if PwmDutyShiftInTicks is ON. (Not applicable

Service Name:	Pwm_17_Gtm_SetPeriodAndDuty (for EP device)	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x03	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant (for different channels)	
Parameters (in):	ChannelNumber	The associated channel where the duty cycle needs to be changed. (Numeric Identifier)
	Period	Period of PWM Signal (0 to 0xFFFFFE for ATOM and 0 to 0xFFFE for TOM) In Autosar mode the period supported is 0xFFFE for both TOM and ATOM
	DutyCycle	- Required duty cycle to be set. 0x0000 to 0x8000 when PwmDutyShiftInTicks is OFF 0 to Period value when PwmDutyShiftInTicks is ON
Parameters (out):	None	
Return Value:	None	
Description:	Service to set the required duty cycle and period for a given channel.	
Caveats:	Applicable to only PWM_VARIABLE_PERIOD class channels.	
Configuration:	PWM_SET_PERIOD_AND_DUTY_API	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
	PWM_E_PERIOD_UNCHANGEABLE	Channel class type is not equal to PWM_VARIABLE_PERIOD.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
	PWM_E_DUTY_OUT_OF_RANGE	If PWM_DUTY_SHIFT_IN_TICKS is OFF then if DutyCycle > 0x8000 then error is reported. if PWM_DUTY_SHIFT_IN_TICKS is ON then If TOM module then if DutyCycle > 0xFFFE or If ATOM module then if DutyCycle > 0xFFFFFE then error is reported. This check is enabled only when Safety is enabled for PWM driver.

Service Name:	Pwm_17_Gtm_SetPeriodAndDuty	
	PWM_E_PERIOD_OUT_OF_RANGE	If TOM module then if Period > 0xFFFFE or If ATOM module then if Period > 0xFFFFFE then error is reported. This check is enabled only when Safety is enabled for PWM driver
Implementation Comments:	- When the period of a channel is zero, the output of the channel is set to 0% duty, irrespective of the duty value specified for the channel. - When the PwmDutyShiftInTicks is OFF, the dutycycle should be passed as percentage, relative to period. In this case, the range will be 0x0 – 0% 0x8000 – 100% - When the PwmDutyShiftInTicks is OFF, the dutycycle should be passed as absolute ticks. In this case, the range will be 0 – 0% Value same as Period – 100%	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.5 Pwm_17_Gtm_SetOutputToldle

Table 81 Specification of API Pwm_17_Gtm_SetOutputToldle

Service Name:	Pwm_17_Gtm_SetOutputToldle	
Syntax:	<pre>void Pwm_17_Gtm_SetOutputToldle (Pwm_17_Gtm_ChannelType ChannelNumber)</pre>	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x04	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNumber	The associated Channel where the Output state needs to be changed. (Numeric Identifier)
Parameters (out):	None	
Return Value:	None	
Description:	Service to set immediately the output state to IDLE level.	
Caveats:	None	
Configuration:	PWM_SET_OUTPUT_TO_IDLE_API	

Service Name:	Pwm_17_Gtm_SetOutputTogle	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	- As per Bugzilla 11802 discussion, this API is valid for any channel class.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.6 Pwm_17_Gtm_GetOutputState

Table 82 Specification of API Pwm_17_Gtm_GetOutputState

Service Name:	Pwm_17_Gtm_GetOutputState	
Syntax:	<pre>void Pwm_17_Gtm_GetOutputState (Pwm_17_Gtm_ChannelType ChannelNumber)</pre>	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x05	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNumber	The associated Channel. (Numeric Identifier)
Parameters (out):	None	
Return Value:	PWM_HIGH	Current state of Channel Output line is High
	PWM_LOW	Current state of channel output line is Low
Description:	Service to read the output state of the port pin connected to the channel.	
Caveats:	None	
Configuration:	PWM_GET_OUTPUT_STATE_API	

Service Name:	Pwm_17_Gtm_GetOutputState	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	- The output will be read from the TOM/ATOM Channel output state and not from the port pin.	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.7 Pwm_17_Gtm_DisableNotification

Table 83 Specification of API Pwm_17_Gtm_DisableNotification

Service Name:	Pwm_17_Gtm_DisableNotification	
Syntax:	<pre>void Pwm_17_Gtm_DisableNotification (Pwm_17_Gtm_ChannelType ChannelNumber)</pre>	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x06	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNumber	The associated Channel (Numeric Identifier)
Parameters (out):	None	
Return Value:	None	
Description:	Service to disable the notification edges of a PWM channel.	
Caveats:	None	
Configuration:	PWM_NOTIFICATION_SUPPORTED	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.

	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	- The service is feasible only when any of the following is true: PwmDutycycleUpdatedEndPeriod is ON OR PwmPeriodUpdatedEndPeriod is ON OR PwmChannelCoherentSelection is ON and PwmCoherentUpdate is ON	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.8 Pwm_17_Gtm_EnableNotification

Table 84 Specification of API Pwm_17_Gtm_EnableNotification

Service Name:	Pwm_17_Gtm_EnableNotification	
Syntax:	<pre>void Pwm_17_Gtm_EnableNotification (Pwm_17_Gtm_ChannelType ChannelNumber, Pwm_17_Gtm_EdgeNotificationType Notification)</pre>	
Source	This API is defined by the AUTOSAR. Please refer [1][2]	
Service ID:	0x07	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant (for different channels)	
Parameters (in):	ChannelNumber	The associated Channel (Numeric Identifier)
	Notification	Type of Notification: PWM_RISING_EDGE, PWM_FALLING_EDGE, PWM_BOTH_EDGES
Parameters (out):	None	
Return Value:	None	

Service Name:	Pwm_17_Gtm_EnableNotification	
Description:	Service to enable the notification of a given channel.	
Caveats:	None	
Configuration:	PWM_NOTIFICATION_SUPPORTED	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set.
DEM:	None	
Safety Error Reporting	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
	PWM_E_PARAM_EDGE	If the Notification passed is not within the range then this error is reported. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	- Notification is not enabled for PWM channels with 0% and 100% duty cycle	
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.	

6.2.9 Pwm_17_Gtm_GetVersionInfo

Table 85 Specification of API Pwm_17_Gtm_GetVersionInfo

Service Name:	Pwm_17_Gtm_GetVersionInfo
Syntax:	<pre>void Pwm_17_Gtm_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)</pre>
Source	This API is defined by the AUTOSAR. Please refer [1][2]
Service ID:	0x08
Sync / Async:	Synchronous
Re-entrancy:	Non reentrant
Parameters (in):	None
Parameters (out):	VersionInfoPtr: Pointer to store the Driver Version Information.
Return Value:	None

Description:	Service to return the version information of the Driver. Pwm_17_Gtm_GetVersionInfo is implemented as macro in Pwm_17_Gtm.h	
Caveats:	None	
Configuration:	PWM_VERSION_INFO_API	
DET:	PWM_E_UNINIT	Pwm_17_Gtm_Init is not called.
DEM:	None	
Implementation Comments:	- None	
I/O Mode:	This API can be invoked when the CPU is in either User-0 Mode or User-1 Mode or Supervisor Mode.	

6.2.10 Pwm_17_Gtm_SyncGrpUpdateDuties

Table 86 Specification of API Pwm_17_Gtm_SyncGrpUpdateDuties

Service Name:	Pwm_17_Gtm_SyncGrpUpdateDuties		
Syntax:	Std_ReturnType Pwm_17_Gtm_SyncGrpUpdateDuties(uint8 PwmGroupId,uint32* PwmChDutyPtr)		
Source	This is a vendor defined API.		
Service ID:	0x15		
Sync/Async:	Synchronous		
Reentrancy:	Reentrant		
Parameters (in):	PwmGroupId	UInt8	Numeric identifier of the PWM group
	PwmChDutyPtr	uint32 *	Pointer to array holding duty cycle values for group of channels
Parameters (out):	-	-	.
Return value:	E_OK / E_NOK		Status indicating API executed properly or not
Description:	Service sets the duty cycle for Multiple PWM channels synchronously		
Preconditions:	Pwm_17_Gtm_Init should be called.		
Timing Constraints:	None		
Caveats:	none		
Configuration:	PwmDevErrorDetect PwmNotificationSupported PwmPolarity PwmChannelClass PwmReferenceChannel PwmShiftValue PwmAssignedHwUnit PwmCoherentUpdate PwmSyncGrpUpdateDutiesApi		
Error Handling:	DET: PWM_E_UNINIT PWM_E_GROUPID, PWM_E_PARAM_BUFFER_MARKER, PWM_E_DUTY_OUT_OF_RANGE		

Implementation Comments:	The passed DutyCycle in the Array should be in ticks when PwmDutyShiftInTicks is ON. In case PwmDutyShiftInTicks is OFF, the DutyCycle should be passed as percentage, relative to period. Channels in the group should be in the same TGC. Only in case of Safety is ON only then PWMDutycycleBufferMarker is checked. Limitaions applicable different types of channels like shifted, center aligned etc. are applicable to channels in the groups based on their respective type. That is if channel is shifted then limitations of shifted channels are applicable to the respective channel. Please refer 10.2 for more details.
I/O Mode:	This API can be invoked when the CPU is in User-0 Mode provided - configuration parameter PwmRunningInUser0Mode is 'True' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in User-1 Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'True'. This API can be invoked when the CPU is in Supervisor Mode provided - configuration parameter PwmRunningInUser0Mode is 'False' and PwmUserModeRuntimeApiEnable is 'False'.

6.3 Interrupt Handling

The interrupt service routine in PWM Driver is used only for Notification purpose.

6.3.1 Pwm_17_Gtm_Isr

Table 87 Specification of API Pwm_17_Gtm_Isr

Service Name:	Pwm_17_Gtm_Isr	
Syntax:	<pre>void Pwm_17_Gtm_Isr (uint8 ChannelNumber)</pre>	
Service ID:	None	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNo	The associated Channel (Numeric Identifier)
Parameters (out):	None	
Return Value:	None	
Description:	All the Timer cells are mapped to this Interrupt Service Routine. The Channel number will be captured during Configuration file generation and the same will be passed to this routine for Notification purpose.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Safety Error Reporting	PWM_E_WRONG_NOTIF	For a hardware unit if ISR is called

Service Name:	Pwm_17_Gtm_Isr	
Syntax:	<pre>void Pwm_17_Gtm_Isr (uint8 ChannelNumber)</pre>	
Service ID:	None	
Sync / Async:	Synchronous	
Re-entrancy:	Reentrant	
Parameters (in):	ChannelNo	The associated Channel (Numeric Identifier)
Parameters (out):	None	
Return Value:	None	
Description:	All the Timer cells are mapped to this Interrupt Service Routine. The Channel number will be captured during Configuration file generation and the same will be passed to this routine for Notification purpose.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
		even if notification is not enabled for the channel then this error is reported. This check is enabled only when Safety is enabled for PWM driver.
	PWM_E_PARAM_CHANNEL	The Channel Number is Invalid. ChannelNumber >= Maximum number of channels of the configuration set. This check is enabled only when Safety is enabled for PWM driver.
Implementation Comments:	None	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

General Notes:

There are 24 SRNs (Service Request Nodes) to service GTM TOM Channel interrupt requests (All Variants)
There are 20 SRNs to service GTM ATOM Channel interrupt requests (All Variants)

- Each SRN should be provided a valid priority in the supplied Gtm_Irq.c and make sure that the corresponding Interrupt sources are enabled.
- The following table shows the SRN Number and the associated TOM/ATOM Channel resource.

Table 88 Service Request Node – Hardware Resource GTM

Hardware Resource		SRN Number
TOM0	CHANNEL0, CHANNEL1	SRC_GTM TOM00
	CHANNEL2, CHANNEL3	SRC_GTM TOM01
	CHANNEL4, CHANNEL5	SRC_GTM TOM02
	CHANNEL6, CHANNEL7	SRC_GTM TOM03

Hardware Resource		SRN Number
	CHANNEL8, CHANNEL9	SRC_GTMTOM04
	CHANNEL10, CHANNEL11	SRC_GTMTOM05
	CHANNEL12, CHANNEL13	SRC_GTMTOM06
	CHANNEL14, CHANNEL15	SRC_GTMTOM07
TOM1	CHANNEL0, CHANNEL1	SRC_GTMTOM10
	CHANNEL2, CHANNEL3	SRC_GTMTOM11
	CHANNEL4, CHANNEL5	SRC_GTMTOM12
	CHANNEL6, CHANNEL7	SRC_GTMTOM13
	CHANNEL8, CHANNEL9	SRC_GTMTOM14
	CHANNEL10, CHANNEL11	SRC_GTMTOM15
	CHANNEL12, CHANNEL13	SRC_GTMTOM16
	CHANNEL14, CHANNEL15	SRC_GTMTOM17
TOM2	CHANNEL0, CHANNEL1	SRC_GTMTOM20
	CHANNEL2, CHANNEL3	SRC_GTMTOM21
	CHANNEL4, CHANNEL5	SRC_GTMTOM22
	CHANNEL6, CHANNEL7	SRC_GTMTOM23
	CHANNEL8, CHANNEL9	SRC_GTMTOM24
	CHANNEL10, CHANNEL11	SRC_GTMTOM25
	CHANNEL12, CHANNEL13	SRC_GTMTOM26
	CHANNEL14, CHANNEL15	SRC_GTMTOM27
TOM3	CHANNEL0, CHANNEL1	SRC_GTMTOM30
	CHANNEL2, CHANNEL3	SRC_GTMTOM31
	CHANNEL4, CHANNEL5	SRC_GTMTOM32
	CHANNEL6, CHANNEL7	SRC_GTMTOM33
	CHANNEL8, CHANNEL9	SRC_GTMTOM34
	CHANNEL10, CHANNEL11	SRC_GTMTOM35
	CHANNEL12, CHANNEL13	SRC_GTMTOM36
	CHANNEL14, CHANNEL15	SRC_GTMTOM37
TOM3	CHANNEL0, CHANNEL1	SRC_GTMTOM40
	CHANNEL2, CHANNEL3	SRC_GTMTOM41
	CHANNEL4, CHANNEL5	SRC_GTMTOM42
	CHANNEL6, CHANNEL7	SRC_GTMTOM43
	CHANNEL8, CHANNEL9	SRC_GTMTOM44
	CHANNEL10, CHANNEL11	SRC_GTMTOM45
	CHANNEL12, CHANNEL13	SRC_GTMTOM46
	CHANNEL14, CHANNEL15	SRC_GTMTOM47
ATOM0	CHANNEL0, CHANNEL1	SRC_GTMATOM00
	CHANNEL2, CHANNEL3	SRC_GTMATOM01
	CHANNEL4, CHANNEL5	SRC_GTMATOM02
	CHANNEL6, CHANNEL7	SRC_GTMATOM03
ATOM1	CHANNEL0, CHANNEL1	SRC_GTMATOM10
	CHANNEL2, CHANNEL3	SRC_GTMATOM11
	CHANNEL4, CHANNEL5	SRC_GTMATOM12
	CHANNEL6, CHANNEL7	SRC_GTMATOM13

Hardware Resource		SRN Number
ATOM2	CHANNEL0, CHANNEL1	SRC_GTMATOM20
	CHANNEL2, CHANNEL3	SRC_GTMATOM21
	CHANNEL4, CHANNEL5	SRC_GTMATOM22
	CHANNEL6, CHANNEL7	SRC_GTMATOM23
ATOM3	CHANNEL0, CHANNEL1	SRC_GTMATOM00
	CHANNEL2, CHANNEL3	SRC_GTMATOM31
	CHANNEL4, CHANNEL5	SRC_GTMATOM32
	CHANNEL6, CHANNEL7	SRC_GTMATOM33
ATOM4	CHANNEL0, CHANNEL1	SRC_GTMATOM40
	CHANNEL2, CHANNEL3	SRC_GTMATOM41
	CHANNEL4, CHANNEL5	SRC_GTMATOM42
	CHANNEL6, CHANNEL7	SRC_GTMATOM43
ATOM5	CHANNEL0, CHANNEL1	SRC_GTMATOM50
	CHANNEL2, CHANNEL3	SRC_GTMATOM51
	CHANNEL4, CHANNEL5	SRC_GTMATOM52
	CHANNEL6, CHANNEL7	SRC_GTMATOM53
ATOM6	CHANNEL0, CHANNEL1	SRC_GTMATOM60
	CHANNEL2, CHANNEL3	SRC_GTMATOM61
	CHANNEL4, CHANNEL5	SRC_GTMATOM62
	CHANNEL6, CHANNEL7	SRC_GTMATOM63
ATOM7	CHANNEL0, CHANNEL1	SRC_GTMATOM70
	CHANNEL2, CHANNEL3	SRC_GTMATOM71
	CHANNEL4, CHANNEL5	SRC_GTMATOM72
	CHANNEL6, CHANNEL7	SRC_GTMATOM73
ATOM8	CHANNEL0, CHANNEL1	SRC_GTMATOM80
	CHANNEL2, CHANNEL3	SRC_GTMATOM81
	CHANNEL4, CHANNEL5	SRC_GTMATOM82
	CHANNEL6, CHANNEL7	SRC_GTMATOM83

6.4 Exclusive Area (EA) Information

Not applicable for PWM driver.

7 Data Description

The below table lists all the global and static variables used in the Pwm Driver:

Table 89 Eg: Global Data Description

Parameter	Type/Range	Description
Pwm_kConfigPtr	Pwm_17_Gtm_ConfigType*	To store Pwm_Configuration start address. Note: When the PwmPBFixedAddress is ON, this is already initialized with &Pwm_ConfigRoot[0]
Pwm_GlobChn	uint32	Array of PWM_MAX_CHANNELS. Each element stores the channel information

Parameter	Type/Range	Description
		which includes Channel type, Notification type, notification status and duty status for 0% or 100% dutycycle.
Pwm_GlobChn_Asil	uint32	Array of PWM_MAX_ASIL_CHANNELS. Each element stores the channel information which includes Channel type, Notification type, notification status and duty status for 0% or 100% dutycycle. Significant only if safety features are enabled for PWM
Pwm_GlobChn_Qm	uint32	Array of PWM_MAX_QM_CHANNELS. Each element stores the channel information which includes Channel type, Notification type, notification status and duty status for 0% or 100% dutycycle. Significant only if safety features are enabled for PWM
Pwm_CurrentPeriodVal_Asil	Uint32	Array to store the period in case of Variable Period type of Channels for ASIL channels
Pwm_CurrentPeriodVal_Qm	Uint32	Array to store the period in case of Variable Period type of Channels for QM channels

8 Error Classification

Table 90 Error Classification

Error Code	Error	Relevance	Value
PWM_E_PARAM_CONFIG	The parameter passed is a NULL pointer if PWM_PB_FIXEDADDR is OFF. The parameter passed is not equal to &Pwm_ConfigRoot[0], when PWM_PB_FIXEDADDR is ON	Development	0x10
PWM_E_UNINIT	API service used without Driver initialization	Development	0x11
PWM_E_PARAM_CHANNEL	Invalid Channel Number	Development	0x12
PWM_E_PERIOD_UNCHANGEABLE	API Service Pwm_17_Gtm_SetPeriodAndDuty is called for an invalid channel class. (Class other than PWM_VARIABLE_PERIOD)	Development	0x13
PWM_E_ALREADY_INITIALIZED	API Service Pwm_17_Gtm_Init() is called while the PWM has already been initialized	Development	0x14

Table 91 Safety Error Classification

Error Code	Error	Relevance	Value
PWM_E_PARAM_CONFIG	The Marker value passed in config pointer is not equal to the calculated Marker.	Safety	0x10
PWM_E_PARAM_CHANNEL	Invalid Channel Number	Safety	0x12
PWM_E_PARAM_EDGE	API service called with Notification edge value out of range	Safety	0x16
PWM_E_PERIOD_OUT_OF_RANGE	API service called with Period out of range	Safety	0x17
PWM_E_DUTY_OUT_OF_RANGE	API service called with Duty out of range	Safety	0x18
PWM_E_WRONG_NOTIF	ISR is called for channel which is not configured for Notification	Safety	0x19

9 Example Usage

This chapter describes how the PWM Driver can be configured and how to use different API provided by it.

9.1 Configuration of the Driver

To configure PWM Driver the following guidelines should be followed properly.

1. MCU Driver: Configure the System Clock.
2. GTM Driver: Port pin and Clock for the channel and the Clock prescale values are configured here

3. PORT Driver: For all the port pins that were used in GTM Driver as output pins, configure the same in PORT Driver. Set the Direction to output and Port function as ALT1 / ALT2 / ALT3 depending on Pin Group under each TOM/ATOM Channel. This can be easily identified in GTM Driver configuration or refer GTM User manual.
4. Interrupt Priority Initialization: The priority of interrupts should be configured in IRQ Driver and appropriate priorities should be given. The TOM/ATOM Interrupt map is given in section Table 88
5. PWM Driver: Select the required API configuration and choose channel dependant parameters like Polarity, Idle State, Period, Duty cycle etc.

9.1.1 Sample Configuration of the Driver

Requirement:

The user requires 4 channels, they are one fixed period channel, one fixed period center aligned channel, one fixed period shifted channel and one variable period channel.

To change the duty cycle and period effectively the user decides to set End period parameters to ON. The user does not wish to include Pwm_17_Gtm_DeInit API. Also, the user decides to enter dutycycle and shift values as percentage. User also disables the safety features.

In the configuration tool: Under PwmModuleDescription/PwmGeneral the following values are set.

PwmDevErrorDetect -> false

PwmChannelCoherentSelection -> false

PwmDutycycleUpdatedEndperiod -> true

PwmPeriodUpdatedEndperiod -> true

PwmNotificationSupported -> true

Under PwmModuleDescription/ PwmConfigurationOfOptApiServices the following values are set

PwmVersionInfoApi -> true

PwmDeInitApi -> false

PwmGetOutputState -> true

PwmSetDutyCycle -> true

PwmSetOutputToldle -> true

PwmSetPeriodAndDuty -> true

PwmDutyShiftInTicks -> false

The header file is shown in **Code Listing 1**. Only the important settings are listed below

Code Listing 1 Pwm Configuration Header file

001:	#define PWM_DEV_ERROR_DETECT	(OFF)
002:	#define PWM_DEINIT_API	(OFF)
003:	#define PWM_NOTIFICATION_SUPPORTED	(ON)
004:	#define PWM_DUTYCYCLE_UPDATED_ENDPERIOD	(ON)
005:	#define PWM_DUTY_PERIOD_UPDATED_ENDPERIOD	(ON)
006:	#define PWM_VERSION_INFO_API	(ON)
007:	#define PWM_GET_OUTPUT_STATE_API	(ON)
008:	#define PWM_SET_DUTY_CYCLE_API	(ON)
009:	#define PWM_SET_OUTPUT_TO_IDLE_API	(ON)
010:	#define PWM_SET_PERIOD_AND_DUTY_API	(ON)
011:	#define PWM_SAFETY_ENABLE	(STD_OFF)

Code Listing 1 Pwm Configuration Header file

```
012:  #define PWM_DUTY_SHIFT_IN_TICKS                (OFF)
013:  #define PWM_MAX_CHANNELS                        ((Pwm_17_Gtm_ChannelType) 4)
014:  #define PWM_MAX_ASIL_CHANNELS    (0)
015:  #define PWM_MAX_QM_CHANNELS    (4)
```

PwmChannel Configuration should be configured as follows

(Read in the following order PwmChannelClass, PwmIdleState, PwmPolarity, PwmNotification, PwmChannelId, PwmDutycycleDefault, PwmShiftValue, PwmPeriodDefault, PwmReferenceChannel, PwmAssignedHwUnit).

PwmChannel0: PWM_FIXED_PERIOD, PwmIdleStateLow, PwmPolarityLow, "NULL", 0, 16384, 0, 48. The reference is left blank. The last TOM/ATOM Channel should be chosen so ATOM0 Channel0 is chosen.

PwmChannel2: PWM_FIXED_PERIOD_CENTER_ALIGNED, PwmIdleStateLow, PwmPolarityLow, "NULL", 2, 16384, 0, 0. The reference is given to PwmChannel0. The last TOM/ATOM Channel should be chosen so ATOM0 Channel1 is chosen. (Note: Period value need not be given here)

PwmChannel3: PWM_FIXED_PERIOD_SHIFTED, PwmIdleStateLow, PwmPolarityLow, "NULL", 3, 16384, 3276, 0. The reference is given to PwmChannel0. The last TOM/ATOM Channel should be chosen so ATOM0 Channel2 is chosen. (Note: Shift is given as 3276, Period need not be given).

PwmChannel4: PWM_VARIABLE_PERIOD, PwmIdleStateHigh, PwmPolarityLow, "PWM_NOTIFICATION_4", 4, 8192, 0, 64. The reference is left blank. The last TOM/ATOM Channel should be chosen so ATOM0 Channel3 is chosen. (Note: Shift value is not required here. Period is given as 65 ticks, Notification function is used here.).

The PwmNotification is required only for Channel4 (PWM_VARIABLE_PERIOD) hence the notification is enabled only for Channel4 by the Pwm Driver.

Assuming the TOM/ATOM Channel resources are free, the next step is to go into GtmConfiguration and configure the TOM/ATOM Channel as follows.

GtmClock is configured with required Divider value. As a simple example, if GTM clock is running at 60 MHz GTM configurable clock has divider value of 11). The formula is (GTM Clock Value / (11 +1)). The following configuration should be done in Gtm_PBCfg.c

The connection of Pins to GTM resources is done in GTM Driver.

The following configuration should be done in Pwm_17_Gtm_PBCfg.c, only the first channel is shown here for sample

Code Listing 2 Pwm Channel Configuration

```
001:  const Pwm_17_Gtm_ChannelConfigType Pwm_ChannelConfiguration[] = {
002:  /*
003:   * Channel Number          : 0
004:   * Channel Symbolic Name   : PwmChannel0
005:   * Channel Class           : PWM_FIXED_PERIOD
006:   */
007:  {
008:    #if (PWM_NOTIFICATION_SUPPORTED == ON)
009:      (Pwm_17_Gtm_NotifiPtrType) NULL_PTR,
010:    #endif
011:    Pwm_lConfigChannel (1U, /* if it is fixed channel is it also a reference
                                channel to shift/centre aligned channel */
012:                       PWM_NOT_REFERENCED, /* reference to the channel number */
013:                       PWM_FIXED_PERIOD, /* channel class */
014:                       STD_ON, /* Coherency */
015:                       PWM_LOW, /* channel Idle state */
```

Code Listing 2 Pwm Channel Configuration

```
016:          PWM_LOW, /* channel polarity */
017:          GTM_ATOM0_CHANNEL1 /* channel hardware resource */
018:      )
019:  (uint32)45, /* Default Period */
020:  (uint32)16384, /* Default Duty Cycle */
021:  (uint16)0, /* Shift Value */
022:  }
```

9.2 Driver Initialization

The **Code Listing 3** depicts the steps involved in the initialization of the PWM Driver. Note: User must take care that the Pwm_17_Gtm_Init API is called before using any other API provided by PWM Driver. Also Mcu_Init must be called before Pwm_17_Gtm_Init.

Code Listing 3 Driver Initialization

```
001:      Mcu_Init(&Mcu_ConfigRoot[0]);
002:      Mcu_InitClock(0);
003:      while(Mcu_GetPllStatus(0) == 0);
004:      Mcu_DistributePllClock();
005:      /* Configure Interrupt priority */
006:      IrqGtm_Init();
007:      /* Port Initialization */
008:      Port_Init(&Port_ConfigRoot[0]);
009:      /* Pwm Initialization */
010:      Pwm_17_Gtm_Init(&Pwm_ConfigRoot[0]);
011:      /* Enable All Interrupts */
012:      EnableAllInterrupts();
013:      /* Continue with the Driver */
```

After the call of Pwm_17_Gtm_Init the channels will start producing the PWM output signals.

9.3 Driver Init check

The **Code Listing 4** has an example implementation of PWM Init check API. Global variable initializations, macros used in demo code, local functions used are present in next code listing.

Note:

1. User shall take care that the Pwm_17_Gtm_Init API is called before calling the Pwm Init Check API.
2. In the example a global variable is declared "DemoGlobalConfigPtr" to hold the address of the config pointer passed to the InitCheck API. Users can also replace the DemoGlobalConfigPtr with the address of the Pwm configuration passed to the Init check function.
3. For verification of global variables initialized in Pwm Init function, the address of the global variables can be assigned to pointers and used further as shown in example. This is required since the global variables are declared as static in Pwm driver code.
4. The code listing is only an example of the Init check and shall be appropriately modified and tested if intended to be used in production/safety application.

Code Listing 4 Driver Initialization

```
001:  /*****
002:  ** Description      :      This routine verifies the initialization the PWM      **
003:  **                  driver.It is assumed that Gtm Driver is initialised      **
```

Code Listing 4 Driver Initialization

```

004:  **                               before calling this function.                               **
005:  **                               Similarly Pwm_17_Gtm_Init is also initialised before         **
006:  **                               this function is called.                               **
007:  **                                                                                               **
008:  *****/
009:  static Std_ReturnType Pwm_InitCheck (const Pwm_17_Gtm_ConfigType *ConfigPtr)
010:  {
011:      Std_ReturnType ErrorFlag = E_OK;
012:      if (NULL_PTR == ConfigPtr)
013:      {
014:          ErrorFlag = E_NOT_OK;
015:      }
016:      else
017:      {
018:          ErrorFlag = Pwm_lGlobalVariableCheck(ConfigPtr);
019:          if(ErrorFlag != E_NOT_OK)
020:          {
021:              ErrorFlag = Pwm_lRegisterCheck(ConfigPtr);
022:          }
023:      }
024:      return(ErrorFlag);
025:  }
026:
027:  /******
028:      Example of Init check usage
029:  *****/
030:  /* Pwm_17_Gtm_Init should be executed before this steps*/
031:      DemoGlobalConfigPtr = (&Pwm_ConfigRoot[0]);
032:      Test_after = Pwm_InitCheck(&Pwm_ConfigRoot[0]);
033:      if(Test_after = E_OK)
034:          /* Continue with the Driver */
035:
036:

```

After verification of the PWM initialization other functionalities can be continued.

Code Listing 5 Global variable declaration

```

001:  /******
002:      Address of the global variable assigned to pointers
003:  *****/
004:
005:  /******
006:      ASIL variables are present in the section
007:      PWM_17_GTM_START_SEC_ASIL0_VAR_32BIT. The address of the variable may
008:      differ based on configuration.
009:  *****/
010:  /* pointer to hold Pwm_CurrentPeriodVal_Asil variable*/
011:  uint32 * PtrPeriod_Asil = (uint32 *)0xb0000068;
012:  /* pointer to hold Pwm_GlobChn_Asil variable*/

```

Code Listing 5 Global variable declaration

```

011:  uint32 * PtrGlobChn_Asil = (uint32 *)0xb000007c;
012:  /*****
013:  End of ASIL variables
014:  *****/
015:
016:  /*****
017:  QM variables are present in the section
      PWM_17_GTM_START_SEC_VAR_FAST_32BIT. The address of the variable may
      differ based on configuration.
018:  *****/
019:  /* pointer to hold Pwm_CurrentPeriodVal_Qm variable*/
020:  uint32 * PtrPeriod_Qm = (uint32 *)0xb00009f8;
021:  /* pointer to hold Pwm_GlobChn_Qm variable*/
022:  uint32 * PtrGlobChn_Qm = (uint32 *)0xb0000a04;
023:  /*****
024:  End of QM variables
025:  *****/
026:
027:  /*****
028:  This variable is present in the section
      PWM_17_GTM_START_SEC_VAR_8BIT */
029:  /*****
030:  *****/
031:  /* pointer to hold Pwm_InitStatus variable*/
032:  uint8 * Pwm_DemoInitStatus = (uint8 *)0xb0000090;
033:  /*****
034:  End of init status variable
035:  *****/
036:
037:  /*****
038:  Note:
039:  The address of the global variable may vary based on configuration
040:  and compiler. User to take care of assigning correct address to the
041:  pointers
042:  *****/
043:
044:  /*Global variable declaration to hold the configuration*/
045:  static const Pwm_17_Gtm_ConfigType * DemoGlobalConfigPtr;

```

Code Listing 6 Macros used in code

```

001:  /*****
002:  Macros used in the demo code
003:  *****/
004:
005:  #define PWM_EXTRACT_CHAN_KIND                (0x7U)
006:  #define PWM_DUTY_0_OR_100_UPDATE_FLGPOS     (7U)
007:  #define PWM_BIT_SET                          (1U)
008:  #define PWM_DEMO_INITIALIZED                 (1U)
009:

```

Code Listing 6 Macros used in code

```

010:  #define PWM_GTM_ATOM_CH_CTRL_ABM_VAL (0u)
011:  #define PWM_GTM_ATOM_CH_CTRL_ABM_MSK (0x1u)
012:  #define PWM_GTM_ATOM_CH_CTRL_ABM_OFF (27u)
013:  #define PWM_GTM_ATOM_CH_CTRL_ACB_VAL (0u)
014:  #define PWM_GTM_ATOM_CH_CTRL_ACB_MSK (0x1fu)
015:  #define PWM_GTM_ATOM_CH_CTRL_ACB_OFF (4u)
016:  #define PWM_GTM_ATOM_CH_CTRL_ARU_EN_VAL (0u)
017:  #define PWM_GTM_ATOM_CH_CTRL_ARU_EN_MSK (0x1u)
018:  #define PWM_GTM_ATOM_CH_CTRL_ARU_EN_OFF (3u)
019:  #define PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_VAL (0u)
020:  #define PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_MSK (0x1u)
021:  #define PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_OFF (9u)
022:  #define PWM_GTM_ATOM_CH_CTRL_MODE_VAL (2u)
023:  #define PWM_GTM_ATOM_CH_CTRL_MODE_MSK (0x3u)
024:  #define PWM_GTM_ATOM_CH_CTRL_MODE_OFF (0u)
025:  #define PWM_GTM_ATOM_CH_CTRL_OSM_VAL (0u)
026:  #define PWM_GTM_ATOM_CH_CTRL_OSM_MSK (0x1u)
027:  #define PWM_GTM_ATOM_CH_CTRL_OSM_OFF (26u)
028:  #define PWM_GTM_ATOM_CH_CTRL_SLA_VAL (0u)
029:  #define PWM_GTM_ATOM_CH_CTRL_SLA_MSK (0x1u)
030:  #define PWM_GTM_ATOM_CH_CTRL_SLA_OFF (25u)
031:  #define PWM_GTM_ATOM_CH_CTRL_TB12_SEL_VAL (0u)
032:  #define PWM_GTM_ATOM_CH_CTRL_TB12_SEL_MSK (0x1u)
033:  #define PWM_GTM_ATOM_CH_CTRL_TB12_SEL_OFF (2u)
034:  #define PWM_GTM_ATOM_CH_CTRL_WR_REQ_LEN (0u)
035:  #define PWM_GTM_ATOM_CH_CTRL_WR_REQ_MSK (0x1u)
036:  #define PWM_GTM_ATOM_CH_CTRL_WR_REQ_OFF (16u)
037:  #define PWM_GTM_TOM_CH_CTRL_SPEM_VAL (0U)
038:  #define PWM_GTM_TOM_CH_CTRL_GCM_VAL (0u)
039:  #define PWM_GTM_TOM_CH_CTRL_OSM_VAL (0u)
040:  #define PWM_GTM_TOM_CH_CTRL_SPEM_LEN (1u)
041:  #define PWM_GTM_TOM_CH_CTRL_SPEM_MSK (0x1u)
042:  #define PWM_GTM_TOM_CH_CTRL_SPEM_OFF (28u)
043:  #define PWM_GTM_TOM_CH_CTRL_GCM_LEN (1u)
044:  #define PWM_GTM_TOM_CH_CTRL_GCM_MSK (0x1u)
045:  #define PWM_GTM_TOM_CH_CTRL_GCM_OFF (29u)
046:  #define PWM_GTM_TOM_CH_CTRL_OSM_LEN (1u)
047:  #define PWM_GTM_TOM_CH_CTRL_OSM_MSK (0x1u)
048:  #define PWM_GTM_TOM_CH_CTRL_OSM_OFF (26u)
049:  #define PWM_GTM_ATOM_CH_CTRL_SL_OFF (11u)
050:  #define PWM_GTM_ATOM_CH_CTRL_SL_MSK (0x1u)
051:  #define PWM_GTM_TOM_CH_CTRL_SL_OFF (11u)
052:  #define PWM_GTM_TOM_CH_CTRL_SL_MSK (0x1u)
053:  #if (PWM_DUTY_SHIFT_IN_TICKS == STD_OFF)
054:  #define PWM_SHIFT_BY15 ((uint32) (15))
055:  #endif
056:  #define PWM_CHANNEL_MODE_POS ((uint32) 0xB)
057:  #define PWM_CHANNEL_MODE_MAX ((uint32) 0x3)
058:  #define PWM_POLSTATE_POS (0x8U)

```

Code Listing 6 Macros used in code

```

059:  #define PWM_POLSTATE_MAX                ((uint32)0x1)
060:  #define PWM_SPLSHFT_POS                 (21U)
061:  #define PWM_SPLSHFTSTATE_MAX            ((uint32)0x1)
062:  #define PWM_RSTSHFT_POS                 (22U)
063:  #define PWM_RSTSHFTSTATE_MAX            ((uint32)0x1)
064:  #define PWM_COHERENCY_POS               (0xAU)
065:  #define PWM_COHERENCY_MAX               ((uint32)0x1)
066:  #if ( (PWM_FIXED_PERIOD_USED == STD_ON) || (PWM_SET_DUTY_CYCLE_API ==
STD_ON) )
067:  #define PWM_ISREFERENCE_POS             (0x14U)
068:  #define PWM_ISREFERENCE_MAX             ((uint32)0x1)
069:  #endif
070:  #define PWM_REFERENCE_CHAN_POS          (13U)
071:  #define PWM_7BIT_MASK                   (0x7FU)
072:  #define PWM_HWUNIT_POS                  ((uint8)(0xFF))
073:  #define PWM_MAX_TIMER                   ((uint32)0xFFFFFEU)
074:  #define PWM_MAX_TIMER_TOM               ((uint32)0xFFFEU)
075:  #define PWM_GTM_CTRL_TRIGOUT_MASK       (1U)
076:  #define PWM_GTM_CTRL_TRIGOUT            (24U)
077:  #define PWM_GTM_CTRL_RST_CCU0           (20U)
078:  #define PWM_GTM_CTRL_RST_CCU0_MASK      (1U)
079:  #define PWM_SHIFTED_RELATED_CHANNEL     (1U)
080:  #define PWM_SHIFTED_CHANNEL             (1U)
081:  #define PWM_NORMAL_CHANNEL              (0U)
082:  #define PWM_NON_COHERENT                (0U)
083:  #define PWM_COHERENT                    (1U)
084:  #define PWM_CONST_16                    (16U)
085:  #define PWM_CONST_2                     (2U)
086:  #define PWM_INVALID_NUMBER              (0xFFU)

```

Code Listing 7 Local function declarations

```

001:  /*****
002:  Function declarations
003:  *****/
004:  IFX_LOCAL_INLINE Pwm_17_Gtm_OutputStateType Pwm_lDemoCoherency (uint32 Value);
005:  IFX_LOCAL_INLINE uint8 Pwm_lDemoAssignedHW(uint32 Value);
006:  IFX_LOCAL_INLINE Pwm_ChannelIdentifierType Pwm_lDemoGtmChannelIdentifier(
007:      Pwm_GlobalChannelNumberType GlobalChannelNumber );
008:  IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelClassType Pwm_lDemoChanClass(uint32 Value);
009:  IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelType Pwm_lDemoFindChannelIndex (
010:      Pwm_17_Gtm_ChannelType ChannelNumber);
011:  IFX_LOCAL_INLINE uint8 Pwm_lDemoFindChannelSignal(
012:      Pwm_17_Gtm_ChannelType ChannelNumber);
013:
014:  IFX_LOCAL_INLINE uint32 Pwm_lDemoGtmCounterVal
015:  (
016:      uint32 Shift,
017:      uint32 Period,

```


Code Listing 7 Local function declarations

```

018:     uint32 DutyCycle
019: );
020:
021: IFX_LOCAL_INLINE uint8 Pwm_1DemoShiftSpl(uint32 Value);
022:
023: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetAbsvalShCen
024:     (Pwm_17_Gtm_PeriodType Period,Pwm_17_Gtm_PeriodType
Value);
025:
026: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetShiftLead(
027:     Pwm_17_Gtm_PeriodType Period,Pwm_17_Gtm_PeriodType
Shift);
028:
029: IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelType Pwm_1DemoReferChan(uint32 Value);
030: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetAbsvalEdge \
031:     (Pwm_17_Gtm_PeriodType Period, uint32
Value);
032:
033: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetCenterLead
034: (
035:     Pwm_17_Gtm_PeriodType Period,
036:     Pwm_17_Gtm_PeriodType ScaledDuty
037: );
038:
039: IFX_LOCAL_INLINE uint8 Pwm_1DemoIsReference(uint32 Value);
040: IFX_LOCAL_INLINE uint8 Pwm_1DemoResetShift(uint32 Value);
041: IFX_LOCAL_INLINE uint8 Pwm_1DemoPolarity(uint32 Value);
042: static Std_ReturnType Pwm_1DemoVerifyTomCtrlValues(uint32 RegValue);
043: static Std_ReturnType Pwm_1DemoVerifyAtomCtrlValues(uint32 RegValue);
044: IFX_LOCAL_INLINE uint8 Pwm_1DemoGetChannelType(const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
045:
046: /*****
047: Local functions to check SFR's
048: *****/
049: IFX_LOCAL_INLINE Std_ReturnType Pwm_1TomFupdCheck(uint8 ModuleNumber,uint8
GroupNumber,\
050:     uint8 ChannelNumber);
051: IFX_LOCAL_INLINE Std_ReturnType Pwm_1AtomFupdCheck(uint8 ModuleNumber,uint8
ChannelNumber);
052: IFX_LOCAL_INLINE Std_ReturnType Pwm_1TomGCtrlCheck(uint8 ModuleNumber,uint8
GroupNumber,\
053:     uint8 ChannelNumber,uint8 Coherency);
054: IFX_LOCAL_INLINE Std_ReturnType Pwm_1AtomGCtrlCheck(uint8 ModuleNumber,\
055:     uint8 ChannelNumber,uint8 Coherency);
056: IFX_LOCAL_INLINE Std_ReturnType Pwm_1AtomEndisOutenCheck(uint8
ModuleNumber,uint8 ChannelNumber);
057: IFX_LOCAL_INLINE Std_ReturnType Pwm_1TomEndisOutenCheck(uint8
ModuleNumber,uint8 GroupNumber,\
058:     uint8 ChannelNumber);

```

Code Listing 7 Local function declarations

```

059:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lGroupRegisterVerify(const
      Pwm_17_Gtm_ConfigType *ConfigPtr);
060:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomIrqEnCheck(uint8 ModuleNumber,\
061:               uint8 ChannelNumber);
062:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomIrqEnCheck(uint8 ModuleNumber,\
063:               uint8 ChannelNumber);
064:   IFX_LOCAL_INLINE uint32 Pwm_lInitVPSr0Extract(const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
065:   IFX_LOCAL_INLINE uint32 Pwm_lInitFPSr0Extract(const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
066:   IFX_LOCAL_INLINE uint32 Pwm_lInitFPS_SR0Extract( const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
067:   IFX_LOCAL_INLINE uint32 Pwm_lInitCASr0Extract( const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
068:   IFX_LOCAL_INLINE uint32 Pwm_lInitVPSr1Extract(const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
069:   IFX_LOCAL_INLINE uint32 Pwm_lInitFPSr1Extract(const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
070:   IFX_LOCAL_INLINE uint32 Pwm_lInitFPS_SR1Extract( const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
071:   IFX_LOCAL_INLINE uint32 Pwm_lInitCASr1Extract( const
      Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
072:   IFX_LOCAL_INLINE uint32 Pwm_lCalcSr0Val( const Pwm_17_Gtm_ChannelConfigType
      *LocalPwmChanPtr);
073:   IFX_LOCAL_INLINE uint32 Pwm_lCalcSr1Val( const Pwm_17_Gtm_ChannelConfigType
      *LocalPwmChanPtr);
074:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomSr0SrlCheck(uint8 ModuleNumber,\
075:               uint8 ChannelNumber,\
076:               const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
077:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomSr0SrlCheck(uint8 ModuleNumber,\
078:               uint8 ChannelNumber,\
079:               const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
080:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomChCtrlCheck( \
081:       const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,\
082:       uint8 ModuleNumber,uint8 ChannelNumber);
083:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomChCtrlCheck( \
084:       const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,\
085:       uint8 ModuleNumber,uint8 ChannelNumber);
086:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lChannelRegisterVerify(const
      Pwm_17_Gtm_ConfigType *ConfigPtr);
087:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lRegisterCheck(const Pwm_17_Gtm_ConfigType
      *ConfigPtr);
088:
089:   /*****
090:   Local functions to check global variables
091:   *****/
092:   /*****
093:   This function checks the flag set in Global variables "PtrGlobChn_Asil"
094:   and "PtrGlobChn_Qm" when the duty is 0% or 100% in the cases applicable.
095:   This function also verifies if the channelkind is stored correctly
096:   *****/

```

Code Listing 7 Local function declarations

```

097:   IFX_LOCAL_INLINE Std_ReturnType Pwm_lCheck_DutyFlag_ChannelKind
098:                                   (Pwm_17_Gtm_ChannelType Channel,uint8
   ChannelKind);
099:
0100:  /*****
0101:  This function checks the flag reset in Global variables "PtrGlobChn_Asil"
0102:  and "PtrGlobChn_Qm" in the cases applicable.
0103:  This function also verifies if the channelkind is stored correctly
0104:  *****/
0105:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lCheck_DutyReset_ChannelKind(\
0106:                                   Pwm_17_Gtm_ChannelType Channel,uint8
   ChannelKind);
0107:
0108:  /*****
0109:  This function extracts the period and duty values for variable period channels
0110:  and calls the corresponding local functions to verify global variable.
0111:  These local functions verify variables "PtrGlobChn_Asil" and "PtrGlobChn_Qm"
0112:  in the cases applicable.
0113:  *****/
0114:  static Std_ReturnType Pwm_lCheckInitVariablePeriod(\
0115:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32 ChannelNumber);
0116:
0117:  /*****
0118:  This function extracts the period and duty values for Fixed period channels
0119:  and calls the corresponding local functions to verify global variable.
0120:  These local functions verify variables "PtrGlobChn_Asil" and "PtrGlobChn_Qm"
0121:  in the cases applicable.
0122:  *****/
0123:  static Std_ReturnType Pwm_lCheckInitFixedPeriod(\
0124:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32 ChannelNumber);
0125:
0126:  /*****
0127:  This function extracts the period and duty values for Fixed period shifted
0128:  channels and calls the corresponding local functions to verify global
   variable.
0129:  These local functions verify variables "PtrGlobChn_Asil" and "PtrGlobChn_Qm"
0130:  in the cases applicable.
0131:  *****/
0132:  static Std_ReturnType Pwm_lCheckInitFixedPeriodShifted(\
0133:      const Pwm_17_Gtm_ChannelConfigType * LocalPwmChanPtr,uint32
   ChannelNumber);
0134:
0135:  /*****
0136:  This function extracts the period and duty values for center aligned
0137:  channels and calls the corresponding local functions to verify global
   variable.
0138:  These local functions verify variables "PtrGlobChn_Asil" and "PtrGlobChn_Qm"
0139:  in the cases applicable.
0140:  *****/
0141:  static Std_ReturnType Pwm_lCheckInitCentreAligned(\

```

Code Listing 7 Local function declarations

```

0142:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32 ChannelNumber);
0143:
0144:  /*****
0145:  This function extracts the type of channel and call local functions to
0146:  verify variables "PtrGlobChn_Asil" and "PtrGlobChn_Qm" in the cases
0147:  applicable.
0148:  *****/
0149:  static Std_ReturnType Pwm_lGlobChnVerify(\
0150:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32
        PwmChannelNumber);
0151:
0152:  /*****
0153:  This function extracts the period value and verify if the period value
0154:  is stored in global variable for variable period channels.
0155:  *****/
0156:  static Std_ReturnType Pwm_lVPCurrentPeriodValVerify(\
0157:      uint32 ChannelNumber,const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr);
0158:
0159:  /*****
0160:  This function verifies if value zero is stored in global variable
0161:  for channels other than variable period type.
0162:  *****/
0163:  static Std_ReturnType Pwm_lOtherCurrentPeriodValVerify(\
0164:      uint32 ChannelNumber);
0165:
0166:  /*****
0167:  Local function to check "CurrentPeriodVal" global variable
0168:  *****/
0169:  Std_ReturnType Pwm_lCurrentPeriodValVerify(\
0170:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr ,\
0171:      Pwm_17_Gtm_ChannelType PwmChannelNumber);
0172:
0173:  /*****
0174:  Local function to global variables initialized in PWM Init
0175:  *****/
0176:  Std_ReturnType Pwm_lGlobalVariableCheck(const Pwm_17_Gtm_ConfigType
        *ConfigPtr);

```

Code Listing 8 Functions reused from code

```

001:  /*****
002:  Reused code
003:  *****/
004:  IFX_LOCAL_INLINE Pwm_17_Gtm_OutputStateType Pwm_lDemoCoherency (uint32
        Value)
005:  {
006:      Pwm_17_Gtm_OutputStateType RetVal;
007:      RetVal = (Pwm_17_Gtm_OutputStateType) \
008:          ( ((uint32)(Value) >> PWM_COHERENCY_POS) &

```

Code Listing 8 Functions reused from code

```

(PWM_COHERENCY_MAX) );
009:     return(RetVal);
010: }
011:
012: IFX_LOCAL_INLINE uint8 Pwm_lDemoAssignedHW(uint32 Value)
013: {
014:     uint8 RetVal;
015:     RetVal = ((uint8)((Value) & (uint32)PWM_HWUNIT_POS));
016:     return(RetVal);
017: }
018:
019: IFX_LOCAL_INLINE Pwm_ChannelIdentifierType
Pwm_lDemoGtmChannelIdentifier(
020:                                     Pwm_GlobalChannelNumberType
GlobalChannelNumber )
021: {
022:     Pwm_ChannelIdentifierType ChannelIdentifier;
023:     Pwm_GlobalChannelNumberType TempChannelNumber;
024:     ChannelIdentifier.ChannelNo = PWM_INVALID_NUMBER;
025:     ChannelIdentifier.ModuleId = PWM_INVALID_NUMBER;
026:     ChannelIdentifier.ModuleNo = PWM_INVALID_NUMBER;
027:     ChannelIdentifier.GroupNo = PWM_INVALID_NUMBER;
028:
029:     /* Check if it's a TOM channel */
030:     if(GlobalChannelNumber < (GTM_MAX_TOM_CHANNELS +
GTM_MAX_TIM_CHANNELS))
031:     {
032:         TempChannelNumber = GlobalChannelNumber - GTM_MAX_TIM_CHANNELS;
033:         /* Calculate Module Id of the TOM channel */
034:         ChannelIdentifier.ModuleId = GTM_TOM_MODULE;
035:         ChannelIdentifier.ModuleNo = \
036:             (uint8)(TempChannelNumber /
GTM_CHANNELS_PER_TOM_MODULE);
037:
038:         /* Calculate Channel number of the TOM channel */
039:         ChannelIdentifier.ChannelNo= TempChannelNumber
%GTM_CHANNELS_PER_TOM_MODULE;
040:         ChannelIdentifier.GroupNo =
041:
(ChannelIdentifier.ChannelNo)/GTM_TOM_CHANNELS_PER_TGC;
042:     }
043:
044:     /* Check if it's an ATOM channel */
045:     if(GlobalChannelNumber >= (GTM_MAX_TOM_CHANNELS +
GTM_MAX_TIM_CHANNELS))
046:     {
047:         TempChannelNumber =
048:             GlobalChannelNumber - (GTM_MAX_TIM_CHANNELS +
GTM_MAX_TOM_CHANNELS);
049:         /* Calculate Module Id of the ATOM channel */

```

Code Listing 8 Functions reused from code

```

050:     ChannelIdentifier.ModuleId = GTM_ATOM_MODULE;
051:     ChannelIdentifier.ModuleNo  = \
052:         (uint8) (TempChannelNumber /
    GTM_CHANNELS_PER_ATOM_MODULE);
053:
054:     /* Calculate Channel number of the ATOM channel */
055:     ChannelIdentifier.ChannelNo =
056:         TempChannelNumber%GTM_CHANNELS_PER_ATOM_MODULE;
057:     }
058:
059:     return (ChannelIdentifier);
060: }
061:
062: IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelClassType Pwm_1DemoChanClass (uint32
Value)
063: {
064:     Pwm_17_Gtm_ChannelClassType RetVal;
065:
066:     RetVal = (Pwm_17_Gtm_ChannelClassType) \
067:         (((uint32) (Value) >> PWM_CHANNEL_MODE_POS) &
    (PWM_CHANNEL_MODE_MAX));
068:     return (RetVal);
069: }
070:
071: IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelType Pwm_1DemoFindChannelIndex (
072:     Pwm_17_Gtm_ChannelType
    ChannelNumber)
073: {
074:     Pwm_17_Gtm_ChannelType ChannelIndex;
075:     #if (PWM_SAFETY_ENABLE == STD_ON)
076:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr;
077:     LocalPwmChanPtr = (DemoGlobalConfigPtr->ChannelConfigPtr) +
    ChannelNumber;
078:     ChannelIndex = LocalPwmChanPtr->SafetyChannelIndex;
079:     #else
080:     ChannelIndex = ChannelNumber;
081:     #endif
082:     return (ChannelIndex);
083: }
084:
085: IFX_LOCAL_INLINE uint8 Pwm_1DemoFindChannelSignal (
086:     Pwm_17_Gtm_ChannelType ChannelNumber)
087: {
088:     uint8 ChannelSignal;
089:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr;
090:     LocalPwmChanPtr = (DemoGlobalConfigPtr->ChannelConfigPtr) +
    ChannelNumber;
091:     ChannelSignal = LocalPwmChanPtr->SafetyChannelSignal;
092:     return (ChannelSignal);

```

Code Listing 8 Functions reused from code

```

093:     }
094:
095:     IFX_LOCAL_INLINE uint32 Pwm_1DemoGtmCounterVal
096:     (
097:         uint32 Shift,
098:         uint32 Period,
099:         uint32 DutyCycle
0100:    )
0101:    {
0102:        uint32 RetVal;
0103:        if((Shift + DutyCycle) == Period)
0104:        {
0105:            RetVal = Period;
0106:        }
0107:        else
0108:        {
0109:            RetVal = (uint32)((Shift + DutyCycle) % Period);
0110:        }
0111:        return(RetVal);
0112:    }
0113:
0114:    IFX_LOCAL_INLINE uint8 Pwm_1DemoShiftSpl(uint32 Value)
0115:    {
0116:        uint8 RetVal;
0117:        RetVal = (uint8)(((uint32)(Value) >>
0118:        PWM_SPLSHFT_POS)&(PWM_SPLSHFTSTATE_MAX));
0119:        return(RetVal);
0120:    }
0121:    IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetAbsvalShCen
0122:        (Pwm_17_Gtm_PeriodType Period,Pwm_17_Gtm_PeriodType
0123:        Value)
0124:    {
0125:        Pwm_17_Gtm_PeriodType ScaledValue;
0126:        #if (PWM_DUTY_SHIFT_IN_TICKS == STD_ON)
0127:            ScaledValue = Value;
0128:        #else
0129:            ScaledValue = (Pwm_17_Gtm_PeriodType)
0130:                (((Value * (uint32)Period)) >> PWM_SHIFT_BY15);
0131:        #endif
0132:        if (Period <= ScaledValue)
0133:        {
0134:
0135:            ScaledValue = PWM_MAX_TIMER;
0136:        }
0137:        if (Period == 0U)
0138:        {
0139:            ScaledValue = 0U;

```

Code Listing 8 Functions reused from code

```

0140:     }
0141:     return (ScaledValue);
0142: }
0143:
0144: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetShiftLead(
0145:     Pwm_17_Gtm_PeriodType Period, Pwm_17_Gtm_PeriodType
    Shift)
0146: {
0147:     Pwm_17_Gtm_PeriodType ScaledShiftLead;
0148:
0149:     ScaledShiftLead = Pwm_1DemoGetAbsvalShCen(Period, Shift);
0150:
0151:     if (ScaledShiftLead >= Period)
0152:     {
0153:         ScaledShiftLead = (Pwm_17_Gtm_PeriodType)0;
0154:     }
0155:
0156:     return ScaledShiftLead;
0157: }
0158:
0159: IFX_LOCAL_INLINE Pwm_17_Gtm_ChannelType Pwm_1DemoReferChan(uint32 Value)
0160: {
0161:     Pwm_17_Gtm_ChannelType RetVal;
0162:
0163:     RetVal = (((uint32)(Value) >> PWM_REFERENCE_CHAN_POS) &
    (PWM_7BIT_MASK));
0164:     return(RetVal);
0165: }
0166:
0167: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetAbsvalEdge \
0168:     (Pwm_17_Gtm_PeriodType Period, uint32
    Value)
0169: {
0170:     Pwm_17_Gtm_PeriodType ScaledValue;
0171:     #if (PWM_DUTY_SHIFT_IN_TICKS == STD_ON)
0172:     ScaledValue = Value;
0173:     #else
0174:     ScaledValue = (Pwm_17_Gtm_PeriodType)((Value * Period) >>
    PWM_SHIFT_BY15);
0175:     #endif
0176:     if (Period <= ScaledValue)
0177:     {
0178:         ScaledValue = Period;
0179:     }
0180:     if (Period == 0U)
0181:     {
0182:         ScaledValue = 0U;
0183:     }
0184:

```


Code Listing 8 Functions reused from code

```

0185:     return (ScaledValue);
0186: }
0187:
0188: IFX_LOCAL_INLINE Pwm_17_Gtm_PeriodType Pwm_1DemoGetCenterLead
0189: (
0190:     Pwm_17_Gtm_PeriodType ScaledDuty
0191: )
0192: {
0193:     Pwm_17_Gtm_PeriodType ScaledCenterLead;
0194:     ScaledCenterLead = PWM_MAX_TIMER;
0195:
0196:     if ( ScaledDuty < (Period) )
0197:     {
0198:         ScaledCenterLead =
0199:             (Pwm_17_Gtm_PeriodType)(( ( (Period) - (ScaledDuty))
0200:                                     + ((Pwm_17_Gtm_PeriodType)1)
0201:                                   )
0202:             / ((Pwm_17_Gtm_PeriodType)PWM_CONST_2)
0203:             )
0204:             - ((Pwm_17_Gtm_PeriodType)1)
0205:         );
0206:     }
0207:     return ScaledCenterLead;
0208: }
0209:
0210: IFX_LOCAL_INLINE uint8 Pwm_1DemoIsReference(uint32 Value)
0211: {
0212:     uint8 RetVal;
0213:
0214:     RetVal = (uint8) \
0215:         (((uint32)(Value) >> PWM_ISREFERENCE_POS) &
0216:         (PWM_ISREFERENCE_MAX));
0217:     return(RetVal);
0218: }
0219:
0220: IFX_LOCAL_INLINE uint8 Pwm_1DemoResetShift(uint32 Value)
0221: {
0222:     uint8 RetVal;
0223:     RetVal = (uint8)((uint32)(Value) >>
0224:         PWM_RSTSHFT_POS)&(PWM_RSTSHFTSTATE_MAX));
0225:     return(RetVal);
0226: }
0227:
0228: IFX_LOCAL_INLINE uint8 Pwm_1DemoPolarity(uint32 Value)
0229: {
0230:     uint8 RetVal;
0231:     RetVal = (uint8)((uint32)(Value) >> PWM_POLSTATE_POS) &
0232:         (PWM_POLSTATE_MAX)
0233:     );

```

Code Listing 8 Functions reused from code

```

0232:   return(RetVal);
0233: }
0234:
0235: static Std_ReturnType Pwm_lDemoVerifyTomCtrlValues(uint32 RegValue)
0236: {
0237:     Std_ReturnType ErrorFlag = E_NOT_OK;
0238:     uint32 ExpectedSpemValue;
0239:     uint32 ExpectedGcmValue;
0240:     uint32 ExpectedOsmValue;
0241:
0242:     ExpectedSpemValue = (uint32)((PWM_GTM_TOM_CH_CTRL_SPEM_VAL & \
0243: PWM_GTM_TOM_CH_CTRL_SPEM_MSK) <<
PWM_GTM_TOM_CH_CTRL_SPEM_OFF);
0244:     ExpectedGcmValue = (uint32)((PWM_GTM_TOM_CH_CTRL_GCM_VAL & \
0245: PWM_GTM_TOM_CH_CTRL_GCM_MSK) <<
PWM_GTM_TOM_CH_CTRL_GCM_OFF);
0246:     ExpectedOsmValue = (uint32)((PWM_GTM_TOM_CH_CTRL_OSM_VAL & \
0247: PWM_GTM_TOM_CH_CTRL_OSM_MSK) <<
PWM_GTM_TOM_CH_CTRL_OSM_OFF);
0248:     if(((RegValue & (PWM_GTM_TOM_CH_CTRL_SPEM_MSK <<
PWM_GTM_TOM_CH_CTRL_SPEM_OFF)) \
0249: == ExpectedSpemValue) &&
0250: ((RegValue & (PWM_GTM_TOM_CH_CTRL_GCM_MSK <<
PWM_GTM_TOM_CH_CTRL_GCM_OFF)) \
0251: == ExpectedGcmValue) && \
0252: ((RegValue & (PWM_GTM_TOM_CH_CTRL_OSM_MSK <<
PWM_GTM_TOM_CH_CTRL_OSM_OFF)) \
0253: == ExpectedOsmValue))
0254: {
0255:     ErrorFlag = E_OK;
0256: }
0257:     return(ErrorFlag);
0258: }
0259:
0260: static Std_ReturnType Pwm_lDemoVerifyAtomCtrlValues(uint32 RegValue)
0261: {
0262:     Std_ReturnType ErrorFlag = E_NOT_OK;
0263:
0264:     uint32 ExpectedAbmValue;
0265:     uint32 ExpectedAcbValue ;
0266:     uint32 ExpectedAruEnValue ;
0267:     uint32 ExpectedCmpCtrlValue;
0268:     uint32 ExpectedModeValue;
0269:     uint32 ExpectedOsmValue ;
0270:     uint32 ExpectedSlaValue ;
0271:     uint32 ExpectedTb12Value ;
0272:     uint32 ExpectedWrReqValue ;
0273:
0274:     ExpectedAbmValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_ABM_VAL & \
0275: PWM_GTM_ATOM_CH_CTRL_ABM_MSK) <<

```

Code Listing 8 Functions reused from code

```

        PWM_GTM_ATOM_CH_CTRL_ABM_OFF);
0276:
0277:     ExpectedAcbValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_ACB_VAL & \
0278:         PWM_GTM_ATOM_CH_CTRL_ACB_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_ACB_OFF);
0279:
0280:     ExpectedAruEnValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_ARU_EN_VAL & \
0281:         PWM_GTM_ATOM_CH_CTRL_ARU_EN_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_ARU_EN_OFF);
0282:
0283:     ExpectedCmpCtrlValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_VAL & \
0284:         PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_OFF);
0285:
0286:     ExpectedModeValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_MODE_VAL & \
0287:         PWM_GTM_ATOM_CH_CTRL_MODE_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_MODE_OFF);
0288:
0289:     ExpectedOsmValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_OSM_VAL & \
0290:         PWM_GTM_ATOM_CH_CTRL_OSM_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_OSM_OFF);
0291:
0292:     ExpectedSlaValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_SLA_VAL & \
0293:         PWM_GTM_ATOM_CH_CTRL_SLA_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_SLA_OFF);
0294:
0295:     ExpectedTb12Value = (uint32)((PWM_GTM_ATOM_CH_CTRL_TB12_SEL_VAL & \
0296:         PWM_GTM_ATOM_CH_CTRL_TB12_SEL_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_TB12_SEL_OFF);
0297:
0298:     ExpectedWrReqValue = (uint32)((PWM_GTM_ATOM_CH_CTRL_WR_REQ_LEN & \
0299:         PWM_GTM_ATOM_CH_CTRL_WR_REQ_MSK) <<
        PWM_GTM_ATOM_CH_CTRL_WR_REQ_OFF);
0300:
0301:     if((RegValue & (PWM_GTM_ATOM_CH_CTRL_ABM_MSK << \
0302:         PWM_GTM_ATOM_CH_CTRL_ABM_OFF)) == ExpectedAbmValue)
0303:     {
0304:         ErrorFlag = E_OK;
0305:     }
0306:     if (ErrorFlag == E_OK)
0307:     {
0308:         if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_ACB_MSK << \
0309:             PWM_GTM_ATOM_CH_CTRL_ACB_OFF)) == ExpectedAcbValue)
0310:         {
0311:             ErrorFlag = E_OK;
0312:         }
0313:     }
0314:     if (ErrorFlag == E_OK)
0315:     {
0316:         if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_ARU_EN_MSK << \

```

Code Listing 8 Functions reused from code

```

0317:          PWM_GTM_ATOM_CH_CTRL_ARU_EN_OFF)) ==
          ExpectedAruEnValue)
0318:      {
0319:          ErrorFlag = E_OK;
0320:      }
0321:  }
0322:  if (ErrorFlag == E_OK)
0323:  {
0324:      if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_MSK << \
0325:          PWM_GTM_ATOM_CH_CTRL_CMP_CTRL_OFF)) ==
          ExpectedCmpCtrlValue)
0326:      {
0327:          ErrorFlag = E_OK;
0328:      }
0329:  }
0330:  if (ErrorFlag == E_OK)
0331:  {
0332:      if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_MODE_MSK << \
0333:          PWM_GTM_ATOM_CH_CTRL_MODE_OFF)) == ExpectedModeValue)
0334:      {
0335:          ErrorFlag = E_OK;
0336:      }
0337:  }
0338:  if (ErrorFlag == E_OK)
0339:  {
0340:      if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_OSM_MSK << \
0341:          PWM_GTM_ATOM_CH_CTRL_OSM_OFF)) == ExpectedOsmValue)
0342:      {
0343:          ErrorFlag = E_OK;
0344:      }
0345:  }
0346:  if (ErrorFlag == E_OK)
0347:  {
0348:      if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_SLA_MSK << \
0349:          PWM_GTM_ATOM_CH_CTRL_SLA_OFF)) == ExpectedSlaValue)
0350:      {
0351:          ErrorFlag = E_OK;
0352:      }
0353:  }
0354:  if (ErrorFlag == E_OK)
0355:  {
0356:      if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_TB12_SEL_MSK << \
0357:          PWM_GTM_ATOM_CH_CTRL_TB12_SEL_OFF)) ==
          ExpectedTb12Value)
0358:      {
0359:          ErrorFlag = E_OK;
0360:      }
0361:  }
0362:  if (ErrorFlag == E_OK)

```

Code Listing 8 Functions reused from code

```

0363:     {
0364:         if ((RegValue & (PWM_GTM_ATOM_CH_CTRL_WR_REQ_MSK << \
0365:             PWM_GTM_ATOM_CH_CTRL_WR_REQ_OFF)) ==
            ExpectedWrReqValue)
0366:         {
0367:             ErrorFlag = E_OK;
0368:         }
0369:     }
0370:     return (ErrorFlag);
0371: }
0372:
0373: IFX_LOCAL_INLINE uint8 Pwm_lDemoGetChannelType(const
    Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0374: {
0375:     uint8 ChannelKind;
0376:     uint8 ChannelType;
0377:     ChannelKind = Pwm_lDemoChanClass(LocalPwmChanPtr->ChannelInfo);
0378:     switch (ChannelKind)
0379:     {
0380:         #if (PWM_VARIABLE_PERIOD_USED == STD_ON)
0381:         case PWM_VARIABLE_PERIOD:
0382:             ChannelType = PWM_NORMAL_CHANNEL;
0383:
0384:             break ;
0385:         #endif
0386:         #if (PWM_FIXED_PERIOD_USED == STD_ON)
0387:         case PWM_FIXED_PERIOD:
0388:             ChannelType = PWM_NORMAL_CHANNEL;
0389:
0390:             break;
0391:         #endif
0392:         #if (PWM_FIXED_PERIOD_SHIFTED_USED == STD_ON)
0393:         case PWM_FIXED_PERIOD_SHIFTED:
0394:             if(Pwm_lDemoShiftSpl(LocalPwmChanPtr->ChannelInfo) == \
0395:                 PWM_SHIFTED_RELATED_CHANNEL)
0396:             {
0397:                 ChannelType = PWM_NORMAL_CHANNEL;
0398:
0399:             }
0400:             else
0401:             {
0402:                 ChannelType = PWM_SHIFTED_CHANNEL;
0403:
0404:             }
0405:             break;
0406:         #endif
0407:         default:
0408:             #if (PWM_USED_FIXED_PERIOD_CENTER_ALIGNED == STD_ON)

```

Code Listing 8 Functions reused from code

```

0409:         ChannelType = PWM_SHIFTED_CHANNEL;
0410:         #endif
0411:         break;
0412:     }
0413:     return (ChannelType);
0414: }

```

Code Listing 9 Local functions to check SFR values

```

001:
002:  /*****
003:  Local functions to check SFR values
004:  *****/
005:
006:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomFupdCheck(uint8 ModuleNumber, uint8
  GroupNumber, \
007:              uint8 ChannelNumber)
008:  {
009:      Ifx_GTM_TOM_TGC_TYPE* TomTgcRegPtr;          /* Ptr to TOM TGC Reg
  */
010:      uint32 RegVal;
011:      uint32 TempRegValue=0U;
012:      Std_ReturnType ErrorFlag = E_NOT_OK;
013:      TomTgcRegPtr = &((Ifx_GTM_TOMx*)(volatile void*)\
014:                      (MODULE_GTM.TOM)).TOM_TGC[ModuleNumber].TGC[GroupNumber];
015:      /* The corresponding channels FUPD should be disabled*/
016:      RegVal = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->FUPD_CTRL.U));
017:      TempRegValue = ((uint32)3U << (uint16)(PWM_CONST_2 * \
018:                      ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
019:      TempRegValue = RegVal & TempRegValue;
020:      /*FUPD for the channel should be dsabled. The value in the register
021:      should be zero*/
022:      if(TempRegValue == 0U)
023:      {
024:          ErrorFlag = E_OK;
025:      }
026:      return(ErrorFlag);
027:  }
028:
029:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomFupdCheck(uint8 ModuleNumber, uint8
  ChannelNumber)
030:  {
031:      Ifx_GTM_ATOM* AtomRegPtr;
032:      uint32 RegVal;
033:      uint32 TempRegValue=0U;
034:      Std_ReturnType ErrorFlag = E_NOT_OK;
035:
036:      AtomRegPtr = &(MODULE_GTM.ATOM[ModuleNumber]);
037:      RegVal = PWM_SFR_INIT_USER_MODE_READ32((AtomRegPtr->AGC.FUPD_CTRL.U));

```

Code Listing 9 Local functions to check SFR values

```

038:     TempRegValue = ((uint32)3U << (uint16)(PWM_CONST_2 * \
039:                     ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
040:     TempRegValue = RegVal & TempRegValue;
041:     if(TempRegValue == 0U)
042:     {
043:         ErrorFlag = E_OK;
044:     }
045:
046:     return (ErrorFlag);
047: }
048:
049: IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomGCtrlCheck(uint8 ModuleNumber,uint8
GroupNumber,\
050:             uint8 ChannelNumber,uint8 Coherency)
051: {
052:     Std_ReturnType ErrorFlag = E_NOT_OK;
053:     Ifx_GTM_TOM_TGC_TYPE* TomTgcRegPtr;
054:     uint32 RegVal;
055:     uint32 TempRegValue=0U;
056:     uint32 MaskValue;
057:
058:     TomTgcRegPtr = &((*Ifx_GTM_TOMx*)(volatile void*)\
059:                     (MODULE_GTM.TOM)).TOM_TGC[ModuleNumber].TGC[GroupNumber]);
060:     RegVal = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->GLB_CTRL.U));
061:     MaskValue = (uint32)((uint32)3U << (PWM_CONST_16 + (\
062:         PWM_CONST_2 * ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC))));
063:     if(Coherency == 1U)
064:     {
065:         TempRegValue = MaskValue;
066:     }
067:     RegVal = RegVal & MaskValue;
068:     if(RegVal == TempRegValue)
069:     {
070:         ErrorFlag = E_OK;
071:     }
072:     return(ErrorFlag);
073:
074: }
075:
076: IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomGCtrlCheck(uint8 ModuleNumber,\
077:             uint8 ChannelNumber,uint8 Coherency)
078: {
079:     Std_ReturnType ErrorFlag = E_NOT_OK;
080:     Ifx_GTM_ATOM *AtomAgcRegPtr;
081:     uint32 RegVal;
082:     uint32 TempRegValue=0U;
083:     uint32 MaskValue;
084:
085:     AtomAgcRegPtr = (&(MODULE_GTM.ATOM[ModuleNumber]));

```

Code Listing 9 Local functions to check SFR values

```

086:     RegVal = PWM_SFR_INIT_USER_MODE_READ32((AtomAgcRegPtr->AGC.GLB_CTRL.U));
087:     MaskValue = (uint32)((uint32)3U << (PWM_CONST_16 + (\
088:         PWM_CONST_2 * ((uint16)ChannelNumber % GTM_ATOM_CHANNELS_PER_AGC)))));
089:     if(Coherency == 1U)
090:     {
091:         TempRegValue = MaskValue;
092:     }
093:     RegVal = RegVal & MaskValue;
094:     if(RegVal == TempRegValue)
095:     {
096:         ErrorFlag = E_OK;
097:     }
098:     return(ErrorFlag);
099: }
0100:
0101: IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomEndisOutenCheck(uint8
ModuleNumber,uint8 ChannelNumber)
0102: {
0103:     Std_ReturnType ErrorFlag = E_NOT_OK;
0104:     Ifx_GTM_ATOM *AtomAgcRegPtr;
0105:     uint32 RegEndisStat;
0106:     uint32 RegOutenStat;
0107:     uint32 RegEndisCtrl;
0108:     uint32 RegOutenCtrl;
0109:     uint32 TempRegEndisStat;
0110:     uint32 TempRegOutenStat;
0111:     uint32 TempRegEndisCtrl;
0112:     uint32 TempRegOutenCtrl;
0113:
0114:     AtomAgcRegPtr = (&(MODULE_GTM.ATOM[ModuleNumber]));
0115:     RegEndisStat = PWM_SFR_INIT_USER_MODE_READ32((AtomAgcRegPtr-
>AGC.ENDIS_STAT.U));
0116:     RegOutenStat = PWM_SFR_INIT_USER_MODE_READ32((AtomAgcRegPtr-
>AGC.OUTEN_STAT.U));
0117:     RegEndisCtrl = PWM_SFR_INIT_USER_MODE_READ32((AtomAgcRegPtr-
>AGC.ENDIS_CTRL.U));
0118:     RegOutenCtrl = PWM_SFR_INIT_USER_MODE_READ32((AtomAgcRegPtr-
>AGC.OUTEN_CTRL.U));
0119:     TempRegEndisStat = ((uint32)3U << (uint16)(PWM_CONST_2 * \
0120:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0121:     TempRegOutenStat = ((uint32)3U << (uint16)(PWM_CONST_2 * \
0122:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0123:     TempRegEndisCtrl = ((uint32)3U << (uint16)(PWM_CONST_2 * \
0124:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0125:     TempRegOutenCtrl = ((uint32)3U << (uint16)(PWM_CONST_2 * \
0126:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0127:     TempRegEndisStat = RegEndisStat & TempRegEndisStat;
0128:     TempRegOutenStat = RegOutenStat & TempRegOutenStat;
0129:     TempRegEndisCtrl = RegEndisCtrl & TempRegEndisCtrl;
0130:     TempRegOutenCtrl = RegOutenCtrl & TempRegOutenCtrl;

```


Code Listing 9 Local functions to check SFR values

```

0131:     if( (TempRegEndisStat!= 0U) && (TempRegOutenStat != 0U)&&\
0132:         (TempRegEndisCtrl!= 0U) && (TempRegOutenCtrl != 0U))
0133:     {
0134:         ErrorFlag = E_OK;
0135:     }
0136:     return(ErrorFlag);
0137: }
0138:
0139: IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomEndisOutenCheck(uint8
    ModuleNumber,uint8 GroupNumber,\
0140:                 uint8 ChannelNumber)
0141: {
0142:     Std_ReturnType ErrorFlag = E_NOT_OK;
0143:     Ifx_GTM_TOM_TGC_TYPE* TomTgcRegPtr;
0144:     uint32 RegEndisStat;
0145:     uint32 RegOutenStat;
0146:     uint32 RegEndisCtrl;
0147:     uint32 RegOutenCtrl;
0148:     uint32 TempRegEndisStat;
0149:     uint32 TempRegOutenStat;
0150:     uint32 TempRegEndisCtrl;
0151:     uint32 TempRegOutenCtrl;
0152:     TomTgcRegPtr = &((*(Ifx_GTM_TOMx*)(volatile void*))\
0153:         (MODULE_GTM.TOM)).TOM_TGC[ModuleNumber].TGC[GroupNumber]);
0154:     RegEndisStat = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->ENDIS_STAT.U));
0155:     RegOutenStat = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->OUTEN_STAT.U));
0156:     RegEndisCtrl = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->ENDIS_CTRL.U));
0157:     RegOutenCtrl = PWM_SFR_INIT_USER_MODE_READ32((TomTgcRegPtr->OUTEN_CTRL.U));
0158:     TempRegEndisStat = ((uint32)3U << (uint16) (PWM_CONST_2 * \
0159:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0160:     TempRegOutenStat = ((uint32)3U << (uint16) (PWM_CONST_2 * \
0161:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0162:     TempRegEndisCtrl = ((uint32)3U << (uint16) (PWM_CONST_2 * \
0163:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0164:     TempRegOutenCtrl = ((uint32)3U << (uint16) (PWM_CONST_2 * \
0165:         ((uint16)ChannelNumber % GTM_TOM_CHANNELS_PER_TGC)));
0166:     TempRegEndisStat = RegEndisStat & TempRegEndisStat;
0167:     TempRegOutenStat = RegOutenStat & TempRegOutenStat;
0168:     TempRegEndisCtrl = RegEndisCtrl & TempRegEndisCtrl;
0169:     TempRegOutenCtrl = RegOutenCtrl & TempRegOutenCtrl;
0170:
0171:     if( (TempRegEndisStat!= 0U) && (TempRegOutenStat != 0U)&&\
0172:         (TempRegEndisCtrl!= 0U) && (TempRegOutenCtrl != 0U))
0173:     {
0174:         ErrorFlag = E_OK;
0175:     }
0176:
0177:     return(ErrorFlag);
0178:

```

Code Listing 9 Local functions to check SFR values

```

0179:  }
0180:
0181:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lGroupRegisterVerify(const
    Pwm_17_Gtm_ConfigType *ConfigPtr)
0182:  {
0183:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr;
0184:      Pwm_ChannelIdentifierType GlobalChannelId;
0185:      Std_ReturnType ErrorFlag = E_OK;
0186:      uint32 PwmChannelNumber;
0187:      uint32 ModuleNumber;
0188:      uint32 GroupNumber;
0189:      uint32 TomChannelNumber;
0190:      uint8  CellUsed;
0191:      uint8  Coherency;
0192:      LocalPwmChanPtr = ConfigPtr->ChannelConfigPtr;
0193:      for ( PwmChannelNumber = (Pwm_17_Gtm_ChannelType)0;
0194:           PwmChannelNumber < ConfigPtr->MaxChannels;
0195:           PwmChannelNumber++)
0196:      {
0197:          {
0198:              CellUsed = Pwm_lDemoAssignedHW(LocalPwmChanPtr->ChannelInfo);
0199:              #if(PWM_DUTYCYCLE_UPDATED_ENDPERIOD == STD_OFF)
0200:              Coherency = Pwm_lDemoCoherency(LocalPwmChanPtr->ChannelInfo);
0201:              #else
0202:              Coherency = PWM_COHERENT;
0203:              #endif
0204:              GlobalChannelId = Pwm_lDemoGtmChannelIdentifier(CellUsed);
0205:              ModuleNumber     = GlobalChannelId.ModuleNo;
0206:              GroupNumber      = GlobalChannelId.GroupNo;
0207:              TomChannelNumber = GlobalChannelId.ChannelNo;
0208:              if((GlobalChannelId.ModuleId) == GTM_TOM_MODULE)
0209:              {
0210:                  ErrorFlag =
    Pwm_lTomFupdCheck(ModuleNumber, GroupNumber, TomChannelNumber);
0211:                  if(ErrorFlag != E_NOT_OK)
0212:                  {
0213:                      ErrorFlag =
    Pwm_lTomGCtrlCheck(ModuleNumber, GroupNumber, TomChannelNumber, Coherency);
0214:                  }
0215:                  else
0216:                  {
0217:                      break ;
0218:                  }
0219:                  if(ErrorFlag != E_NOT_OK)
0220:                  {
0221:                      ErrorFlag = Pwm_lTomEndisOutenCheck(ModuleNumber, GroupNumber, \
0222:                                                           TomChannelNumber);
0223:                  }
0224:                  else

```

Code Listing 9 Local functions to check SFR values

```

0225:      {
0226:          break ;
0227:      }
0228:  }
0229:  if((GlobalChannelId.ModuleId) == GTM_ATOM_MODULE)
0230:  {
0231:      ErrorFlag = Pwm_lAtomFupdCheck(ModuleNumber,TomChannelNumber);
0232:      if(ErrorFlag != E_NOT_OK)
0233:      {
0234:          ErrorFlag =
Pwm_lAtomGCtrlCheck(ModuleNumber,TomChannelNumber,Coherency);
0235:      }
0236:      else
0237:      {
0238:          break ;
0239:      }
0240:      if(ErrorFlag != E_NOT_OK)
0241:      {
0242:          ErrorFlag = Pwm_lAtomEndisOutenCheck(ModuleNumber,TomChannelNumber);
0243:      }
0244:      else
0245:      {
0246:          break ;
0247:      }
0248:  }
0249:  LocalPwmChanPtr++;
0250:  }
0251:  return(ErrorFlag);
0252:  }
0253:
0254:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomIrqEnCheck(uint8 ModuleNumber,\
0255:      uint8 ChannelNumber)
0256:  {
0257:      Ifx_GTM_TOM_CH_TYPE* TomChannelRegPtr;
0258:      uint32 RegVal;
0259:      Std_ReturnType ErrorFlag = E_NOT_OK;
0260:      /* For Temp storage of Reg Value */
0261:      TomChannelRegPtr = &(((Ifx_GTM_TOMx*)(volatile void*) \
0262:          (MODULE_GTM.TOM)).TOM_CH[ModuleNumber].CH[ChannelNumber]));
0263:      RegVal = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->IRQ_EN.U);
0264:      if(RegVal == 0U)
0265:      {
0266:          ErrorFlag = E_OK;
0267:      }
0268:      return(ErrorFlag);
0269:  }
0270:
0271:  IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomIrqEnCheck(uint8 ModuleNumber,\
0272:      uint8 ChannelNumber)

```

Code Listing 9 Local functions to check SFR values

```

0273:  {
0274:      Ifx_GTM_ATOM_CH_TYPE* AtomChannelRegPtr;
0275:      uint32 RegVal;
0276:      Std_ReturnType ErrorFlag = E_NOT_OK;
0277:      /* For Temp storage of Reg Value */
0278:      AtomChannelRegPtr = &((*Ifx_GTM_ATOMx*)(void *)
0279:          (MODULE_GTM.ATOM)).ATOM_CH[ModuleNumber].CH[ChannelNumber]);
0280:      RegVal = PWM_SFR_INIT_USER_MODE_READ32(AtomChannelRegPtr->IRQ_EN.U);
0281:      if(RegVal == 0U)
0282:      {
0283:          ErrorFlag = E_OK;
0284:      }
0285:      return(ErrorFlag);
0286:  }
0287:
0288:  IFX_LOCAL_INLINE uint32 Pwm_lInitVPSr0Extract(const
    Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0289:  {
0290:      uint32          Cm0Value;
0291:      Cm0Value        = LocalPwmChanPtr->PeriodDefault;
0292:      return(Cm0Value);
0293:  }
0294:
0295:  IFX_LOCAL_INLINE uint32 Pwm_lInitFPSr0Extract(const
    Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0296:  {
0297:      uint32          Cm0Value;
0298:      Cm0Value        = LocalPwmChanPtr->PeriodDefault;
0299:      return(Cm0Value);
0300:  }
0301:
0302:  IFX_LOCAL_INLINE uint32 Pwm_lInitFPS_SR0Extract( const
    Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0303:  {
0304:      uint32          CalcSR0Value;
0305:      uint32          Period;
0306:      uint32          ScaledDuty;
0307:      uint32          ShiftVal;
0308:      uint32          MainChannel;
0309:      uint32          Duty;
0310:      uint8           ShiftedSpl;
0311:
0312:      ShiftedSpl      = Pwm_lDemoShiftSpl(LocalPwmChanPtr->ChannelInfo);
0313:      Duty             = LocalPwmChanPtr->DutycycleDefault;
0314:      MainChannel      = Pwm_lDemoReferChan(LocalPwmChanPtr->ChannelInfo);
0315:      Period = ((DemoGlobalConfigPtr->ChannelConfigPtr) + MainChannel )-
    >PeriodDefault;
0316:      ScaledDuty       = (uint32)Pwm_lDemoGetAbsvalEdge(Period, Duty);
0317:      ShiftVal         = (uint32)Pwm_lDemoGetShiftLead( Period,

```

Code Listing 9 Local functions to check SFR values

```

0318:                                     (Pwm_17_Gtm_PeriodType) (LocalPwmChanPtr->ShiftValue) );
0319:
0320:     if(ShiftedSpl == 1U)
0321:     {
0322:         CalcSR0Value = Period;
0323:     }
0324:
0325:     else
0326:     {
0327:
0328:         if((ScaledDuty == 0U))
0329:         {
0330:             CalcSR0Value = PWM_SR0_VALUE;
0331:         }
0332:
0333:         else if(ScaledDuty == Period)
0334:         {
0335:             CalcSR0Value = (Period+1U);
0336:         }
0337:         else
0338:         {
0339:             CalcSR0Value = (Pwm_1DemoGtmCounterVal(ShiftVal, Period, ScaledDuty));
0340:         }
0341:
0342:     }
0343:     return (CalcSR0Value);
0344: }
0345:
0346: IFX_LOCAL_INLINE uint32 Pwm_1InitCASr0Extract( const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0347: {
0348:     uint32                CalcSR0Value;
0349:     uint32                Period;
0350:     uint32                ScaledDuty;
0351:     uint32                ShiftVal;
0352:     uint32                MainChannel;
0353:     uint32                Duty;
0354:
0355:     Duty                  = LocalPwmChanPtr->DutycycleDefault;
0356:     MainChannel           = Pwm_1DemoReferChan(LocalPwmChanPtr->ChannelInfo);
0357:     Period = ((DemoGlobalConfigPtr->ChannelConfigPtr) + MainChannel )-
>PeriodDefault;
0358:     ScaledDuty = Pwm_1DemoGetAbsvalEdge(Period, Duty);
0359:     ShiftVal = (uint32)Pwm_1DemoGetCenterLead(Period, ScaledDuty);
0360:     if((ScaledDuty == 0U))
0361:     {
0362:         CalcSR0Value = PWM_SR0_VALUE;
0363:     }
0364:     else if(ScaledDuty == Period)

```

Code Listing 9 Local functions to check SFR values

```

0365:     {
0366:         CalcSR0Value = (Period+1U);
0367:     }
0368:     else
0369:     {
0370:         CalcSR0Value = (Pwm_1DemoGtmCounterVal(ShiftVal, Period, ScaledDuty));
0371:     }
0372:     return (CalcSR0Value);
0373: }
0374:
0375: IFX_LOCAL_INLINE uint32 Pwm_1InitVPSr1Extract(const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0376: {
0377:     uint32                Period;
0378:     uint32                CmlValue;
0379:     uint32                Duty;
0380:     Period                = LocalPwmChanPtr->PeriodDefault;
0381:     Duty                  = (uint32)LocalPwmChanPtr->DutycycleDefault;
0382:     CmlValue              = (uint32)Pwm_1DemoGetAbsvalEdge(Period, Duty);
0383:     return(CmlValue);
0384: }
0385:
0386: IFX_LOCAL_INLINE uint32 Pwm_1InitFPSr1Extract(const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0387: {
0388:     uint32                Period;
0389:     uint32                CmlValue;
0390:     uint32                Duty;
0391:     Period                = LocalPwmChanPtr->PeriodDefault;
0392:     Duty                  = (uint32)LocalPwmChanPtr->DutycycleDefault;
0393:     CmlValue              = (uint32)Pwm_1DemoGetAbsvalEdge(Period, Duty);
0394:     return(CmlValue);
0395: }
0396: }
0397:
0398: IFX_LOCAL_INLINE uint32 Pwm_1InitFPS_SR1Extract( const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0399: {
0400:     uint32                CalcSR1Value;
0401:     uint32                Period;
0402:     uint32                ScaledDuty;
0403:     uint32                ShiftVal;
0404:     uint32                MainChannel;
0405:     uint32                Duty;
0406:     uint8                ShiftedSpl;
0407:
0408:     ShiftedSpl            = Pwm_1DemoShiftSpl(LocalPwmChanPtr->ChannelInfo);
0409:     Duty                  = LocalPwmChanPtr->DutycycleDefault;
0410:     MainChannel            = Pwm_1DemoReferChan(LocalPwmChanPtr->ChannelInfo);

```

Code Listing 9 Local functions to check SFR values

```

0411:   Period = ((DemoGlobalConfigPtr->ChannelConfigPtr) + MainChannel )-
>PeriodDefault;
0412:   ScaledDuty      = (uint32)Pwm_1DemoGetAbsvalEdge(Period, Duty);
0413:   ShiftVal        = (uint32)Pwm_1DemoGetShiftLead( Period,
0414:               (Pwm_17_Gtm_PeriodType) (LocalPwmChanPtr->ShiftValue) );
0415:
0416:   if(ShiftedSpl == 1U)
0417:   {
0418:       CalcSR1Value = ScaledDuty;
0419:   }
0420:   else
0421:   {
0422:       if((ScaledDuty == 0U))
0423:       {
0424:           CalcSR1Value = Period+1U;
0425:       }
0426:       else if(ScaledDuty == Period)
0427:       {
0428:           CalcSR1Value = PWM_SR1_VALUE;
0429:       }
0430:       else
0431:       {
0432:           CalcSR1Value = ShiftVal;
0433:       }
0434:   }
0435:   return (CalcSR1Value);
0436: }
0437:
0438: IFX_LOCAL_INLINE uint32 Pwm_1InitCASr1Extract( const
Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0439: {
0440:     uint32          CalcSR1Value;
0441:     uint32          Period;
0442:     uint32          ScaledDuty;
0443:     uint32          ShiftVal;
0444:     uint32          MainChannel;
0445:     uint32          Duty;
0446:
0447:     Duty            = LocalPwmChanPtr->DutycycleDefault;
0448:     MainChannel      = Pwm_1DemoReferChan(LocalPwmChanPtr->ChannelInfo);
0449:     Period = ((DemoGlobalConfigPtr->ChannelConfigPtr) + MainChannel )-
>PeriodDefault;
0450:     ScaledDuty = Pwm_1DemoGetAbsvalEdge(Period, Duty);
0451:     ShiftVal = (uint32)Pwm_1DemoGetCenterLead(Period, ScaledDuty);
0452:     if((ScaledDuty == 0U))
0453:     {
0454:         CalcSR1Value = Period+1U;
0455:     }
0456:     else if(ScaledDuty == Period)

```

Code Listing 9 Local functions to check SFR values

```

0457:     {
0458:         CalcSR1Value = PWM_SR1_VALUE;
0459:     }
0460:     else
0461:     {
0462:         CalcSR1Value = ShiftVal;
0463:     }
0464:     return (CalcSR1Value);
0465: }
0466:
0467: IFX_LOCAL_INLINE uint32 Pwm_lCalcSr0Val( const Pwm_17_Gtm_ChannelConfigType
    *LocalPwmChanPtr)
0468: {
0469:     uint32 CalcSr0Value;
0470:     uint8 ChannelKind;
0471:     ChannelKind = Pwm_lDemoChanClass(LocalPwmChanPtr->ChannelInfo);
0472:
0473:     switch (ChannelKind)
0474:     {
0475:         case PWM_VARIABLE_PERIOD:
0476:             CalcSr0Value = Pwm_lInitVPSr0Extract(LocalPwmChanPtr);
0477:             break;
0478:
0479:         case PWM_FIXED_PERIOD:
0480:             CalcSr0Value = Pwm_lInitFPSr0Extract(LocalPwmChanPtr);
0481:             break;
0482:
0483:         case PWM_FIXED_PERIOD_SHIFTED:
0484:             CalcSr0Value = Pwm_lInitFPS_SR0Extract(LocalPwmChanPtr);
0485:             break;
0486:         default:
0487:             CalcSr0Value = Pwm_lInitCASr0Extract(LocalPwmChanPtr);
0488:             break;
0489:     }
0490:     return(CalcSr0Value);
0491: }
0492:
0493: IFX_LOCAL_INLINE uint32 Pwm_lCalcSr1Val( const Pwm_17_Gtm_ChannelConfigType
    *LocalPwmChanPtr)
0494: {
0495:     uint32 CalcSr1Value;
0496:     uint8 ChannelKind;
0497:     ChannelKind = Pwm_lDemoChanClass(LocalPwmChanPtr->ChannelInfo);
0498:
0499:     switch (ChannelKind)
0500:     {
0501:         case PWM_VARIABLE_PERIOD:
0502:             CalcSr1Value = Pwm_lInitVPSr1Extract(LocalPwmChanPtr);
0503:             break;

```


Code Listing 9 Local functions to check SFR values

```

0504:
0505:     case PWM_FIXED_PERIOD:
0506:         CalcSrlValue = Pwm_lInitFPSr1Extract(LocalPwmChanPtr);
0507:         break;
0508:
0509:     case PWM_FIXED_PERIOD_SHIFTED:
0510:         CalcSrlValue = Pwm_lInitFPS_SR1Extract(LocalPwmChanPtr);
0511:         break;
0512:     default:
0513:         CalcSrlValue = Pwm_lInitCASr1Extract(LocalPwmChanPtr);
0514:         break;
0515:     }
0516:     return(CalcSrlValue);
0517: }
0518:
0519: IFX_LOCAL_INLINE Std_ReturnType Pwm_lTomSr0SrlCheck(uint8 ModuleNumber,\
0520:                                                     uint8 ChannelNumber,\
0521:                                                     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0522: {
0523:     Ifx_GTM_TOM_CH_TYPE* TomChannelRegPtr;    /* Ptr to TOM Channel Reg          */
0524:     Std_ReturnType ErrorFlag = E_NOT_OK;
0525:     uint32 RegValSR0;                        /* For Temp storage of Reg Value */
0526:     uint32 RegValCM0;                        /* For Temp storage of Reg Value */
0527:     uint32 SR0Value;
0528:     uint32 RegValSR1;                        /* For Temp storage of Reg Value */
0529:     uint32 RegValCM1;                        /* For Temp storage of Reg Value */
0530:     uint32 SR1Value;
0531:     uint8 Coherency;
0532:     #if(PWM_DUTYCYCLE_UPDATED_ENDPERIOD == STD_OFF)
0533:     Coherency = Pwm_lDemoCoherency(LocalPwmChanPtr->ChannelInfo);
0534:     #else
0535:     Coherency = PWM_COHERENT;
0536:     #endif
0537:
0538:     TomChannelRegPtr = &(((Ifx_GTM_TOMx*)(volatile void*) \
0539:                             (MODULE_GTM.TOM)).TOM_CH[ModuleNumber].CH[ChannelNumber]));
0540:     RegValSR0 = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->SR0.U);
0541:     RegValCM0 = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->CM0.U);
0542:     RegValSR1 = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->SR1.U);
0543:     RegValCM1 = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->CM1.U);
0544:     SR0Value = Pwm_lCalcSr0Val(LocalPwmChanPtr);
0545:     SR1Value = Pwm_lCalcSr1Val(LocalPwmChanPtr);
0546:     if(Coherency)
0547:     {
0548:         if((RegValSR0 == SR0Value)&&(RegValSR1 == SR1Value))
0549:         {
0550:             ErrorFlag = E_OK;
0551:         }
0552:     }

```

Code Listing 9 Local functions to check SFR values

```

0553:     else
0554:     {
0555:         if((RegValSR0 == SR0Value) && (RegValCM0 == SR0Value)&&\
0556:         (RegValSR1 == SR1Value) && (RegValCM1 == SR1Value))
0557:         {
0558:             ErrorFlag = E_OK;
0559:         }
0560:     }
0561:     return(ErrorFlag);
0562: }
0563:
0564: IFX_LOCAL_INLINE Std_ReturnType Pwm_lAtomSr0Sr1Check(uint8 ModuleNumber,\
0565:             uint8 ChannelNumber,\
0566:             const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0567: {
0568:     Ifx_GTM_ATOM_CH_TYPE* AtomChannelRegPtr;    /* Ptr to TOM Channel Reg
0569: */
0569:     Std_ReturnType ErrorFlag = E_NOT_OK;
0570:     uint32 RegValSR0;                          /* For Temp storage of Reg Value */
0571:     uint32 RegValCM0;                          /* For Temp storage of Reg Value */
0572:     uint32 SR0Value;
0573:     uint32 RegValSR1;                          /* For Temp storage of Reg Value */
0574:     uint32 RegValCM1;                          /* For Temp storage of Reg Value */
0575:     uint32 SR1Value;
0576:     uint8 Coherency;
0577:     #if(PWM_DUTYCYCLE_UPDATED_ENDPERIOD == STD_OFF)
0578:     Coherency = Pwm_lDemoCoherency(LocalPwmChanPtr->ChannelInfo);
0579:     #else
0580:     Coherency = PWM_COHERENT;
0581:     #endif
0582:
0583:     AtomChannelRegPtr = &((* (Ifx_GTM_ATOMx*) (void *)
0584:             (MODULE_GTM.ATOM)).ATOM_CH[ModuleNumber].CH[ChannelNumber]);
0585:     RegValSR0 = PWM_SFR_INIT_USER_MODE_READ32 (AtomChannelRegPtr->SR0.U);
0586:     RegValCM0 = PWM_SFR_INIT_USER_MODE_READ32 (AtomChannelRegPtr->CM0.U);
0587:     RegValSR1 = PWM_SFR_INIT_USER_MODE_READ32 (AtomChannelRegPtr->SR1.U);
0588:     RegValCM1 = PWM_SFR_INIT_USER_MODE_READ32 (AtomChannelRegPtr->CM1.U);
0589:     SR0Value = Pwm_lCalcSr0Val (LocalPwmChanPtr);
0590:     SR1Value = Pwm_lCalcSr1Val (LocalPwmChanPtr);
0591:
0592:     if(Coherency)
0593:     {
0594:         if((RegValSR0 == SR0Value)&&(RegValSR1 == SR1Value))
0595:         {
0596:             ErrorFlag = E_OK;
0597:         }
0598:     }
0599:     else
0600:     {

```

Code Listing 9 Local functions to check SFR values

```

0601:     if((RegValSR0 == SR0Value) && (RegValCM0 == SR0Value)&&\
0602:       (RegValSR1 == SR1Value) && (RegValCM1 == SR1Value))
0603:     {
0604:         ErrorFlag = E_OK;
0605:     }
0606: }
0607: return(ErrorFlag);
0608: }
0609:
0610: IFX_LOCAL_INLINE Std_ReturnType Pwm_1TomChCtrlCheck( \
0611:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,\
0612:     uint8 ModuleNumber,uint8 ChannelNumber)
0613: {
0614:     Std_ReturnType ErrorFlag = E_OK;
0615:     Ifx_GTM_TOM_CH_TYPE* TomChannelRegPtr;
0616:     uint32 RegValue;
0617:     uint32 TempRegValue;
0618:     uint8 IsReference;
0619:     uint8 ResetShift;
0620:     uint8 Polarity = 0U;
0621:     uint8 ChannelType;
0622:     uint8 ShiftedSpl;
0623:
0624:     ChannelType = Pwm_1DemoGetChannelType(LocalPwmChanPtr);
0625:     IsReference  = Pwm_1DemoIsReference(LocalPwmChanPtr->ChannelInfo);
0626:     ResetShift   = Pwm_1DemoResetShift(LocalPwmChanPtr->ChannelInfo);
0627:
0628:     TomChannelRegPtr = &((((Ifx_GTM_TOMx*) (void *) (MODULE_GTM.TOM)). \
0629:                             TOM_CH[ModuleNumber])).CH[ChannelNumber]);
0630:     RegValue = PWM_SFR_INIT_USER_MODE_READ32(TomChannelRegPtr->CTRL.U);
0631:
0632:     TempRegValue = (IsReference << PWM_GTM_CTRL_TRIGOUT);
0633:     if((RegValue & (PWM_GTM_CTRL_TRIGOUT_MASK<<PWM_GTM_CTRL_TRIGOUT)) !=
TempRegValue)
0634:     {
0635:         ErrorFlag = E_NOT_OK;
0636:     }
0637:
0638:     TempRegValue = ResetShift<<PWM_GTM_CTRL_RST_CCU0;
0639:
0640:     if (ErrorFlag != E_NOT_OK)
0641:     {
0642:         if((RegValue & ( PWM_GTM_CTRL_RST_CCU0_MASK << PWM_GTM_CTRL_RST_CCU0)) !=
TempRegValue)
0643:         {
0644:             ErrorFlag = E_NOT_OK;
0645:         }
0646:     }
0647:

```

Code Listing 9 Local functions to check SFR values

```

0648:     if (ErrorFlag != E_NOT_OK)
0649:     {
0650:         Polarity      = Pwm_1DemoPolarity(LocalPwmChanPtr->ChannelInfo);
0651:         ShiftedSpl     = Pwm_1DemoShiftSpl (LocalPwmChanPtr->ChannelInfo);
0652:
0653:         if((ChannelType == PWM_SHIFTED_CHANNEL) && (ShiftedSpl !=
            PWM_SHIFTED_RELATED_CHANNEL))
0654:         {
0655:
0656:             Polarity = (Polarity ^ 1U);
0657:             Polarity = (Polarity & PWM_GTM_TOM_CH_CTRL_SL_MSK);
0658:
0659:         }
0660:         if((RegValue & (Polarity << PWM_GTM_TOM_CH_CTRL_SL_OFF)) != (Polarity <<
            PWM_GTM_TOM_CH_CTRL_SL_OFF))
0661:         {
0662:             ErrorFlag = E_NOT_OK;
0663:         }
0664:     }
0665:
0666:     if (ErrorFlag != E_NOT_OK)
0667:     {
0668:         ErrorFlag = Pwm_1DemoVerifyTomCtrlValues(RegValue);
0669:     }
0670:     return (ErrorFlag);
0671: }
0672:
0673: IFX_LOCAL_INLINE Std_ReturnType Pwm_1AtomChCtrlCheck( \
0674:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr, \
0675:     uint8 ModuleNumber, uint8 ChannelNumber)
0676: {
0677:     Std_ReturnType ErrorFlag = E_OK;
0678:     Ifx_GTM_ATOM_CH_TYPE* AtomChannelRegPtr;
0679:     uint32 RegValue;
0680:     uint32 TempRegValue;
0681:     uint8 IsReference;
0682:     uint8 ResetShift;
0683:     uint8 Polarity = 0U;
0684:     uint8 ChannelType;
0685:     uint8 ShiftedSpl;
0686:
0687:     ChannelType = Pwm_1DemoGetChannelType(LocalPwmChanPtr);
0688:     IsReference  = Pwm_1DemoIsReference(LocalPwmChanPtr->ChannelInfo);
0689:     ResetShift   = Pwm_1DemoResetShift (LocalPwmChanPtr->ChannelInfo);
0690:     AtomChannelRegPtr = &((*(Ifx_GTM_ATOMx*)(void *)
0691:         (MODULE_GTM.ATOM)).ATOM_CH[ModuleNumber].CH[ChannelNumber]);
0692:     RegValue = PWM_SFR_INIT_USER_MODE_READ32 (AtomChannelRegPtr->CTRL.U);
0693:
0694:     TempRegValue = (IsReference << PWM_GTM_CTRL_TRIGOUT);

```

Code Listing 9 Local functions to check SFR values

```

0695:     if((RegValue & (PWM_GTM_CTRL_TRIGOUT_MASK<<PWM_GTM_CTRL_TRIGOUT)) !=
        TempRegValue)
0696:     {
0697:         ErrorFlag =  E_NOT_OK;
0698:     }
0699:
0700:     TempRegValue = ResetShift<<PWM_GTM_CTRL_RST_CCU0;
0701:     if (ErrorFlag != E_NOT_OK)
0702:     {
0703:         if((RegValue & ( PWM_GTM_CTRL_RST_CCU0_MASK << PWM_GTM_CTRL_RST_CCU0)) !=
            TempRegValue)
0704:         {
0705:             ErrorFlag =  E_NOT_OK;
0706:         }
0707:     }
0708:
0709:     if (ErrorFlag != E_NOT_OK)
0710:     {
0711:         Polarity      = Pwm_lDemoPolarity(LocalPwmChanPtr->ChannelInfo);
0712:         ShiftedSpl     = Pwm_lDemoShiftSpl(LocalPwmChanPtr->ChannelInfo);
0713:         if((ChannelType == PWM_SHIFTED_CHANNEL) && \
0714:            (ShiftedSpl != PWM_SHIFTED_RELATED_CHANNEL))
0715:         {
0716:             Polarity = (Polarity ^ 1U);
0717:             Polarity = (Polarity & PWM_GTM_ATOM_CH_CTRL_SL_MSK);
0718:         }
0719:
0720:
0721:         if((RegValue & (Polarity << PWM_GTM_ATOM_CH_CTRL_SL_OFF)) != (Polarity <<
            PWM_GTM_ATOM_CH_CTRL_SL_OFF))
0722:         {
0723:             ErrorFlag =  E_NOT_OK;
0724:         }
0725:     }
0726:     if (ErrorFlag != E_NOT_OK)
0727:     {
0728:         ErrorFlag =  Pwm_lDemoVerifyAtomCtrlValues(RegValue);
0729:     }
0730:     return (ErrorFlag);
0731: }
0732:
0733: IFX_LOCAL_INLINE Std_ReturnType Pwm_lChannelRegisterVerify(const
        Pwm_17_Gtm_ConfigType *ConfigPtr)
0734: {
0735:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr;
0736:     Pwm_ChannelIdentifierType GlobalChannelId;
0737:     Std_ReturnType ErrorFlag = E_OK;
0738:     uint32 ModuleNumber;
0739:     uint8  CellUsed;

```

Code Listing 9 Local functions to check SFR values

```

0740:     uint8 ChannelNumber;
0741:
0742:     LocalPwmChanPtr = ConfigPtr->ChannelConfigPtr;
0743:     uint32 PwmChannelNumber;
0744:
0745:     for ( PwmChannelNumber = (Pwm_17_Gtm_ChannelType)0;
0746:           PwmChannelNumber < ConfigPtr->MaxChannels;
0747:           PwmChannelNumber++)
0748:     )
0749:     {
0750:         CellUsed = Pwm_1DemoAssignedHW(LocalPwmChanPtr->ChannelInfo);
0751:         GlobalChannelId = Pwm_1DemoGtmChannelIdentifier(CellUsed);
0752:         ModuleNumber = GlobalChannelId.ModuleNo;
0753:         ChannelNumber = GlobalChannelId.ChannelNo;
0754:
0755:         if((GlobalChannelId.ModuleId) == GTM_TOM_MODULE)
0756:         {
0757:             ErrorFlag = Pwm_1TomIrqEnCheck(ModuleNumber,ChannelNumber);
0758:             if(ErrorFlag != E_NOT_OK)
0759:             {
0760:                 ErrorFlag = Pwm_1TomSr0SrlCheck(ModuleNumber,ChannelNumber,\
0761:                                                  LocalPwmChanPtr);
0762:             }
0763:             else
0764:             {
0765:                 break ;
0766:             }
0767:             if(ErrorFlag != E_NOT_OK)
0768:             {
0769:                 ErrorFlag = Pwm_1TomChCtrlCheck(LocalPwmChanPtr,ModuleNumber,\
0770:                                                  ChannelNumber);
0771:                 if(ErrorFlag == E_NOT_OK)
0772:                 {
0773:                     break ;
0774:                 }
0775:             }
0776:             else
0777:             {
0778:                 break ;
0779:             }
0780:         }
0781:         if((GlobalChannelId.ModuleId) == GTM_ATOM_MODULE)
0782:         {
0783:             ErrorFlag = Pwm_1AtomIrqEnCheck(ModuleNumber,ChannelNumber);
0784:
0785:             if(ErrorFlag != E_NOT_OK)
0786:             {
0787:                 ErrorFlag = Pwm_1AtomSr0SrlCheck(ModuleNumber,ChannelNumber,\
0788:                                                  LocalPwmChanPtr);

```

Code Listing 9 Local functions to check SFR values

```

0789:
0790:     }
0791:     else
0792:     {
0793:         break ;
0794:     }
0795:     if(ErrorFlag != E_NOT_OK)
0796:     {
0797:         ErrorFlag =
Pwm_lAtomChCtrlCheck(LocalPwmChanPtr,ModuleNumber,ChannelNumber);
0798:
0799:         if(ErrorFlag == E_NOT_OK)
0800:         {
0801:             break ;
0802:         }
0803:     }
0804:     else
0805:     {
0806:         break ;
0807:     }
0808: }
0809:
0810: LocalPwmChanPtr++;
0811: }
0812: return(ErrorFlag);
0813: }
0814:
0815: IFX_LOCAL_INLINE Std_ReturnType Pwm_lRegisterCheck(const Pwm_17_Gtm_ConfigType
*ConfigPtr)
0816: {
0817:     Std_ReturnType ErrorFlag;
0818:     ErrorFlag =E_OK;
0819:     ErrorFlag = Pwm_lGroupRegisterVerify(ConfigPtr);
0820:     if(ErrorFlag != E_NOT_OK)
0821:     {
0822:         ErrorFlag = Pwm_lChannelRegisterVerify(ConfigPtr);
0823:     }
0824:     return(ErrorFlag);
0825: }
0826:

```

Code Listing 10 Local functions to check Global variables

```

001:
002:     /*****
003: Local functions to check Global variables
004: *****/
005:
006: IFX_LOCAL_INLINE Std_ReturnType Pwm_lCheck_DutyFlag_ChannelKind(\

```

Code Listing 10 Local functions to check Global variables

```

007:         Pwm_17_Gtm_ChannelType Channel,uint8 ChannelKind)
008:     {
009:         Std_ReturnType ErrorFlag = E_OK;
010:         #if (PWM_SAFETY_ENABLE == STD_ON)
011:             uint8 ChSignal;
012:         #endif
013:         Pwm_17_Gtm_ChannelType ChIndex ;
014:         ChIndex = Pwm_1DemoFindChannelIndex(Channel);
015:         #if (PWM_SAFETY_ENABLE == STD_ON)
016:             ChSignal = Pwm_1DemoFindChannelSignal(Channel);
017:             if (ChSignal == PWM_ASIL_TYPE)
018:             {
019:                 #if (PWM_MAX_ASIL_CHANNELS != 0)
020:                     if((PtrGlobChn_Asil[(ChIndex)] &
021:                        (PWM_BIT_SET) << (PWM_DUTY_0_OR_100_UPDATE_FLGPOS)) ==
022:                        0U)
023:                     {
024:                         ErrorFlag = E_NOT_OK;
025:                     }
026:                     if(ErrorFlag != E_NOT_OK)
027:                     {
028:                         if((PtrGlobChn_Asil[(ChIndex)] & PWM_EXTRACT_CHAN_KIND) !=\
029:                            (uint32)ChannelKind)
030:                         {
031:                             ErrorFlag = E_NOT_OK;
032:                         }
033:                     }
034:                 #endif
035:             }
036:             else
037:             {
038:                 #if (PWM_MAX_QM_CHANNELS != 0)
039:                     if((PtrGlobChn_Qm[(ChIndex)] &\
040:                        (PWM_BIT_SET << PWM_DUTY_0_OR_100_UPDATE_FLGPOS)) == 0U)
041:                     {
042:                         ErrorFlag = E_NOT_OK;
043:                     }
044:                     if(ErrorFlag != E_NOT_OK)
045:                     {
046:                         if((PtrGlobChn_Qm[(ChIndex)] & PWM_EXTRACT_CHAN_KIND) !=\
047:                            (uint32)ChannelKind)
048:                         {
049:                             ErrorFlag = E_NOT_OK;
050:                         }
051:                     }
052:                 #endif
053:             }
054:         }

```


Code Listing 10 Local functions to check Global variables

```

055:     return (ErrorFlag);
056: }
057:
058: IFX_LOCAL_INLINE Std_ReturnType Pwm_lCheck_DutyReset_ChannelKind(\
059:     Pwm_17_Gtm_ChannelType Channel,uint8 ChannelKind)
060: {
061:     Std_ReturnType ErrorFlag = E_OK;
062:     #if (PWM_SAFETY_ENABLE == STD_ON)
063:     uint8 ChSignal;
064:     #endif
065:     Pwm_17_Gtm_ChannelType ChIndex ;
066:     ChIndex = Pwm_lDemoFindChannelIndex(Channel);
067:     #if (PWM_SAFETY_ENABLE == STD_ON)
068:     ChSignal = Pwm_lDemoFindChannelSignal(Channel);
069:     if (ChSignal == PWM_ASIL_TYPE)
070:     {
071:         #if (PWM_MAX_ASIL_CHANNELS != 0)
072:         if((PtrGlobChn_Asil[(ChIndex)]&\
073:             (PWM_BIT_SET) << (PWM_DUTY_0_OR_100_UPDATE_FLGPOS)) !=
074:             0U)
075:         {
076:             ErrorFlag = E_NOT_OK;
077:         }
078:         if(ErrorFlag != E_NOT_OK)
079:         {
080:             if((PtrGlobChn_Asil[ChIndex] & PWM_EXTRACT_CHAN_KIND) !=\
081:                 (uint32)ChannelKind)
082:             {
083:                 ErrorFlag = E_NOT_OK;
084:             }
085:         }
086:     }
087:     else
088:     #endif
089:     {
090:         #if (PWM_MAX_QM_CHANNELS != 0)
091:         if((PtrGlobChn_Qm[(ChIndex)]&\
092:             (PWM_BIT_SET << PWM_DUTY_0_OR_100_UPDATE_FLGPOS)) != 0U)
093:         {
094:             ErrorFlag = E_NOT_OK;
095:         }
096:         if(ErrorFlag != E_NOT_OK)
097:         {
098:             if((PtrGlobChn_Qm[(ChIndex)] & PWM_EXTRACT_CHAN_KIND) !=\
099:                 (uint32)ChannelKind)
100:             {
101:                 ErrorFlag = E_NOT_OK;
102:             }

```

Code Listing 10 Local functions to check Global variables

```

0103:     }
0104:
0105:     }
0106:     #endif
0107: }
0108: return (ErrorFlag);
0109: }
0110:
0111: static Std_ReturnType Pwm_lCheckInitVariablePeriod(\
0112:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32
    ChannelNumber)
0113: {
0114:     Std_ReturnType ErrorFlag;
0115:     uint32          Period;
0116:     uint32          ScaledDuty;
0117:     uint32          Duty;
0118:     Period           = LocalPwmChanPtr->PeriodDefault;
0119:     Duty             = (uint32)LocalPwmChanPtr->DutycycleDefault;
0120:     ScaledDuty = (uint32)Pwm_lDemoGetAbsvalEdge(Period, Duty);
0121:     #if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
0122:     #if (PWM_NOTIF_FOR_100_0_ENABLE == STD_OFF)
0123:     if ((ScaledDuty == 0U) || ( ScaledDuty == Period))
0124:     {
0125:         ErrorFlag = Pwm_lCheck_DutyFlag_ChannelKind(ChannelNumber,\
0126:             PWM_VARIABLE_PERIOD);
0127:     }
0128:     else
0129:     #endif
0130:     #endif
0131:     {
0132:         ErrorFlag = Pwm_lCheck_DutyReset_ChannelKind(ChannelNumber,\
0133:             PWM_VARIABLE_PERIOD);
0134:     }
0135:     return (ErrorFlag);
0136: }
0137:
0138: static Std_ReturnType Pwm_lCheckInitFixedPeriod(\
0139:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32
    ChannelNumber)
0140: {
0141:     Std_ReturnType ErrorFlag = E_NOT_OK;
0142:     uint32          Period;
0143:     uint32          ScaledDuty;
0144:     uint32          Duty;
0145:     Period           = LocalPwmChanPtr->PeriodDefault;
0146:     Duty             = LocalPwmChanPtr->DutycycleDefault;
0147:     ScaledDuty = (uint32)Pwm_lDemoGetAbsvalEdge(Period, Duty);
0148:     #if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
0149:     #if (PWM_NOTIF_FOR_100_0_ENABLE == STD_OFF)

```

Code Listing 10 Local functions to check Global variables

```

0150:     if ((ScaledDuty == 0U) || ( ScaledDuty == Period))
0151:     {
0152:         ErrorFlag =
            Pwm_lCheck_DutyFlag_ChannelKind(ChannelNumber, PWM_FIXED_PERIOD);
0153:     }
0154:     else
0155:     #endif
0156:     #endif
0157:     {
0158:         ErrorFlag = Pwm_lCheck_DutyReset_ChannelKind(ChannelNumber, \
0159:             PWM_FIXED_PERIOD);
0160:     }
0161:     return (ErrorFlag);
0162: }
0163:
0164: static Std_ReturnType Pwm_lCheckInitFixedPeriodShifted(\
0165:     const Pwm_17_Gtm_ChannelConfigType * LocalPwmChanPtr, uint32
    ChannelNumber)
0166: {
0167:     Std_ReturnType ErrorFlag = E_NOT_OK;
0168:     uint32
            Period;
0169:     uint32
            ScaledDuty;
0170:     uint32
            MainChannel;
0171:     uint32
            Duty;
0172:     Duty
            = LocalPwmChanPtr->DutycycleDefault;
0173:     MainChannel = Pwm_lDemoReferChan(LocalPwmChanPtr->ChannelInfo);
0174:     Period = ((DemoGlobalConfigPtr->ChannelConfigPtr)+MainChannel)-
        >PeriodDefault;
0175:     ScaledDuty = (uint32)Pwm_lDemoGetAbsvalEdge(Period, Duty);
0176:     #if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
0177:     if ((ScaledDuty == 0U) || ( ScaledDuty == Period))
0178:     {
0179:         ErrorFlag = Pwm_lCheck_DutyFlag_ChannelKind(ChannelNumber, \
0180:             PWM_FIXED_PERIOD_SHIFTED);
0181:     }
0182:     else
0183:     #endif
0184:     {
0185:         ErrorFlag = Pwm_lCheck_DutyReset_ChannelKind(ChannelNumber, \
0186:             PWM_FIXED_PERIOD_SHIFTED);
0187:     }
0188:     return(ErrorFlag);
0189: }
0190:
0191: static Std_ReturnType Pwm_lCheckInitCentreAligned(\
0192:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr, uint32
    ChannelNumber)
0193: {
0194:     Std_ReturnType ErrorFlag = E_NOT_OK;

```

Code Listing 10 Local functions to check Global variables

```

0195:     uint32                Period;
0196:     uint32                ScaledDuty;
0197:     uint32                MainChannel;
0198:     uint32                Duty;
0199:     Duty                  = LocalPwmChanPtr->DutycycleDefault;
0200:     MainChannel = Pwm_1DemoReferChan(LocalPwmChanPtr->ChannelInfo);
0201:     Period = ((DemoGlobalConfigPtr->ChannelConfigPtr)+MainChannel)-
>PeriodDefault;
0202:     ScaledDuty = (uint32)Pwm_1DemoGetAbsvalEdge(Period, Duty);
0203:     #if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
0204:     if ((ScaledDuty == 0U) || ( ScaledDuty == Period))
0205:     {
0206:         ErrorFlag = Pwm_1Check_DutyFlag_ChannelKind(ChannelNumber,\
0207:             PWM_FIXED_PERIOD_CENTER_ALIGNED);
0208:     }
0209:     else
0210:     #endif
0211:     {
0212:         ErrorFlag = Pwm_1Check_DutyReset_ChannelKind(ChannelNumber,\
0213:             PWM_FIXED_PERIOD_CENTER_ALIGNED);
0214:     }
0215:     return(ErrorFlag);
0216: }
0217:
0218: static Std_ReturnType Pwm_1GlobChnVerify(\
0219:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr,uint32
PwmChannelNumber)
0220: {
0221:     Std_ReturnType ErrorFlag = E_NOT_OK;
0222:     uint32 ChannelKind;
0223:     ChannelKind = Pwm_1DemoChanClass(LocalPwmChanPtr->ChannelInfo);
0224:     switch (ChannelKind)
0225:     {
0226:         #if (PWM_VARIABLE_PERIOD_USED == STD_ON)
0227:         case PWM_VARIABLE_PERIOD:
0228:             ErrorFlag = Pwm_1CheckInitVariablePeriod(LocalPwmChanPtr,\
0229:                 PwmChannelNumber);
0230:             break;
0231:         #endif
0232:         #if (PWM_FIXED_PERIOD_USED == STD_ON)
0233:         case PWM_FIXED_PERIOD:
0234:             ErrorFlag =
Pwm_1CheckInitFixedPeriod(LocalPwmChanPtr,PwmChannelNumber);
0235:             break;
0236:         #endif
0237:         #if (PWM_FIXED_PERIOD_SHIFTED_USED == STD_ON)
0238:         case PWM_FIXED_PERIOD_SHIFTED:
0239:             ErrorFlag = Pwm_1CheckInitFixedPeriodShifted(LocalPwmChanPtr,\
0240:                 PwmChannelNumber);

```

Code Listing 10 Local functions to check Global variables

```

0241:         break;
0242:     #endif
0243:     default:
0244:         #if (PWM_USED_FIXED_PERIOD_CENTER_ALIGNED == STD_ON)
0245:             ErrorFlag =
                Pwm_lCheckInitCentreAligned(LocalPwmChanPtr, PwmChannelNumber);
0246:         #endif
0247:         break;
0248:     }
0249:     return (ErrorFlag);
0250: }
0251:
0252: static Std_ReturnType Pwm_lVPCurrentPeriodValVerify(\
0253:     uint32 ChannelNumber, const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr)
0254: {
0255:     Std_ReturnType ErrorFlag = E_OK;
0256:     uint32 Period;
0257:     uint8 ChIndex;
0258:     #if (PWM_SAFETY_ENABLE == STD_ON)
0259:     uint8 ChSignal;
0260:     #endif
0261:     Period = LocalPwmChanPtr->PeriodDefault;
0262:     ChIndex = Pwm_lDemoFindChannelIndex(ChannelNumber);
0263:     #if (PWM_SAFETY_ENABLE == STD_ON)
0264:     ChSignal = Pwm_lDemoFindChannelSignal(ChannelNumber);
0265:     #if (PWM_MAX_ASIL_CHANNELS != 0)
0266:     if (ChSignal == PWM_ASIL_TYPE)
0267:     {
0268:         if(PtrPeriod_Asil[ChIndex] != Period)
0269:         {
0270:             ErrorFlag = E_NOT_OK;
0271:         }
0272:     }
0273:     else
0274:     #endif
0275:     #endif
0276:     {
0277:         #if (PWM_MAX_QM_CHANNELS != 0)
0278:         if(PtrPeriod_Qm[ChIndex] != Period)
0279:         {
0280:             ErrorFlag = E_NOT_OK;
0281:         }
0282:         #endif
0283:     }
0284:     return (ErrorFlag);
0285: }
0286:
0287: static Std_ReturnType Pwm_lOtherCurrentPeriodValVerify(\
0288:     uint32 ChannelNumber)

```

Code Listing 10 Local functions to check Global variables

```

0289:  {
0290:      Std_ReturnType ErrorFlag = E_NOT_OK;
0291:      uint8 ChIndex;
0292:      #if (PWM_SAFETY_ENABLE == STD_ON)
0293:          uint8 ChSignal;
0294:      #endif
0295:      ChIndex = Pwm_lDemoFindChannelIndex(ChannelNumber);
0296:      #if (PWM_SAFETY_ENABLE == STD_ON)
0297:          ChSignal = Pwm_lDemoFindChannelSignal(ChannelNumber);
0298:          #if (PWM_MAX_ASIL_CHANNELS != 0)
0299:              if (ChSignal == PWM_ASIL_TYPE)
0300:              {
0301:                  if(PtrPeriod_Asil[ChIndex] == 0U)
0302:                  {
0303:                      ErrorFlag = E_OK;
0304:                  }
0305:              }
0306:          else
0307:          #endif
0308:          #endif
0309:          {
0310:              #if (PWM_MAX_QM_CHANNELS != 0)
0311:                  if(PtrPeriod_Qm[ChIndex] == 0U)
0312:                  {
0313:                      ErrorFlag = E_OK;
0314:                  }
0315:              #endif
0316:          }
0317:      return (ErrorFlag);
0318:  }
0319:
0320:  /*****
0321:  Local function to check "CurrentPeriodVal" global variable
0322:  *****/
0323:  Std_ReturnType Pwm_lCurrentPeriodValVerify(\
0324:      const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr ,\
0325:      Pwm_17_Gtm_ChannelType PwmChannelNumber)
0326:  {
0327:      Std_ReturnType ErrorFlag = E_NOT_OK;
0328:      uint8 ChannelKind;
0329:      ChannelKind = Pwm_lDemoChanClass(LocalPwmChanPtr->ChannelInfo);
0330:      if(ChannelKind == PWM_VARIABLE_PERIOD)
0331:      {
0332:          ErrorFlag =
0333:          Pwm_lVPCurrentPeriodValVerify(PwmChannelNumber,LocalPwmChanPtr);
0334:      }
0335:      else
0336:      {
0337:          ErrorFlag = Pwm_lOtherCurrentPeriodValVerify(PwmChannelNumber);
0338:      }
0339:  }

```

Code Listing 10 Local functions to check Global variables

```

0337:     }
0338:     return (ErrorFlag);
0339: }
0340:
0341:
0342: /*****
0343: Local function to global variables initialized in PWM Init
0344: *****/
0345: Std_ReturnType Pwm_lGlobalVariableCheck(const Pwm_17_Gtm_ConfigType
    *ConfigPtr)
0346: {
0347:     const Pwm_17_Gtm_ChannelConfigType *LocalPwmChanPtr;
0348:     Std_ReturnType ErrorFlag = E_OK;
0349:     uint32 PwmChannelNumber;
0350:     #if ( (PWM_PB_FIXEDADDR == STD_ON) && (PWM_DEV_ERROR_DETECT == STD_ON) )
0351:     if(PWM_DEMO_INITIALIZED == * Pwm_DemoInitStatus )
0352:     {
0353:         ErrorFlag = E_OK;
0354:     }
0355:     else
0356:     {
0357:         ErrorFlag = E_NOT_OK;
0358:     }
0359:     #endif
0360:     if(E_NOT_OK != ErrorFlag)
0361:     {
0362:         LocalPwmChanPtr = ConfigPtr->ChannelConfigPtr;
0363:         for ( PwmChannelNumber = (Pwm_17_Gtm_ChannelType)0;
0364:             PwmChannelNumber < ConfigPtr->MaxChannels;
0365:             PwmChannelNumber++
0366:         )
0367:         {
0368:             #if (PWM_SET_PERIOD_AND_DUTY_API == STD_ON)
0369:             if(E_NOT_OK != ErrorFlag)
0370:             {
0371:                 ErrorFlag = Pwm_lCurrentPeriodValVerify(LocalPwmChanPtr,\
0372:                     PwmChannelNumber);
0373:             }
0374:             else
0375:             {
0376:                 break ;
0377:             }
0378:             #endif
0379:
0380:             if(E_NOT_OK != ErrorFlag)
0381:             {
0382:                 ErrorFlag = Pwm_lGlobChnVerify(LocalPwmChanPtr,PwmChannelNumber);
0383:             }
0384:             else

```

Code Listing 10 Local functions to check Global variables

```

0385:      {
0386:          break ;
0387:      }
0388:
0389:      LocalPwmChanPtr++;
0390:  }
0391:  }
0392:  return (ErrorFlag);
0393:  }
0394:

```

9.4 API Usages

9.4.1 Set Duty Cycle

The following code snippet shows how to call Pwm_17_Gtm_SetDutyCycle API.

Code Listing 11 Driver Set Dutycycle

```

0395:      /* Change Duty to 25% when PwmDutyShiftInTicks is OFF*/
0396:      Pwm_17_Gtm_SetDutyCycle(PwmChannel0,0x2000);
0397:
0398:      /* Change Duty to 25% when PwmDutyShiftInTicks is ON*/
0399:      /* For channel 0, the period is 48, 25% would result in 12 ticks */
0400:      Pwm_17_Gtm_SetDutyCycle(PwmChannel0,12);

```

9.4.2 Set Period and Duty Cycle

The following code snippet shows how to call Pwm_17_Gtm_SetPeriodAndDuty API.

Code Listing 12 Driver Set Period and Duty Cycle

```

0401:      /* Change Duty to 25% and period 60 ticks when PwmDutyShiftInTicks
      is OFF */
0402:      Pwm_17_Gtm_SetPeriodAndDuty(PwmChannel4,60,0x2000);
0403:
0404:      /* Change Duty to 25% and period 60 ticks when PwmDutyShiftInTicks is ON
      */
0405:      /* For Period 60, 25% would result in 15 ticks */
0406:      Pwm_17_Gtm_SetPeriodAndDuty(PwmChannel4,60,15);

```

9.4.3 Notification and other APIs

The following **Code Listing 13** shows how to call notification APIs, Pwm_17_Gtm_SetOutputToldle and Pwm_17_Gtm_GetOutputState.

Code Listing 13 Driver Utility API's

```

0407:      /* Pwm Initialization */
0408:      Pwm_17_Gtm_Init(&Pwm_ConfigRoot[0]);
0409:      /* Enable Notification*/

```


Code Listing 13 Driver Utility API's

```

0410:      Pwm_17_Gtm_EnableNotification(PwmChannel4,PWM_RISING_EDGE);
0411:      /* Disable Notification */
0412:      Pwm_17_Gtm_DisableNotification(PwmChannel4);
0413:      /* Set Output to Idle */
0414:      Pwm_17_Gtm_SetOutputToIdle(PwmChannel4);
0415:      /* Get Output State */
0416:      State = Pwm_17_Gtm_GetOutputState(PwmChannel4);

```

10 Limitations and Assumptions

10.1 Assumptions and Deviations from Software Specification

1. For PWM_FIXED_PERIOD_SHIFTED channels, for both 0 shift and 100% shift values, the channel behaves as edge aligned channel.
2. Duty cycle in ticks:

According to AUTOSAR specification, duty cycle and shift values are relative to period. However support is provided to enter duty cycle and shift values in ticks, by introducing a config parameter PwmDutyShiftInTicks. When this parameter is true, the duty cycle and shift values are entered as absolute ticks. Setting this parameter to false enables the user to enter the duty cycle and shift value as percentage (relative to period) according to AUTOSAR specification.

When PwmDutyShiftInTicks is false:

Duty and Shift values are relative. Hence the range is 0x0 to 0x8000 as per Autosar:

0 – 0%

0x8000 – 100%

When PwmDutyShiftInTicks is true:

Duty and Shift values are entered as timer ticks. Hence the range is 0 to Period ticks.

0 – 0%

Value same as period ticks – 100%

When the duty ticks value is more than period value, 100% duty cycle is produced.

In case of shift channels, when the shift equals period value, it behaves as edge aligned channel, similar to having zero shift value.

The Pwm module's user should ensure the integrity if several function calls are made during run time in different tasks or ISRs for the same PWM channel.

The users of the Pwm module should not call the function Pwm_Init during a running operation.

The PWM Driver depends on PORT Driver: To set the port pin functionality.

The PWM Driver depends on the MCU Driver: To set prescaler, system clock and PLL.

The PWM Driver depends on the DET: Development Error Tracer in Development mode.

The Pwm module's environment should not call any function of the Pwm module before having called Pwm_Init.

Resource conflict is handled in xdm of GTM module

The SFR's used by the PWM driver are initialized to default value by the GTM driver.

In API Pwm_17_Gtm_SyncGrpUpdateDuties, when safety is enabled, the user shall add an additional array element at the end of the Duty cycle array, which is pointed by the argument (PwmChDutyPtr). This element shall have a marker same as the configuration value of PWMDutycycleBufferMarker. It is recommended that the value configured is not to be a valid DUTY cycle value.

Pwm_17_Gtm_Init shall be invoked by the user only after PLL setup (Using Mcu_DistributePllClock) because if PLL is not set correctly then it may lead to wrong signal generation.

The PWM driver deviations from the specification are listed below:

1. Deviation from PWM136_Conf: Multiplicity of PwmChannelClass parameter is set to 1 irrespective of DET switch, since the Pwm channel class configuration is important for the driver implementation even if DET is OFF. Also PWM_FIXED_PERIOD_CENTER_ALIGNED range is added along with the other Autosar ranges as microcontroller specific feature.
2. Deviation from PWM139_Conf: Range of the configuration parameter PwmIndex is provided as 0 to 255, since the type of InstanceId is uint8.
3. Deviation from PWM131_Conf: According to Bugzilla 26988, the configuration parameter PwmDevErrorDetect is renamed to PwmDevErrorDetect.

10.2 Limitations

Software Configuration Limitation:

- User has to take care for sequential allocation of TOM/ATOM Channel resources for PWM Grouping.
- The port pin allocation should be done in such a way that it does not conflict with other allocations.

Pwm driver Limitation:

- The driver supports period range of up to 0xFFFFE using ATOM in autosar mode. If the period is greater than 0xFFFFE then the calculation of duty may cause an overflow which is not taken care in the driver.
- Fixed period shifted channel and Fixed Period Center Aligned channels should be present in the same TGC/AGC of the same TOM/ATOM module where the referenced Fixed Period channel is present.
- The transition from 100% to other percentages with the shift value will not be the same as the ideal case. Refer [Figure 7](#).
- In case of non-coherent update the register CMx (CM0 and CM1) are updated directly. Depending on the point of time of the update of CMx registers in relation to the actual values of CN0 and CMx, the new duty cycle is applied in the current period or the following period. The new duty cycle may jitter from update to update by a maximum of one period.
- In case of Shift = 0 for shifted channels (PwmHandleShiftByOffset = false) there is a limitation of handling leading edge notification of the channel due to a HW (GTM) issue. There is an easier workaround possible for this issue i.e. to use the referred fixed period channel to handle the notification. Refer [2.1.2.4](#) for design of shifted channels.
- The values updated for 0% and 100% duty for BC and CA step devices are different. There can be a delay of 2 ticks before reaching 0% duty in case of BC step, similarly 1 ticks delay in reaching 100% duty in some cases depending on polarity. Please refer [2.1.2.4](#)
- In case of coherent updated shifted channels where Shift is not equal to zero, when duty change is requested from a condition where shift + duty is greater than period to shift + duty less than period then the signal will remain on duty for more than a period value. Please refer [2.1.2.6](#) for more details.
- The SetOutputTogle functionality will be updated in the next cycle. That is if a request is made to move to the idle state the change will not happen immediately but after the end of the period of the current cycle where the duty is requested.
- In case Shifted channels (PwmHandleShiftByOffset = true) the changes will be happen by the end of the period of the same channel. It will not depend on the reference channel. Refer [2.1.2.7](#) for details. The limitations in [Table 4](#) are also applicable.

- In the Delnit function one cycle of incorrect duty can be expected in case of fixed period and variable period channels and two cycles of incorrect duty in case of Shifted and center aligned channels.
- For the shifted channels (PwmHandleShiftByOffset = false) in the conditions Shift + Duty exactly equal to period and if a transition is asked for SetOutputToldle function or to a duty of 100% then an incorrect duty for one cycle could be observed.
- In case when handling of shifted channels by offset is enabled (PwmHandleShiftByOffset = true) other drivers (GTM, GPT) should not use the channels in the same TGC/ AGC (A host trigger is given and it can impact the other drivers behavior). For TOM, channels 0-7 are in same TGC and 8-15 in another TGC. For ATOM 0-7 are in a single AGC.
- It is preferred that all the shifted channels using reference channel in case of PwmHandleShiftByOffset = true should be present in the same TGC. Because all the channels in the TGC are started by a Host trigger to enable all channels at the same time, Then the immediate next TGC is enabled, So in case if the shifted channel is present in next TGC it could have an impact.
- When notification for 0% or 100% is enabled by the user for fixed and variable period channels the notification for falling edge does not work for 0% duty for polarity high, similarly rising edge notification will not work 0% duty for polarity low, however the user can configure notification as both edges to get notification without worrying about the polarity.
- For fixed and variable period channels when notification is asked for falling edge for a channel whose polarity is HIGH and idle state LOW after calling SetOutputToldle function, notifications are not generated. Similarly if a channel with polarity LOW and idle state HIGH and rising edge notification is asked, notification is not generated. This is because internally SetOutputToldle function moves to 0% or 100% duty based on IDLE state and polarity.
- GTM errata GTM_AI.202 is applicable and user has to take care of this limitation. Errata is as described below.
In case of channel x has configuration of RST_CC0=1 (i.e. CN0 is reset by trigger input) and CN0 counts from 0 to MAX:
 - If CM1=0, CM0>0 -> no CCU1 interrupt is generated
 - If CM1=1, CM0=MAX+1 -> only one time a CCU1 interrupt is generated
Scope
TOM / ATOM SOMP mode.
Effects
For the described configuration no CCU1 interrupt is generated.
Workaround
Use for triggering channel y (i.e. the channel that triggers on channel x the reset of counter CN0) the configuration of CM0=MAX, CM1=1.
In case of duty cycle configuration of CM1=0 and CM0>0 on channel x use instead of CCU1 interrupt on channel x the CCU0 interrupt of triggering channel y.
In case of duty cycle configuration of CM1=1 and CM0=MAX+1 on channel x Use instead of CCU1 interrupt on channel x the CCU1 interrupt of triggering channel y.
- All the above mentioned limitations are applicable for channels which are part of Pwm_17_Gtm_SyncGrpUpdateDuties () group.
- In case of Pwm_17_Gtm_SyncGrpUpdateDuties Api user has to make sure that the API is completed before END of the period. There are chances that in some cases few registers are updated and end of the period may occur and the contents of shadow registers are updated. This may cause some channels are updated and some are not for one cycle. In order to avoid the condition, User have to take care to call the API in such a way that it's execution is completed before the END of the period
- In case of Pwm_17_Gtm_SyncGrpUpdateDuties Api when PwmHandleShiftByOffset = true please note that all shifted channels are running independently and the End of the Period is different for different

channels and update of the duty will happen by the End of the period (in coherent update) of respective channel. (This is different from the Fixed period if Shift is greater than 0). Please refer [2.1.2.7](#) for details. In this case it may not appear as synchronous update for one cycle. User has to take care of this condition.

10.3 Hardware features NOT Supported

None.

www.infineon.com